

CMPE 256 – Recommender Systems

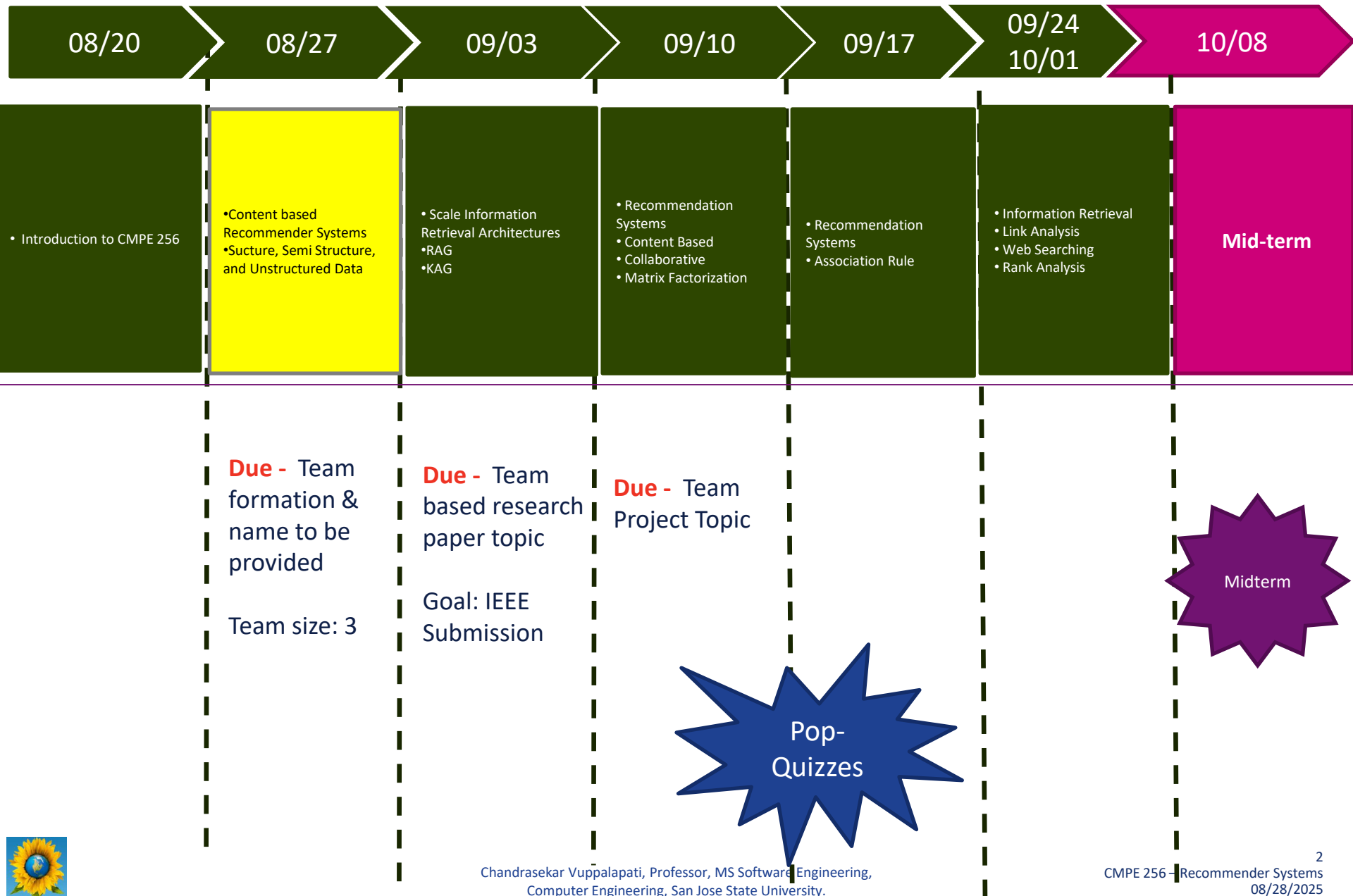


Chandrasekar Vuppalapati

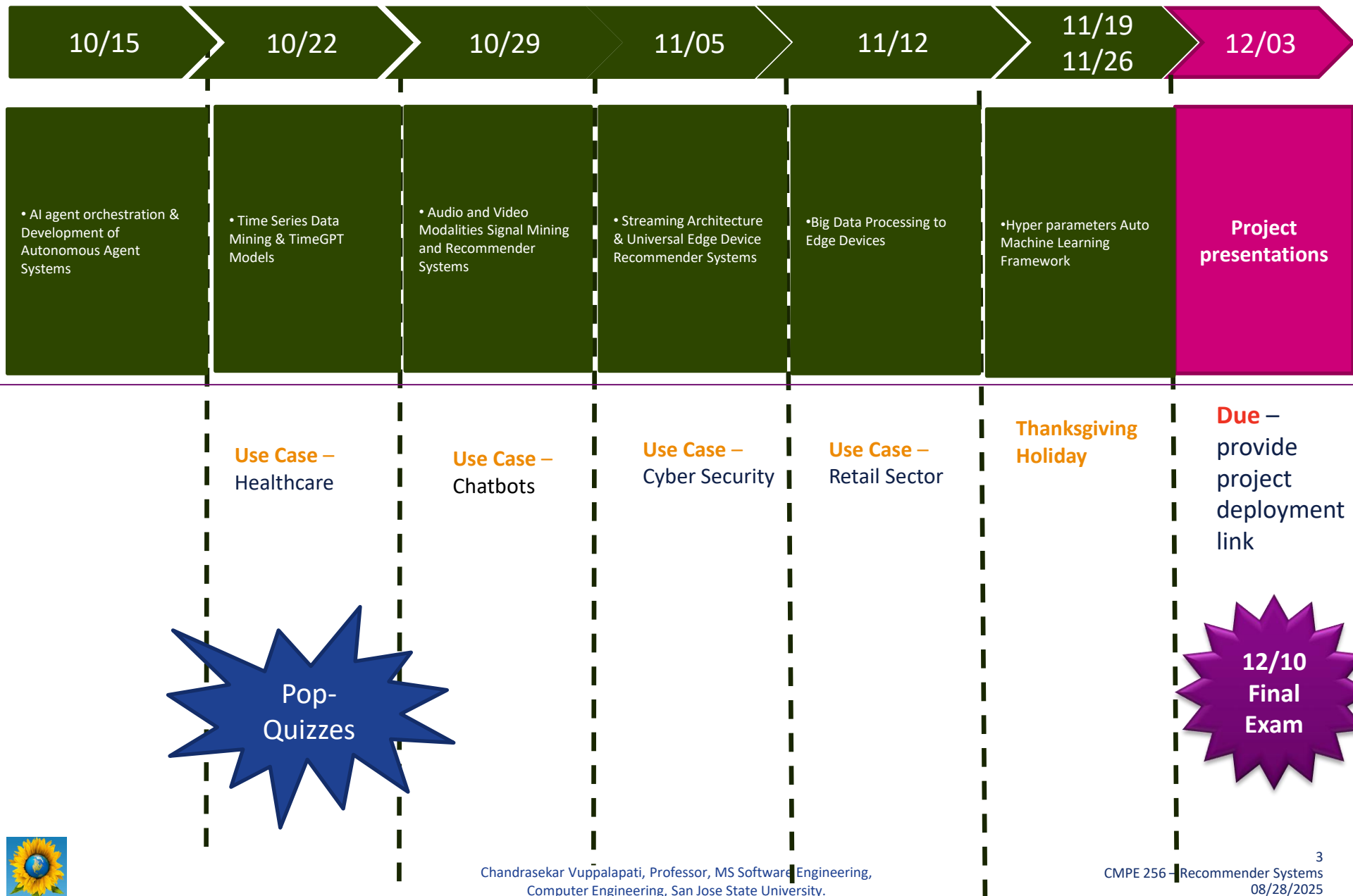
Professor, MS Software Engineering – Data Science,
Department of Computer Engineering, San Jose State
University.



Class Roadmap



Class Roadmap



$$\frac{\partial}{\partial \theta} \int_{\mathbb{R}_n} T(x) f(x, \theta) dx = \int_{\mathbb{R}_n} \frac{\partial}{\partial \theta} T(x) f(x, \theta) dx$$

$$\frac{\partial}{\partial a} \ln f_{a, \sigma^2}(\xi_1) = \frac{(\xi_1 - a)}{\sigma^2} f_{a, \sigma^2}(\xi_1) = \frac{1}{\sqrt{2\pi\sigma}} \exp\left\{-\frac{(\xi_1 - a)^2}{2\sigma^2}\right\}$$

$$\int_{\mathbb{R}_n} T(x) \cdot \frac{\partial}{\partial \theta} f(x, \theta) dx = M\left(T(\xi) \cdot \frac{\partial}{\partial \theta} \ln L(\xi, \theta)\right) = \int_{\mathbb{R}_n} \frac{\partial}{\partial \theta} T(x) f(x, \theta) dx$$

$$\int_{\mathbb{R}_n} T(x) \cdot \left(\frac{\partial}{\partial \theta} \ln L(x, \theta)\right) \cdot f(x, \theta) dx = \int_{\mathbb{R}_n} T(x) \cdot \left(\frac{\frac{\partial}{\partial \theta} f(x, \theta)}{f(x, \theta)}\right) \cdot f(x, \theta) dx$$

$$\frac{\partial}{\partial \theta} M T(\xi) = \frac{\partial}{\partial \theta} \int_{\mathbb{R}_n} T(x) f(x, \theta) dx = \int_{\mathbb{R}_n} \frac{\partial}{\partial \theta} T(x) f(x, \theta) dx$$

$$\left\{ \frac{(\xi_1 - a)^2}{\sigma^2} \right\} \frac{\partial}{\partial a} \ln f_{a, \sigma^2}(\xi_1) = \frac{(\xi_1 - a)}{\sigma^2}$$

[Resources](#)[Contact Us](#)[We're Hi](#)[Products & Services](#)[Solutions](#)[Machine Learning](#)[Customers](#)[Company](#)[Home](#) ▶ [Machine Learning](#)

MACHINE LEARNING

Why do Machine Learning on Big D

Advanced analytics (Machine Learning) is the best way to exploit the opportunity presented by Big Data.

Machine learning incorporates data analyses covering predictive analytics, data mining, pattern recognition, and multivariate statistics. Only Skytree can deliver machine learning that scales to deal with ever increasing volumes of data.

New to Machine Learning?

Our [Jump Start to Machine Learning](#) is a tailored approach that will get your organization up and running with the Advanced Analytics power of Machine Learning as quickly as possible.

[Get Started](#)

Skytree Server

MACHINE LEARNING PLATFORM

Skytree® – The Machine Learning Company® is disrupting the Advanced Analytics market with a Machine Learning platform that gives organizations the power to discover deep analytic insights, predict future trends, make recommendations and reveal untapped markets and customers. Predictive Analytics is quickly becoming a must-have technology in the age of Big Data, and Skytree is at the forefront with enterprise-grade Machine Learning. Skytree's flagship product – Skytree Server – is the only general purpose scalable Machine Learning system on the market, built for the highest accuracy at unprecedented speed and scale.

Designed from the ground up to perform machine learning on massive datasets efficiently, Skytree Server's scalable architecture performs state-of-the-art machine learning methods that were never before possible on large data sets. Skytree Server's fundamental design incorporates advanced algorithms from the machine learning and theoretical computer science research frontier to achieve speedups of up to tens of thousands of times over existing approaches in the marketplace.

WHAT CAN YOU DO WITH SKYTREE SERVER?

Predict a Category/Class (classification)

Marketing: Is this customer likely to buy or not? What product aspects make a

Predict a Value/Number (regression)

Trading: What is the most likely price tomorrow? What is the uncertainty of that

Risk management: What are the different outliers (unusual objects) among these records?

Astronomy: How likely are

Machine Learning (ML) is the modern science of discovering patterns and making predictions from complex data.

AT-A-GLANCE

The first general purpose machine learning/ advanced analytics platform, which can perform machine learning on massive datasets at high speed.

KEY FEATURES

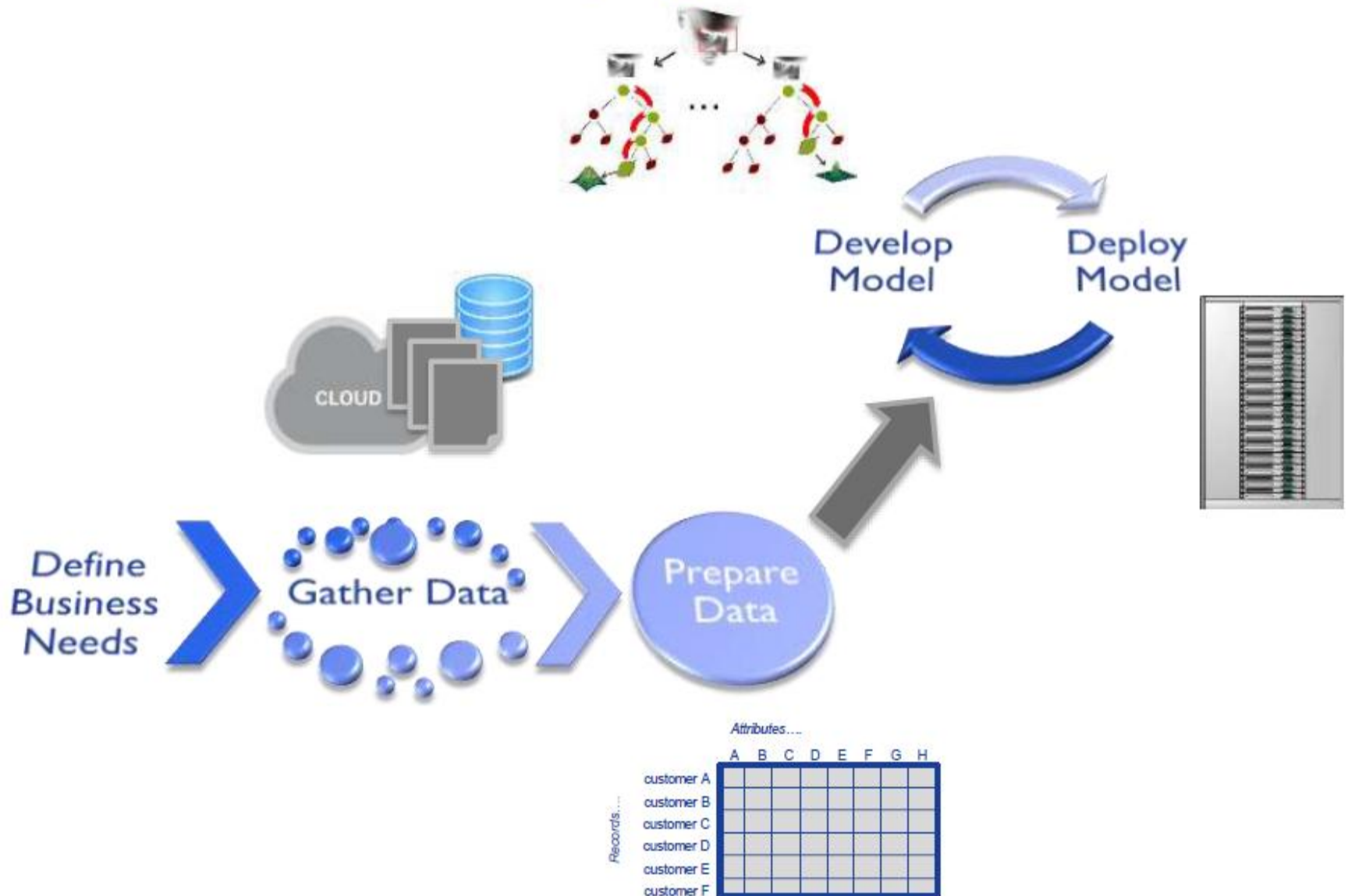
- Fast, Efficient Algorithms
- High Scalability
- State-of-the-Art Machine Learning (ML) Methodology

MACHINE LEARNING METHODS

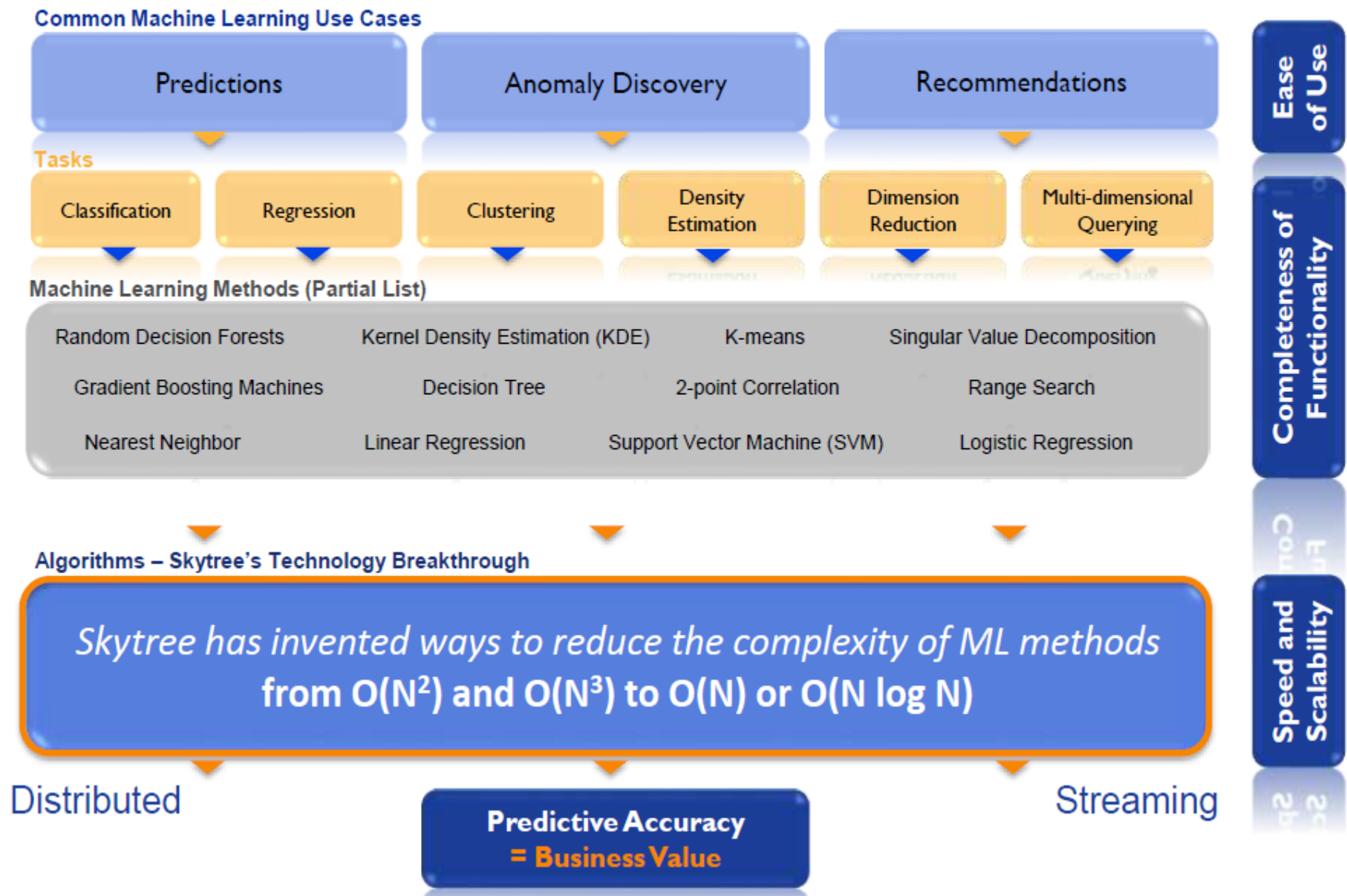
- Nearest Neighbor
- Collaborative Filtering
- Support Vector Machines

What is Machine Learning?

Process: From idea to deployment



Skytree's product: High-performance ML software



What is Machine Learning? A Marketing Example

SKYTREE
THE MACHINE LEARNING COMPANY

Basic Analytics

“Human Driven”

1 spreadsheet (human-defined equations)
“business intelligence” (human-defined plots)
database (human-defined queries)

2 “Compute the avg income for each age”
“Plot age vs. income”
“Find all people w/ income > 65K and age < 45”

3 Target some people with some offers =
\$

Advanced Analytics

“Machine Learning”

1 **discover** the subtle patterns underneath
predict based on those patterns
Identify the unusual transactions /behaviors

2 “Given examples of buys & non-buys,
score each potential customer or product”

3 Target predicted buyers with high-likelihood offers =
\$\$\$\$\$\$\$\$\$\$

Generally the performance of human-defined rules is not comparable to machine-optimized; this is **transformational**.

How Cisco Defines the IoT

The Internet of Everything builds upon the IoT to connect people, data, and more. (2:00 min)

Watch Now

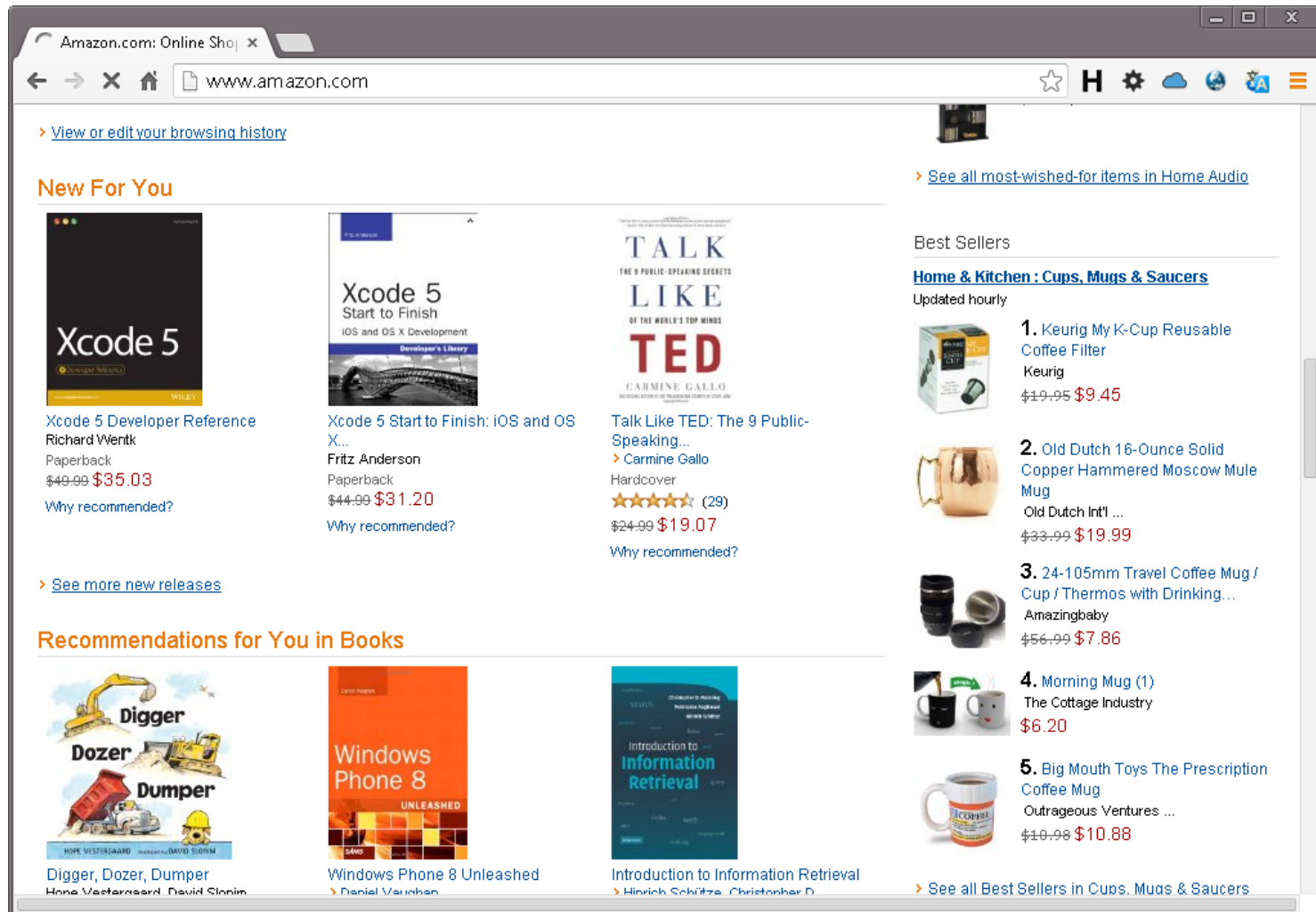


<http://www.cisco.com/web/solutions/trends/iot/overview.html>

Recommendation Systems

Recommendation Systems

There is an extensive class of Web applications that involve predicting user responses to options. Such a facility is called a recommendation system.



The screenshot displays the Amazon.com homepage with several recommendation sections:

- New For You:** Features three books: "Xcode 5 Developer Reference" by Richard Wentk (Paperback, \$49.99 to \$35.03), "Xcode 5 Start to Finish: iOS and OS X Development" by Fritz Anderson (Paperback, \$44.99 to \$31.20), and "Talk Like TED: The 9 Public-Speaking Secrets of the World's Top Minds" by Carmine Gallo (Hardcover, \$24.99 to \$19.07).
- Recommendations for You in Books:** Features three books: "Digger, Dozer, Dumper" by Hope Vestergaard and David Slonim, "Windows Phone 8 Unleashed" by Daniel Vaughan, and "Introduction to Information Retrieval" by Hinrich Schütze and Christopher D.
- Best Sellers:** Features a list of mugs and saucers under the heading "Home & Kitchen: Cups, Mugs & Saucers". The list includes: 1. Keurig My K-Cup Reusable Coffee Filter (Keurig, \$19.95 to \$9.45), 2. Old Dutch 16-Ounce Solid Copper Hammered Moscow Mule Mug (Old Dutch Int'l, \$33.99 to \$19.99), 3. 24-105mm Travel Coffee Mug / Cup / Thermos with Drinking... (Amazingbaby, \$56.99 to \$7.86), 4. Morning Mug (1) The Cottage Industry (\$6.20), and 5. Big Mouth Toys The Prescription Coffee Mug (Outrageous Ventures, \$10.99 to \$10.88).



Recommendation Systems

Recommendation systems use a number of different technologies. We can classify these systems into two broad groups.

Content based Systems:

Content-based systems examine properties of the items recommended. For instance, if a Netflix user has watched many cowboy movies, then recommend a movie classified in the database as having the “cowboy” genre.

Collaborative filtering Systems:

Collaborative filtering systems recommend items based on similarity measures between users and/or items. The items recommended to a user are those preferred by similar users.



Content Based Systems

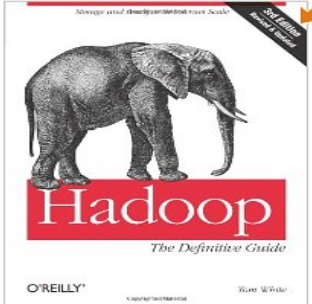
Amazon.com: Hadoop: The Definitive Guide by Tom White

Instant Order Update for ChandraSekar. You purchased this item on August 12, 2013.

Browse Programming Books C | Java | PHP | Python

Hadoop: The Definitive Guide and thousands of other textbooks are available for instant download on your Kindle Fire tablet or on

Click to LOOK INSIDE!



Hadoop: The Definitive Guide [Paperback]
Tom White (Author)
★★★★☆ (43 customer reviews)

List Price: ~~\$49.99~~
Price: **\$28.99** Prime
You Save: **\$21.00 (42%)**

In Stock.
Ships from and sold by Amazon.com. Gift-wrap available.

Want it tomorrow, March 19? Order within **5 hrs 21 mins** and choose **One-Day Shipping** at checkout. [Details](#)

58 new from \$27.48 **33 used** from \$20.00

Get \$10 For Each Friend Who Joins Amazon Student [Learn more >](#)

Formats	Amazon Price	New from	Used from
Kindle Edition	\$24.79	--	--
Rent from	\$12.00		
Paperback	\$28.99 Prime	\$27.48	\$20.00

Click to open expanded view

Share your own customer images

+ Add to Collection

properties of
the items
recommended

Frequently Bought Together



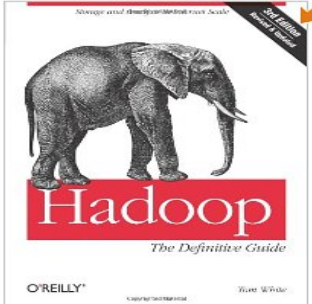
Collaborative filtering Systems:

Instant Order Update for ChandraSekar. You purchased this item on August 12, 2013.

Browse Programming Books C | Java | PHP | Python

Hadoop: The Definitive Guide and thousands of other textbooks are available for instant download on your Kindle Fire tablet or on

Click to LOOK INSIDE!



Hadoop: The Definitive Guide [Paperback]
Tom White (Author)
★★★★☆ (43 customer reviews)

List Price: ~~\$49.99~~
Price: **\$28.99** Prime
You Save: **\$21.00 (42%)**

In Stock.
Ships from and sold by Amazon.com. Gift-wrap available.

Want it tomorrow, March 19? Order within **5 hrs 21 mins** and choose **One-Day Shipping** at checkout. [Details](#)

58 new from \$27.48 **33 used** from \$20.00

Get \$10 For Each Friend Who Joins Amazon Student [Learn more >](#)

Formats	Amazon Price	New from	Used from
Kindle Edition	\$24.79	--	--
Rent from	\$12.00		
Paperback	\$28.99 Prime	\$27.48	\$20.00

Click to open expanded view

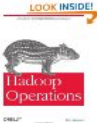
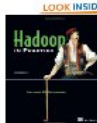
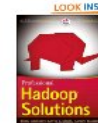
[Share your own customer images](#)

[Add to Collection](#)

Search inside another edition of this book

similarity
measures
between users
and/or items

Customers Who Bought This Item Also Bought

 Hadoop Operations Eric Sammer ★★★★★ (15) Paperback \$33.45 Prime	 Programming Hive Edward Capriolo ★★★★★ (8) Paperback \$29.98 Prime	 HBase: The Definitive Guide Lars George ★★★★★ (9) Paperback \$29.99 Prime	 Hadoop in Practice Alex Holmes ★★★★★ (12) Paperback \$34.28 Prime	 Professional Hadoop Solutions Boris Lublinsky ★★★★★ (5) Paperback \$35.09 Prime	 Programming Pig Alan Gates ★★★★★ (8) Paperback \$29.99 Prime	 Hadoop in Action Chuck Lam ★★★★★ (11) Paperback \$30.28 Prime
---	---	--	---	--	---	--

The Utility Matrix

In a recommendation-system application there are two classes of entities, which we shall refer to as **users** and **items**.

Users have preferences for certain items, and these preferences must be teased out of the data.

The data itself is represented as a utility matrix, giving for each user-item pair, a value that represents what is known about the degree of preference of that user for that item. Values come from an ordered set, e.g., integers 1–5 representing the number of stars that the user gave as a rating for that item. We assume that the matrix is sparse, meaning that most entries are “unknown.” An unknown rating implies that we have no explicit information about the user’s preference for the item.

	Harry Potter I	Harry Potter II	Harry Potter III	Twilight	Star Wars 1	Star Wars 2	Star Wars 3
Sriya	4			5	1		
Shruti	5	5	4				
John				2	4	5	
Tracy		3					3

The goal of a recommendation system is to predict the blanks in the utility matrix.

Scale 1-5

The Utility Matrix

We should also be aware of a slightly different goal that makes sense in many applications. It is not necessary to predict every blank entry in a utility matrix. Rather, it is only necessary to discover some entries in each row that are likely to be high. In most applications, the recommendation system does not offer users a ranking of all items, but rather suggests a few that the user should value highly. It may not even be necessary to find all items with the highest expected ratings, but only to find a large subset of those with the highest ratings.



THINK

BIG

BIG

The Long Tail

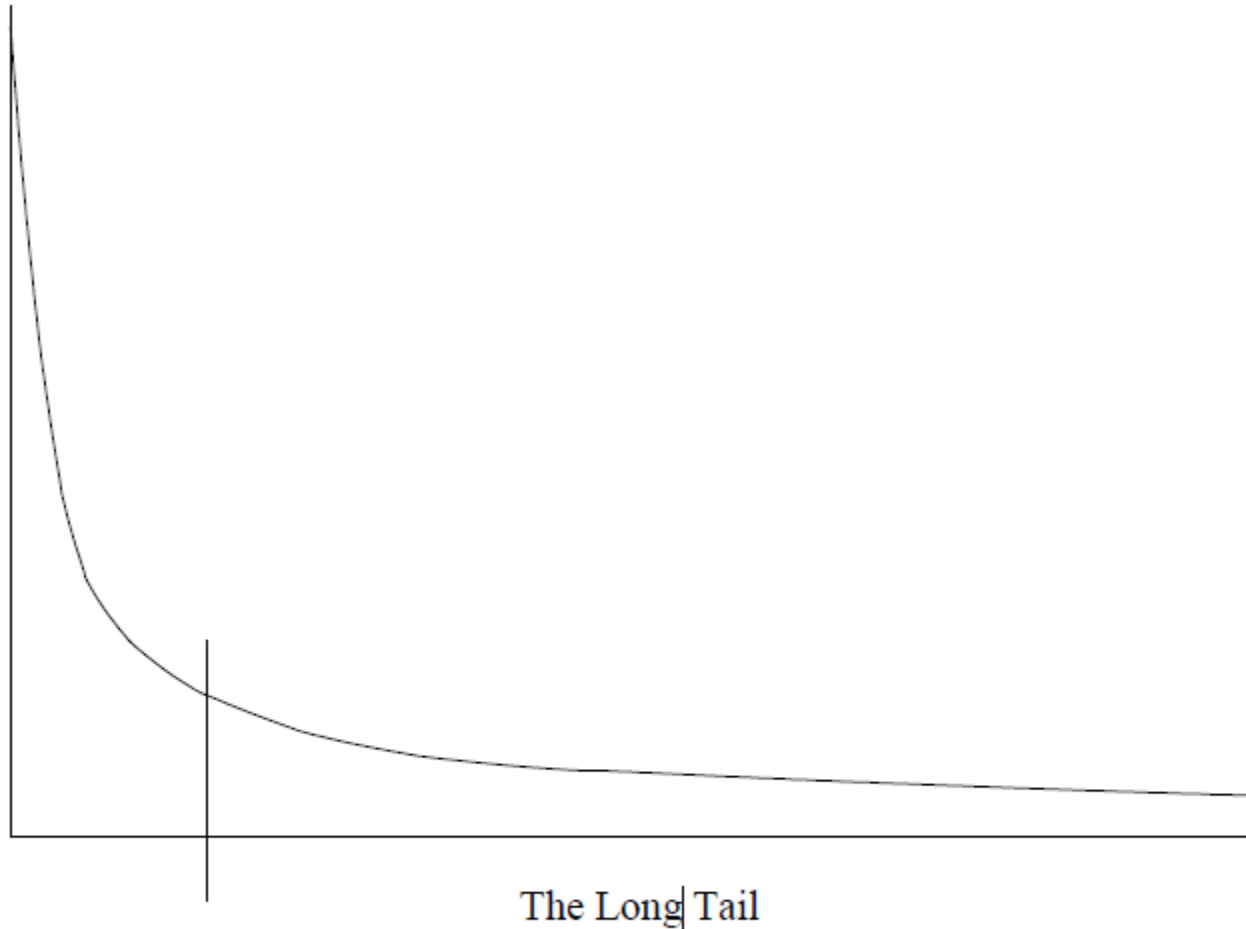
Physical delivery systems are characterized by a scarcity of resources. Brick-and-mortar stores have **limited shelf space, and can show the customer only a small fraction of all the choices that exist**. On the other hand, on-line stores can make anything that exists available to the customer. Thus, a physical bookstore may have several thousand books on its shelves, but Amazon offers millions of books. A physical newspaper can print several dozen articles per day, while on-line news services offer thousands per day.

Recommendation in the physical world is fairly simple. First, it is not possible to tailor the store to each individual customer. Thus, the choice of what is made available is governed only by the aggregate numbers. Typically, a bookstore will display only the books that are most popular, and a newspaper will print only the articles it believes the most people will be interested in. In the first case, sales figures govern the choices, in the second case, editorial judgment serves.

The distinction between the physical and on-line worlds has been called the long tail phenomenon. The vertical axis represents popularity (the number of times an item is chosen). The items are ordered on the horizontal axis according to their popularity. Physical institutions provide only the most popular items to the left of the vertical line, while the corresponding on-line institutions provide the entire range of items: the tail as well as the popular items. **The long-tail phenomenon forces on-line institutions to recommend items to individual users.** It is not possible to present all available items to the user, the way physical institutions can. Neither can we expect users to have heard of each of the items they might like.



The Long Tail

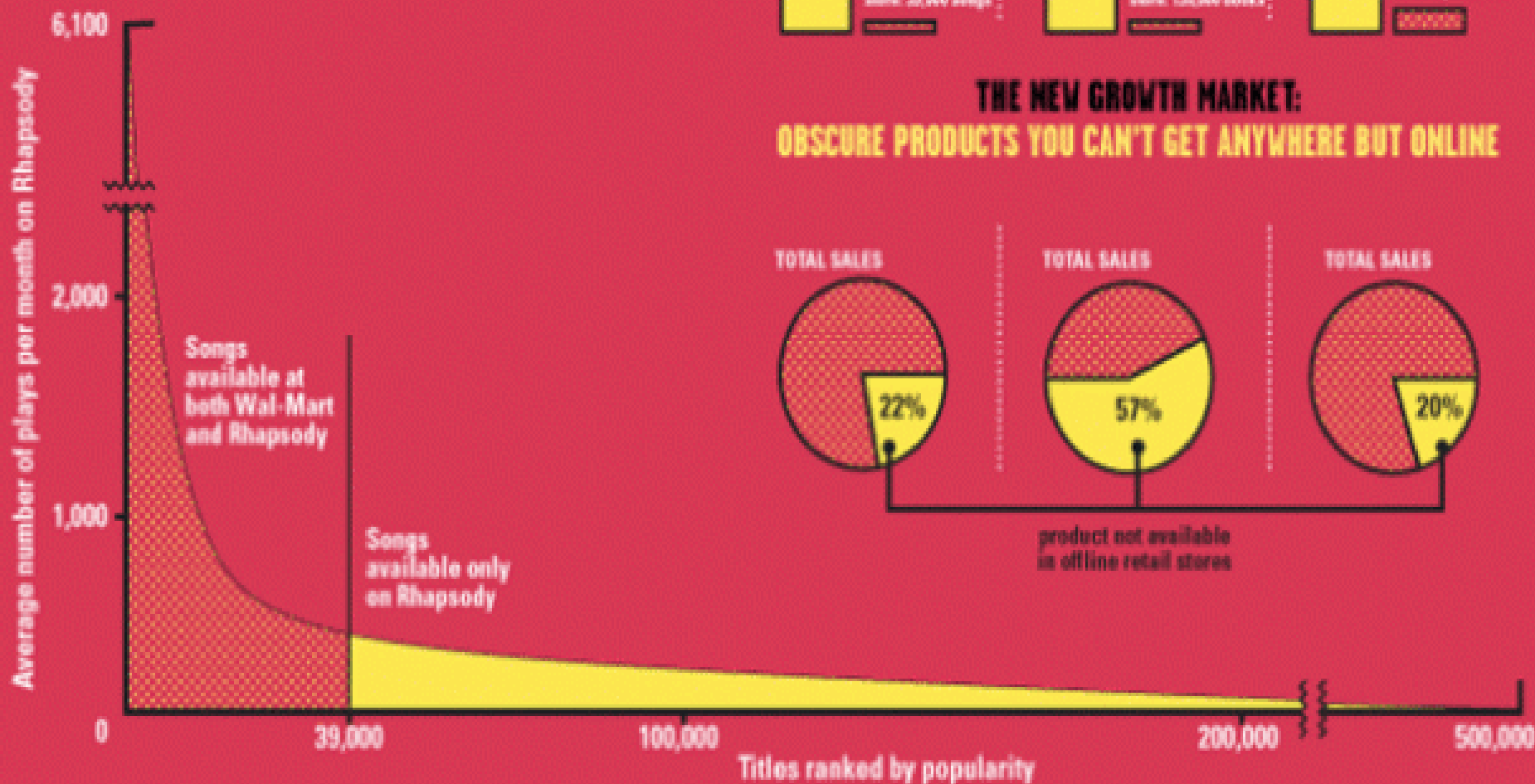


The long tail: physical institutions can only provide what is popular, while on-line institutions can make everything available



ANATOMY OF THE LONG TAIL

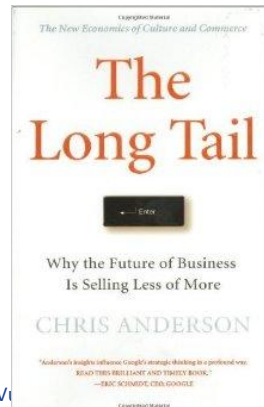
Online services carry far more inventory than traditional retailers. Rhapsody, for example, offers 19 times as many songs as Wal-Mart's stock of 39,000 tunes. The appetite for Rhapsody's more obscure tunes (charted below in yellow) makes up the so-called Long Tail. Meanwhile, even as consumers flock to mainstream books, music, and films (right), there is real demand for niche fare found only online.



The Long Tail

The Long Tail? What has that got to do with digital marketing, I hear you ask? Well, everything, actually. The concept of the Long Tail is one which Chris Anderson, in his book entitled “The Long Tail – Why the Future of Business is Selling Less of More”. What a great title....and what a brilliantly accurate description for what really amounts to a retail strategy tailor made for the world of ecommerce. This strategy allows the retailer to drill right down through a broad retail market, identifying a tiny, niche market, thereby allowing them to target a very specific customer, with a very specific product that is aimed at that market only.

The perfect example of “The Long Tail” is Amazon.com; so, I want to buy a book....but I don’t want to go to a physical bookstore where (i) I don’t know for sure that the shop in question will stock the book I am looking for and (ii) I don’t want to have to spend time browsing for it. Luckily for me, (and everyone else), Amazon recognises that within the broad “book market” there are hundreds of different micro markets – crime, fiction, sport, lifestyle, politics, self help....the list is endless. So what? What’s your point, Alan!



The Long Tail

Well, the point is this; each of the micro book markets, referred to above, targets only people with very specific interests; in other words, the author of a sports book is not seeking to appeal to everyone that buys books, but only those that are seeking out sports books. The benefit of this scenario, to the author and seller, is that although they are operating within a greatly reduced market (the sports category represents just a fraction of all book categories), they are appealing to a very large audience within this specific interest category....by reaching exactly the desired market, sales will increase exponentially.

The Long Tail applies to almost every market....go to www.ebay.com, www.amazon.com, www.netflix.com and other mass auction sites and check out the incredible number of specific product categories and micro categories...and see for yourself, the Long Tail in action!



Source: <http://whichwords.co.uk/?m=201209>



A
Smarter
Planet



What is a Smarter Planet?



The Internet
of Things

http://www.ibm.com/smarterplanet/us/en/overview/article/iot_video.html

Application of Recommendation Systems

Application of Recommendation Systems

Product Recommendation Systems: Perhaps the most important use of recommendation systems is at on-line retailers. We have noted how Amazon or similar on-line vendors strive to present each returning user with some suggestions of products that they might like to buy. These suggestions are not random, but are based on the purchasing decisions made by similar customers.

The screenshot shows the Amazon.com website with a browser window. The address bar displays "www.amazon.com". Below the navigation bar, there are two sections of product recommendations:

- Related to Items You've Viewed**: This section is divided into two columns. The first column, "You viewed", shows a book titled "Enterprise Architecture Using the..." by Carol O'Rourke, Neal Fishman, and Warren... with a 4.5-star rating (17 reviews) and a price of \$105.00. The second column, "Customers who viewed this also viewed", shows several other books related to Enterprise Architecture, including "Enterprise Architecture Planning...", "Enterprise Architecture As Strategy...", "Enterprise Architecture at Work...", "Data Stores, Data Warehousing, and...", "FEAC Certified Enterprise Architect...", and "An Introduction To Enterprise...". Each book listing includes its title, authors, format (Paperback or Hardcover), star rating, number of reviews, and price.
- More Items to Consider**: This section also has two columns. The first column, "You viewed", shows a book titled "plain & simple Windows Phone 7". The second column, "Customers who viewed this also viewed", shows various Windows-related products, including "Windows 7 Plain & Simple", "HTC BX Windows 8 AT&T", "Windows Phone 7 For Dummies", "Windows 8 Quick Reference", "Windows Phone 7 Secrets", and "Professional Windows Phone".

At the bottom left of the screenshot, there is a small icon of a sun with a face.



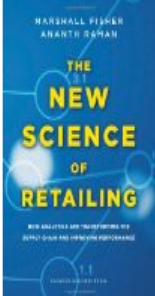
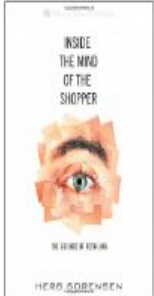
Application of Recommendation Systems

Amazon.com: Online Shopping | www.amazon.com

Paperback \$40.00 \$28.07 Why recommended?	Paperback \$44.00 \$31.20 Why recommended?	Paperback \$39.00 \$31.99 Why recommended?	Paperback \$44.00 \$31.20 Why recommended?	Paperback \$49.99 \$35.03 Why recommended?	Hardcover ★★★★★ (29) \$24.00 \$19.07 Why recommended?	Hardcover ★★★★★ (2) \$16.00 \$10.93 Why recommended?
---	---	---	---	---	---	--

> [See more new releases](#)

Recommendations for You in Books

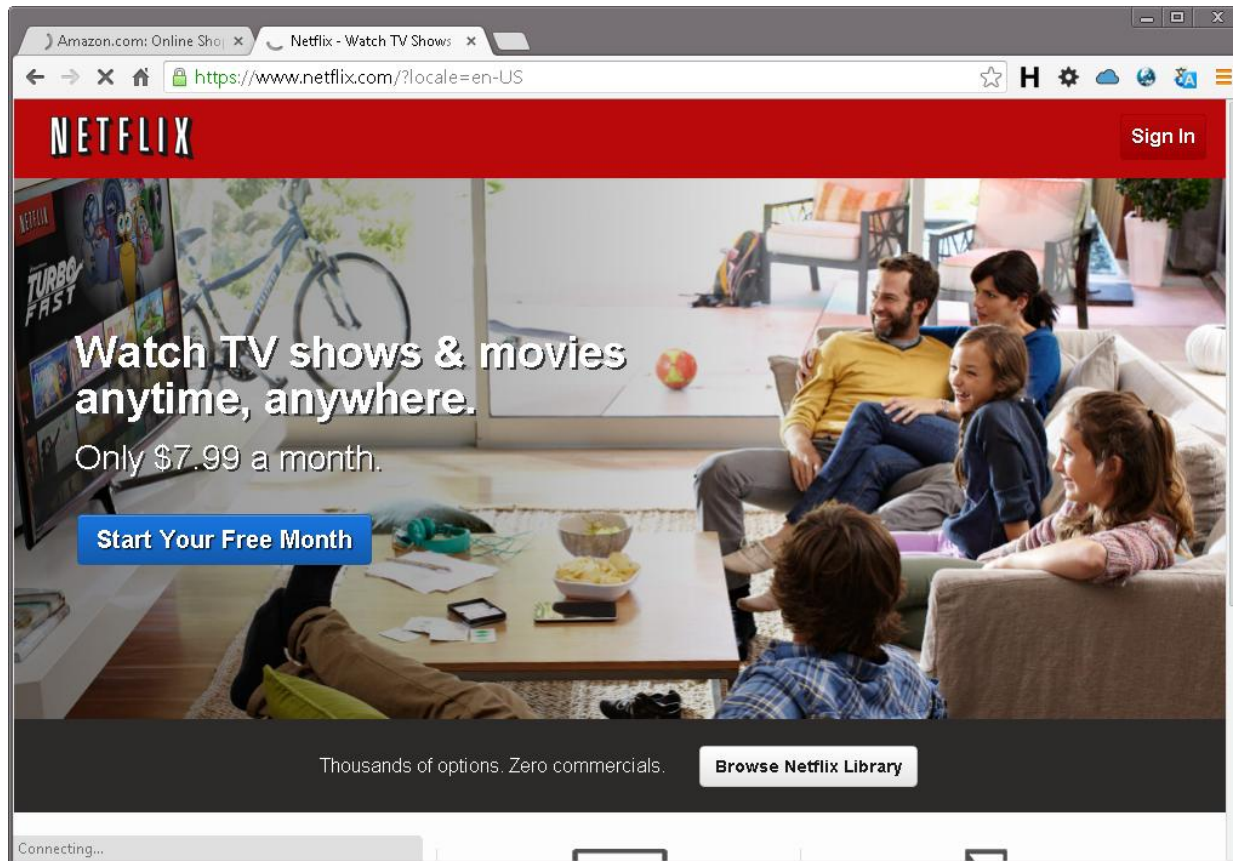
 Windows Phone 8 Unleashed > Daniel Vaughan Paperback ★★★★★ (7) \$59.00 \$38.42 Why recommended?	 The New Science of Retailing: How... Marshall Fisher, Ananth Raman Hardcover ★★★★★ (8) \$35.00 \$23.15 Why recommended?	 Essential Windows Phone 8 > Shawn Wildermuth Paperback ★★★★★ (7) \$44.00 \$28.54 Why recommended?	 The New Rules of Retail: Competing in... > Robin Lewis, Michael Dart Hardcover ★★★★★ (18) \$26.00 \$18.56 Why recommended?	 Programming Windows: Writing Windows... Charles Petzold Paperback ★★★★★ (11) \$59.00 \$42.44 Why recommended?	 Inside the Mind of the Shopper: The... > Herb Sorensen Hardcover ★★★★★ (19) \$36.00 \$22.67 Why recommended?	 Saxon Math Intermediate 5 SAXON PUBLISHERS Hardcover ★★★★★ (8) \$74.40 \$53.34 Why recommended?
---	---	--	--	---	---	---

> [See more recommendations](#)



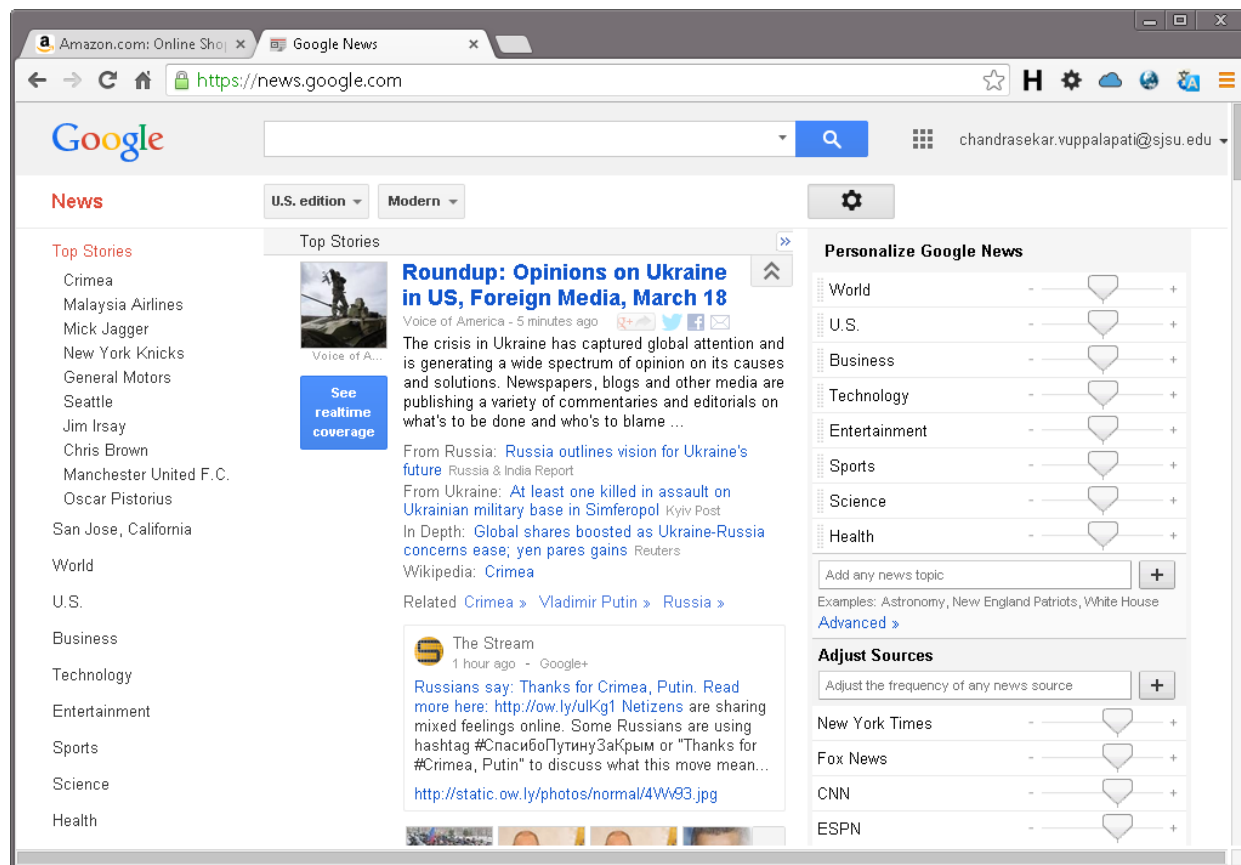
Application of Recommendation Systems

Movie Recommendations: Netflix offers its customers recommendations of movies they might like. These recommendations are based on ratings provided by users. The importance of predicting ratings accurately is so high, that Netflix offered a prize of one million dollars for the first algorithm that could beat its own recommendation system by 10%.



Application of Recommendation Systems

News Articles: News services have attempted to identify articles of interest to readers, based on the articles that they have read in the past. The similarity might be based on the similarity of important words in the documents, or on the articles that are read by people with similar reading tastes. The same principles apply to recommending blogs from among the millions of blogs available, videos on YouTube, or other sites where content is provided regularly.



Content Based Recommendation System

Content-Based systems focus on properties of items. Similarity of items is determined by measuring the similarity in their properties.

Item Profiles

In a content-based system, we must construct for each item a profile, which is a record or collection of records representing important characteristics of that item. In simple cases, the profile consists of some characteristics of the item that are easily discovered.

For example, consider the features of a movie that might be relevant to a recommendation system.

1. The set of [actors](#) of the movie. Some viewers prefer movies with their favorite actors.
2. [The director](#). Some viewers have a preference for the work of certain directors.
3. The [year in which](#) the movie was made. Some viewers prefer old movies; others watch only the latest releases.
4. The [genre or general type](#) of movie. Some viewers like only comedies, others dramas or romances.



Item Profile (In-class assignment)

Developing Recommendation Systems for Books - identify Key item attributes...



Source: J. Ullman and A. Rajaraman for "Mining Massive Datasets" Chandrasekar Vuppapapati, Professor, MS Software Engineering, Computer Engineering, San Jose State University.

Content Based Recommendation System

Content-Based systems focus on properties of items. Similarity of items is determined by measuring the similarity in their properties.

Item Profiles

In a content-based system, we must construct for each item a profile, which is a record or collection of records representing important characteristics of that item. In simple cases, the profile consists of some characteristics of the item that are easily discovered.

For example, consider the features of a movie that might be relevant to a recommendation system.

1. The set of actors of the movie. Some viewers prefer movies with their favorite actors.
2. The director. Some viewers have a preference for the work of certain directors.
3. The year in which the movie was made. Some viewers prefer old movies; others watch only the latest releases.
4. The genre or general type of movie. Some viewers like only comedies, others dramas or romances.



\$1 A WEEK FOR 12 WEEKS

Sponsored by:  Microsoft Cloud

Rebellion

Disney, Shareholders Reach Deal

4 to Annex Crimea



5 Food Prices Surge as Drought Exacts a Toll



Show 5 More

Top Picks



Disney Reveals New 'Star Wars' Details, 'Incredibles 2'



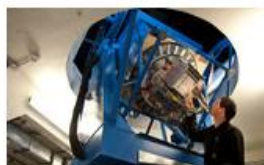
McConnell Campaigns on Clout



SXSW Has Lost Its Balance



Plan to Let Bison Roam Farther Irks Ranchers



Discovery Bolsters Big-Bang Theory



A Financial Survival Strategy for the Sandwich Generation

Journal Report: Big Issues >

Should You Delay Taking Social Security Until 70?

Two financial advisers debate at what age older adults should start collecting benefits.

• Should Couples Keep Their Financial Assets Separate?



Stupid Reasons People Give For Borrowing From 401(k)s

Only withdraw from your account in a life or death emergency, says this financial adviser.

- One of the Worst Financial Moves You Can Make
- The Sandwich Generation Needs a Financial Plan
- Do You Know Who You Are Marrying (Financially)?

The Experts



Sponsor:  T.RowePrice

Video >



WORLD 03/18/2014

1:22

Scientists Discover Evidence of Big Bang

 Tweet 7

 Like 3



WORLD

Scientists Discover Evidence of Big Bang

1:22

WSJ.D

Google Beats Apple to Market in Smartwatches

Google looks set to beat Apple to market in a new category of mobile devices -- smartwatches.



Can Sonos Be Google for Your Music?

Sonos Hires an Xbox Executive

Oracle Reports Slight Rise in Profit

U.S. >

Prosecutors Subpoena Port Authority for Records on Chairman

Federal prosecutors in New Jersey subpoenaed the Port Authority of New York and New Jersey this week, seeking records related to



WSJ
Live

SCHEDULE

WHAT'S NEWS

Click For Sound



Disney Reveals New 'Star Wars' Details, 'Incredibles 2'

[In Your Spring, Adventure Inspirations](#)[Lost Writers De Theories About Series 10 Years](#)

ARTICLE

COMMENTS (1)

[CARS](#) [DISNEY](#) [FROZEN](#) [PIRATES OF THE CARIBBEAN](#) [ROBERT IGER](#) [THE INCREDIBLES](#) Recommend 154 Email  Print    By BEN FRITZ 

Scene from "The Incredibles" (2004). —Walt Disney/courtesy Everett Collection

Disney has a third "Cars" and a second "The Incredibles" movie in the works, but won't be releasing a fifth "Pirates of the Caribbean" anytime soon.

At Walt Disney Co.'s annual shareholders meeting held in Portland, Ore. Tuesday, chief executive **Robert Iger** confirmed the two sequels from his company's **Pixar Animation Studios** are in the works and will be

released sometime after 2015.

Asked about another sequel to the hit "Pirates" franchise starring Johnny Depp, Mr. Iger said that another is "in development," but "not officially greenlit." "Pirates of the Caribbean 5" had been scheduled for release in the summer of 2015, but Disney

Pay less to power your electric car



Discovering Features of a Document

There are other classes of items where it is not immediately apparent what the values of features should be. We shall consider two of them: document collections and images. Documents present special problems, and we shall discuss the technology for extracting features from documents in this section. There are many kinds of documents for which a recommendation system can be useful. For example, there are many news articles published each day, and we cannot read all of them. A recommendation system can suggest articles on topics a user is interested in, but how can we distinguish among topics? Web pages are also a collection of documents. Can we suggest pages a user might want to see? Likewise, blogs could be recommended to interested users, if we could classify blogs by topics.

Unfortunately, these classes of documents do not tend to have readily available information giving features. A substitute that has been useful in practice is the identification of words that characterize the topic of a document.

How we do the identification:

First, eliminate stop words – the several hundred most common words, which tend to say little about the topic of a document. For the remaining words, compute the TF.IDF score for each word in the document. The ones with the highest scores are the words that characterize the document.



Discovering Features of a Document

We may then take as the features of a document the n words with the highest TF.IDF scores. It is possible to pick n to be the same for all documents, or to let n be a fixed percentage of the words in the document. We could also choose to make all words whose TF.IDF scores are above a given threshold to be a part of the feature set.

Now, documents are represented by sets of words. Intuitively, we expect these words to express the subjects or main ideas of the document. For example, in a news article, we would expect the words with the highest TF.IDF score to include the names of people discussed in the article, unusual properties of the event described, and the location of the event. To measure the similarity of two documents, there are several natural distance measures we can use:

1. We could use the Jaccard distance between the sets of words
2. We could use the cosine distance between the sets, treated as vectors.



Discovering Features of a Document

Application of NLTK (Natural Language Toolkit)

- Load the raw text.
- Split into tokens.
- Convert to lowercase.
- Remove punctuation from each token.
- Filter out remaining tokens that are not alphabetic.
- Filter out tokens that are stop words.



NLTK 3.4.5 documentation

[PREVIOUS](#) | [NEXT](#) | [MODULES](#) | [INDEX](#)

Installing NLTK Data

NLTK comes with many corpora, toy grammars, trained models, etc. A complete list is posted at: http://nltk.org/nltk_data/

To install the data, first install NLTK (see <http://nltk.org/install.html>), then use NLTK's data downloader as described below.

Apart from individual data packages, you can download the entire collection (using "all"), or just the data required for the examples and exercises in the book (using "book"), or just the corpora and no grammars or trained models (using "all-corpora").

Interactive installer

For central installation on a multi-user machine, do the following from an administrator account.

Run the Python interpreter and type the commands:

```
>>> import nltk
```

TABLE OF CONTENTS

[NLTK News](#)[Installing NLTK](#)[Installing NLTK Data](#)[Contribute to NLTK](#)[FAQ](#)[Wiki](#)[API](#)[HOWTO](#)

SEARCH

<https://www.nltk.org/data.html>

Chandrasekar Vuppulapati, Professor, MS Software Engineering,
Computer Engineering, San Jose State University.



Source: J. Ullman and A. Rajaraman for "Mining Massive Datasets"

Discovering Features of a Document

```
python -m nltk.downloader all
```

```
Anaconda Prompt - python -m nltk.downloader all

(base) C:\Hanumayamma\Clustering>python -m nltk.downloader all
C:\Program Files (x86)\Microsoft Visual Studio\Shared\Anaconda3_64\lib\runpy.py:125: RuntimeWarning: 'nltk.downloader' found
in sys.modules after import of package 'nltk', but prior to execution of 'nltk.downloader'; this may result in unpredictable
behaviour
  warn(RuntimeWarning(msg))
[nltk_data] Downloading collection 'all'
[nltk_data]
[nltk_data] | Downloading package abc to
[nltk_data] | C:\Users\chvuppall\AppData\Roaming\nltk_data...
[nltk_data] | Unzipping corpora\abc.zip.
[nltk_data] | Downloading package alpino to
[nltk_data] | C:\Users\chvuppall\AppData\Roaming\nltk_data...
[nltk_data] | Unzipping corpora\alpino.zip.
```



Discovering Features of a Document

```
python -m nltk.downloader all
```

```
Anaconda Prompt - python -m nltk.downloader all

(base) C:\Hanumayamma\Clustering>python -m nltk.downloader all
C:\Program Files (x86)\Microsoft Visual Studio\Shared\Anaconda3_64\lib\runpy.py:125: RuntimeWarning: 'nltk.downloader' found in sys.modules after import of package 'nltk', but prior to execution of 'nltk.downloader'; this may result in unpredictable behaviour
  warn(RuntimeWarning(msg))
[nltk_data] Downloading collection 'all'
[nltk_data]
[nltk_data]   Downloading package abc to
[nltk_data]   C:\Users\chvuppall\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping corpora\abc.zip.
[nltk_data]   Downloading package alpino to
[nltk_data]   C:\Users\chvuppall\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping corpora\alpino.zip.
```

```
import nltk
nltk.download('punkt')
```

```
import nltk
nltk.download('averaged_perceptron_tagger')
```



Source:

J. Ullman and A. Rajamaram for "Mining Massive Datasets" Computer Engineering, San Jose State University.

Chandrasekar Vuppalapati, Professor, MS Software Engineering,

Discovering Features of a Document

```
python -m nltk.downloader all
```

```
Anaconda Prompt

[nltk_data] Unzipping corpora\mte_teip5.zip.
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] C:\Users\chvuppap\AppData\Roaming\nltk_data...
[nltk_data] Unzipping taggers\averaged_perceptron_tagger.zip.
[nltk_data] Downloading package averaged_perceptron_tagger_ru to
[nltk_data] C:\Users\chvuppap\AppData\Roaming\nltk_data...
[nltk_data] Unzipping
[nltk_data] taggers\averaged_perceptron_tagger_ru.zip.
[nltk_data] Downloading package perluniprops to
[nltk_data] C:\Users\chvuppap\AppData\Roaming\nltk_data...
[nltk_data] Unzipping misc\perluniprops.zip.
[nltk_data] Downloading package nonbreaking_prefixes to
[nltk_data] C:\Users\chvuppap\AppData\Roaming\nltk_data...
[nltk_data] Unzipping corpora\nonbreaking_prefixes.zip.
[nltk_data] Downloading package vader_lexicon to
[nltk_data] C:\Users\chvuppap\AppData\Roaming\nltk_data...
[nltk_data] Downloading package porter_test to
[nltk_data] C:\Users\chvuppap\AppData\Roaming\nltk_data...
[nltk_data] Unzipping stemmers\porter_test.zip.
[nltk_data] Downloading package wmt15_eval to
[nltk_data] C:\Users\chvuppap\AppData\Roaming\nltk_data...
[nltk_data] Unzipping models\wmt15_eval.zip.
[nltk_data] Downloading package mwa_ppdb to
[nltk_data] C:\Users\chvuppap\AppData\Roaming\nltk_data...
[nltk_data] Unzipping misc\mwa_ppdb.zip.
[nltk_data] Done downloading collection all

(base) C:\Hanumayamma\Clustering>
(base) C:\Hanumayamma\Clustering>
```



Discovering Features of a Document

```
# -*- coding: utf-8 -*-  
''''
```

Created on Mon Aug 20 14:29:52 2018

```
@author: cvuppalapati  
''''
```

```
# https://machinelearningmastery.com/clean-text-machine-learning-python/  
#import nltk and download  
import nltk  
#nltk.download()  
# load text  
filename = 'textProc.txt'  
file = open(filename, 'rt')  
text = file.read()  
file.close()  
# split into words by white space  
# split into sentences  
from nltk import sent_tokenize  
sentences = sent_tokenize(text)
```



Source: J. Ullman and A. Rajamaran for "Mining Massive Datasets" Computer Engineering, San Jose State University.

Chandrasekar Vuppalapati, Professor, MS Software Engineering,

Discovering Features of a Document

```
print("\n sentences \n")
print(sentences)
# remove punctuation from each word

# split into words
from nltk.tokenize import word_tokenize
tokens = word_tokenize(text)
print("\n Words - tokens \n")
print(tokens)

# remove all tokens that are not alphabetic
words = [word for word in tokens if word.isalpha()]
```



Discovering Features of a Document

```
# filter out stop words
from nltk.corpus import stopwords
stop_words = set(stopwords.words('english'))

print("\n stop_words \n")
print(stop_words)

words = [w for w in words if not w in stop_words]

print("\n filter out stop words \n")
print(words)

# stemming of words
from nltk.stem.porter import PorterStemmer
porter = PorterStemmer()
stemmed = [porter.stem(word) for word in words]

print("\n PorterStemmer \n")
print(stemmed)
```



Source: J. Ullman and A. Rajamaran for "Mining Massive Datasets" Computer Engineering, San Jose State University.

Chandrasekar Vuppalapati, Professor, MS Software Engineering,

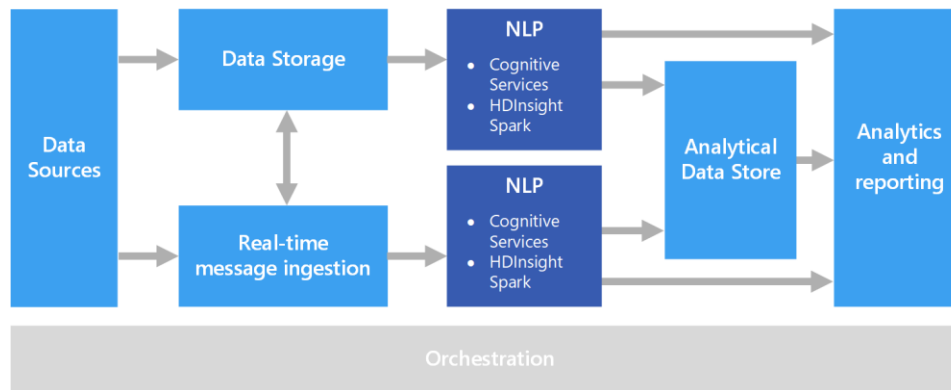
Natural language processing

NLP

NLP

Background

- **NLP**— Natural language processing (NLP) is used for tasks such as sentiment analysis, topic detection, language detection, key phrase extraction, and document categorization.
- NLP can be use to **classify** documents, such as labeling documents as sensitive or spam. The output of NLP can be used for subsequent processing or search.
- Another use for NLP is to **summarize** text by identifying the entities present in the document. These entities can also be used to tag documents with keywords, which enables search and retrieval based on content. Entities might be combined into topics, with summaries that describe the important topics present in each document. The detected topics may be used to categorize the documents for navigation, or to enumerate related documents given a selected topic.
- Another use for NLP is to **score text for sentiment**, to assess the positive or negative tone of a document



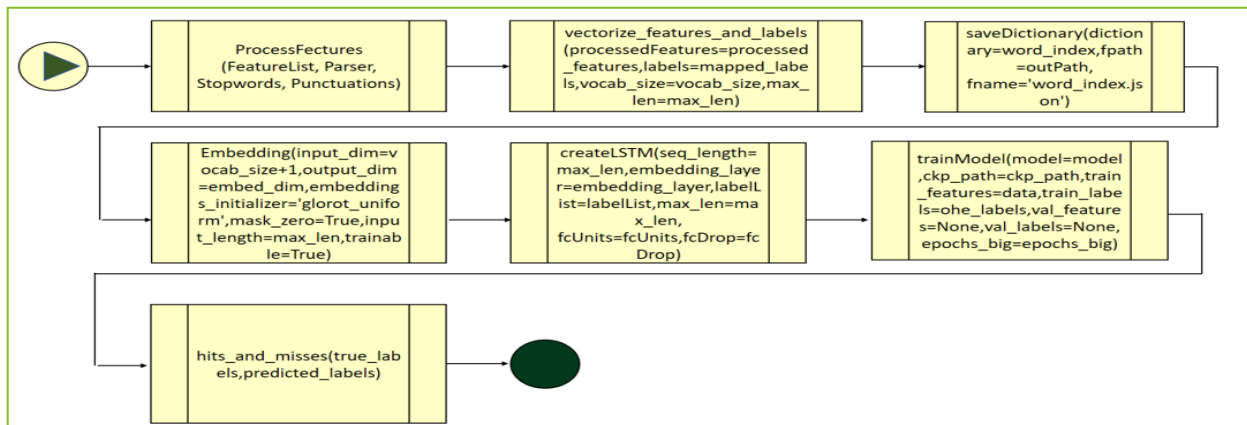
Source: NLP <https://docs.microsoft.com/en-us/azure/architecture/data-guide/scenarios/natural-language-processing>



NLP Techniques

Background

- **Tokenizer.** Splitting the text into words or phrases.
- **Stemming and lemmatization.** Normalizing words so that that different forms map to the canonical word with the same meaning. For example, "running" and "ran" map to "run."
- **Entity extraction.** Identifying subjects in the text.
- **Part of speech detection.** Identifying text as a verb, noun, participle, verb phrase, and so on.
- **Sentence boundary detection.** Detecting complete sentences within paragraphs of text.



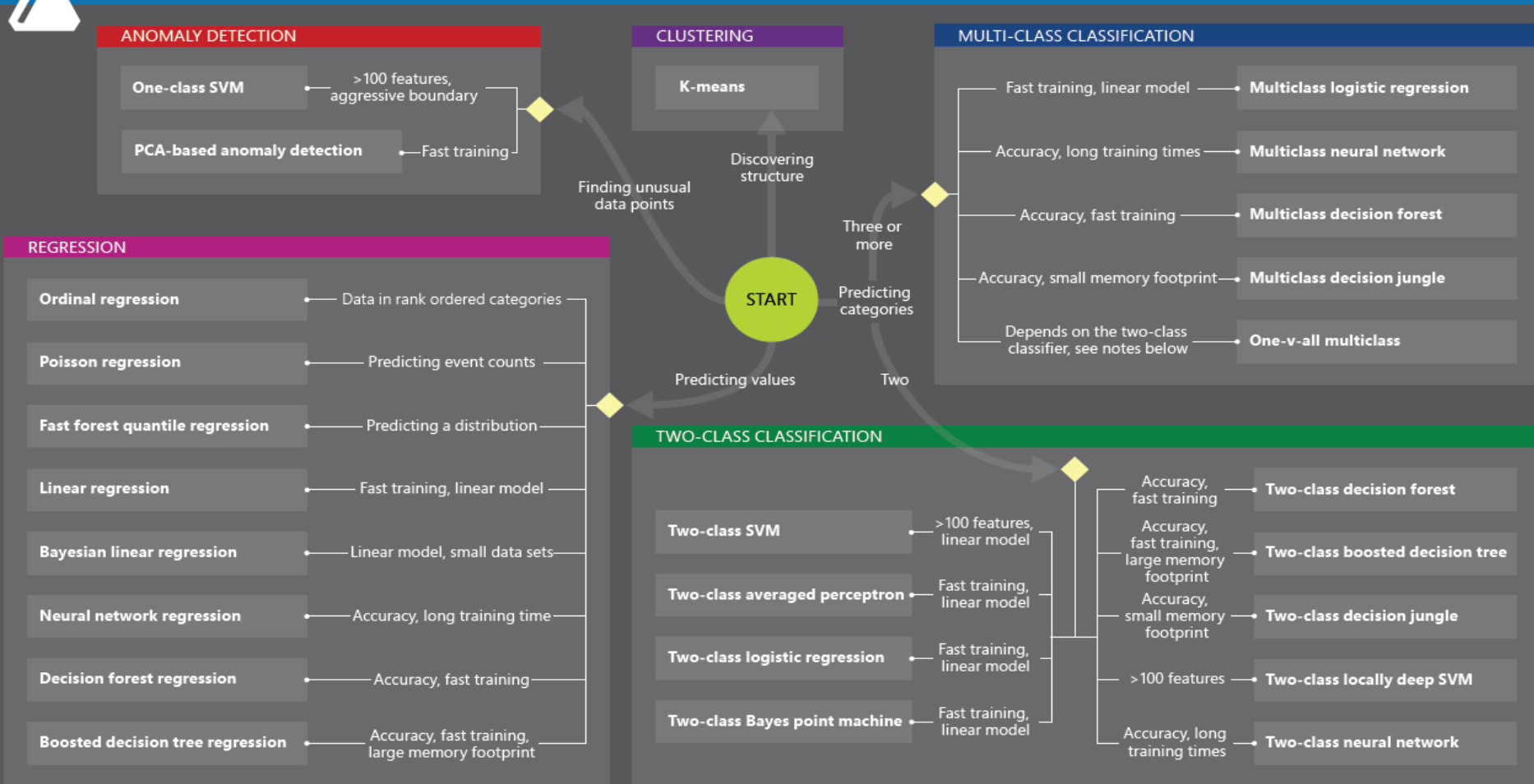
Source: NLP <https://docs.microsoft.com/en-us/azure/architecture/data-guide/scenarios/natural-language-processing>





Microsoft Azure Machine Learning: Algorithm Cheat Sheet

This cheat sheet helps you choose the best Azure Machine Learning Studio algorithm for your predictive analytics solution. Your decision is driven by both the nature of your data and the question you're trying to answer.

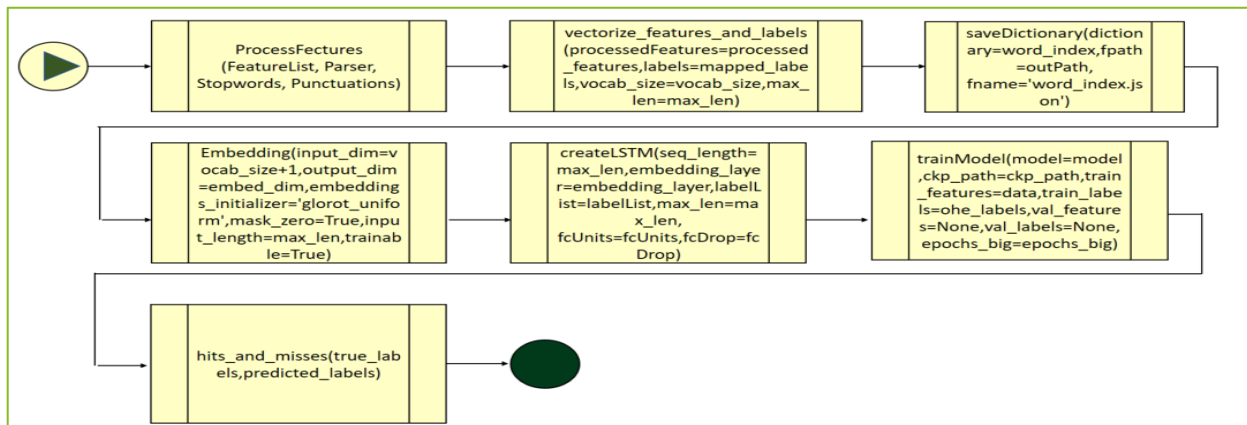


Algorithms

NLP Techniques

Background

- **Tokenizer.** Splitting the text into words or phrases.
- **Stemming and lemmatization.** Normalizing words so that that different forms map to the canonical word with the same meaning. For example, "running" and "ran" map to "run."
- **Entity extraction.** Identifying subjects in the text.
- **Part of speech detection.** Identifying text as a verb, noun, participle, verb phrase, and so on.
- **Sentence boundary detection.** Detecting complete sentences within paragraphs of text.



Source: NLP <https://docs.microsoft.com/en-us/azure/architecture/data-guide/scenarios/natural-language-processing>



Language processing task	NLTK modules	Functionality
Accessing Corpora	nltk.corpus	Standardized interfaces to corpora and lexicons
String processing	nltk.tokenize, nltk.stem	Tokenizers, sentence tokenizers, stemmers
Collocation discovery	nltk.collocations	t-test, chi-squared, point-wise mutual information
Part-of-speech tagging	nltk.tag	n-gram, backoff, Brill, HMM, TnT

Source: Natural Language Processing with Python Steven Bird, Ewan Klein, and Edward Loper



Language processing task	NLTK modules	Functionality
Accessing corpora	<code>nltk.corpus</code>	Standardized interfaces to corpora and lexicons
String processing	<code>nltk.tokenize</code> , <code>nltk.stem</code>	Tokenizers, sentence tokenizers, stemmers
Collocation discovery	<code>nltk.collocations</code>	t-test, chi-squared, point-wise mutual information
Part-of-speech tagging	<code>nltk.tag</code>	n-gram, backoff, Brill, HMM, TnT
Classification	<code>nltk.classify</code> , <code>nltk.cluster</code>	Decision tree, maximum entropy, naive Bayes, EM, k-means
Chunking	<code>nltk.chunk</code>	Regular expression, n-gram, named entity
Parsing	<code>nltk.parse</code>	Chart, feature-based, unification, probabilistic, dependency
Semantic interpretation	<code>nltk.sem</code> , <code>nltk.inference</code>	Lambda calculus, first-order logic, model checking
Evaluation metrics	<code>nltk.metrics</code>	Precision, recall, agreement coefficients
Probability and estimation	<code>nltk.probability</code>	Frequency distributions, smoothed probability distributions
Applications	<code>nltk.app</code> , <code>nltk.chat</code>	Graphical concordancer, parsers, WordNet browser, chatbots

Source: Natural Language Processing with Python Steven Bird, Ewan Klein, and Edward Loper



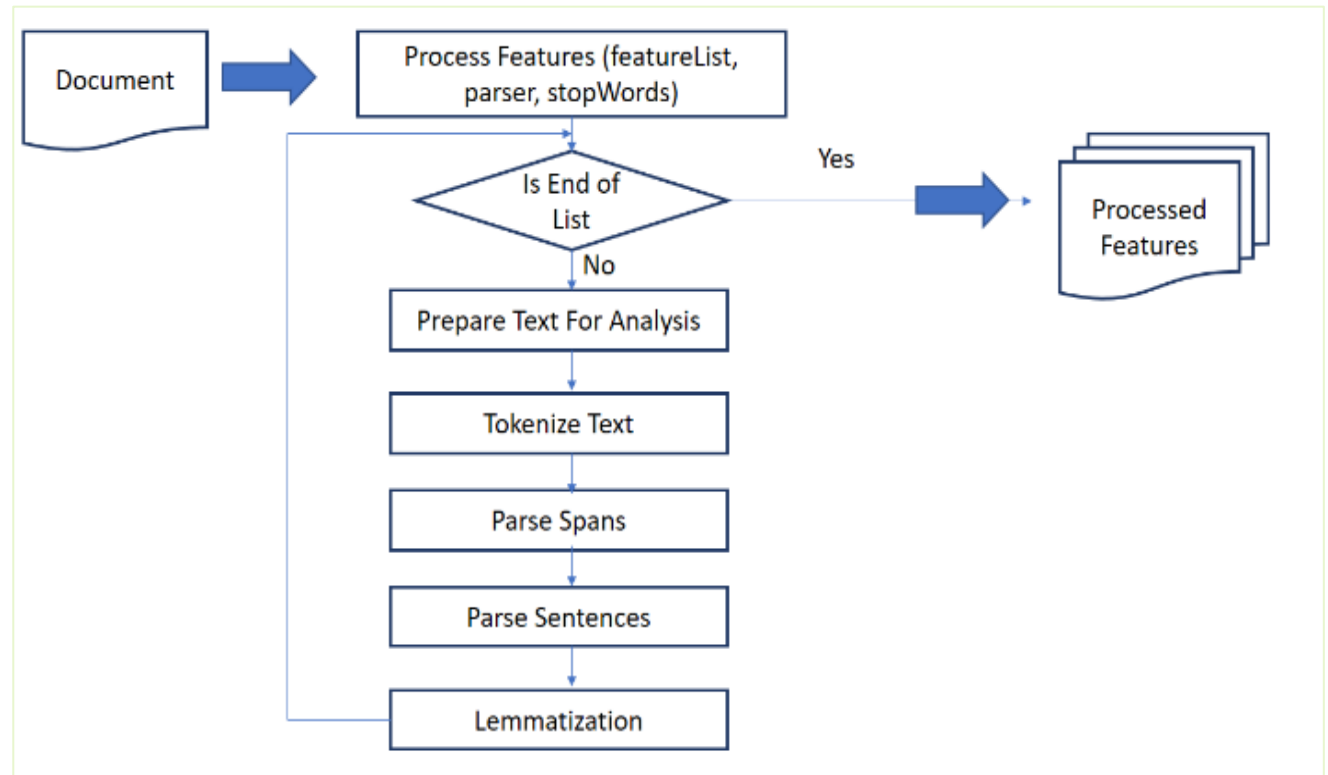
NLP

As-Is Architecture

Process Features of Documents

- Prepare Text for Analysis
 - Tokenize Text
 - Parse Spans
 - Parse Sentences
 - Get Lemma
 - Lemmatization means to get the "dictionary entry" for a word

Return Processed Features List



NLP

NLP Techniques

Background

- **Tokenizer.** Splitting the text into words or phrases.

Code --- MLPOC>python D_tokenizer.py

```
from nltk.tokenize import word_tokenize
s = "Good muffins cost $3.88\nin New York. Please buy me two of them.\n\nThanks."
print(word_tokenize(s))
```

```
'Good', 'muffins', 'cost', '$', '3.88', 'in', 'New', 'York', '.', 'Please', 'buy', 'me', 'two', 'of', 'them', '.', 'Thanks', '.']
```

NLTK Tokenizer Package

Tokenizers divide strings into lists of substrings. For example, tokenizers can be used to find the words and punctuation in a string:



NLP

NLP Techniques

Background

- **Tokenizer.** Splitting the text into words or phrases.

Code --- MLPOC>python D_tokenizer.py

```
from nltk.tokenize import wordpunct_tokenize
s = "Good muffins cost $3.88\nin New York. Please buy me two of them.\n\nThanks."
print(wordpunct_tokenize(s))
```

```
(base) C:\Hanumayamma\MLPOC>python D_tokenizer.py
['Good', 'muffins', 'cost', '$', '3.88', 'in', 'New', 'York', '.', 'Please', 'buy', 'me', 'two', 'of', 'them', '.', 'Thanks', '.']
['Good', 'muffins', 'cost', '$', '3', '.', '88', 'in', 'New', 'York', '.', 'Please', 'buy', 'me', 'two', 'of', 'them', '.', 'Thanks', '.']
```



NLP

NLP Techniques

Background

- **Tokenizer.** Splitting the text into words or phrases.

Code --- MLPOC>python D_tokenizer.py

```
from nltk.tokenize import wordpunct_tokenize
s = "Good muffins cost $3.88\nin New York. Please buy me two of them.\n\nThanks."
print(wordpunct_tokenize(s))
```

```
(base) C:\Hanumayamma\MLPOC>python D_tokenizer.py
['Good', 'muffins', 'cost', '$', '3.88', 'in', 'New', 'York', '.', 'Please', 'buy', 'me', 'two', 'of', 'them', '.', 'Thanks', '.']
['Good', 'muffins', 'cost', '$', '3', '.', '88', 'in', 'New', 'York', '.', 'Please', 'buy', 'me', 'two', 'of', 'them', '.', 'Thanks', '.']
```

Punkt sentence tokenization



NLP

NLP Techniques

Background

- **Tokenizer.** Splitting the text into words or phrases.

Code --- MLPOC>python D_tokenizer.py

```
from nltk.tokenize import sent_tokenize, word_tokenize
s = "Good muffins cost $3.88\nin New York. Please buy me two of them.\n\nThanks."
print(sent_tokenize(s))
```

```
(base) C:\Hanumayamma\MLPOC>python D_tokenizer.py
['Good', 'muffins', 'cost', '$', '3.88', 'in', 'New', 'York', '.', 'Please', 'buy', 'me', 'two', 'of', 'them', '.', 'Thanks', '.']
['Good', 'muffins', 'cost', '$', '3', '.', '88', 'in', 'New', 'York', '.', 'Please', 'buy', 'me', 'two', 'of', 'them', '.', 'Thanks', '.']
['Good muffins cost $3.88\nin New York.', 'Please buy me two of them.', 'Thanks.']
```

**sentence
tokenizer**



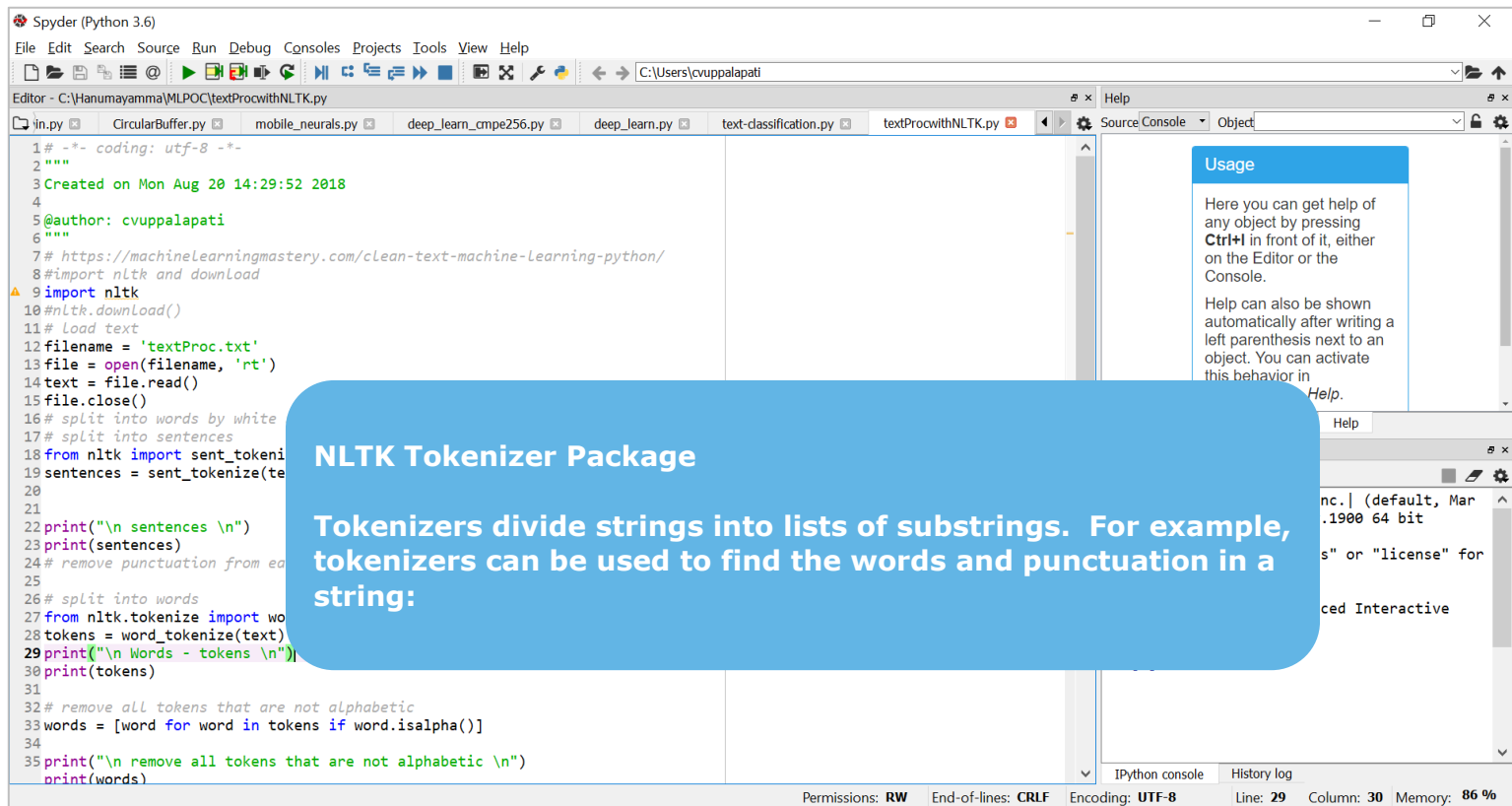
NLP

NLP Techniques

Background

- **Tokenizer.** Splitting the text into words or phrases.

Code --- MLPOC>python textProcwithNLTK.py



The screenshot shows the Spyder Python IDE interface. The editor window displays a Python script for text processing using NLTK. The script includes comments and code for importing NLTK, downloading the data, loading a text file, and tokenizing it into sentences and words. A blue callout box is overlaid on the script, explaining the NLTK Tokenizer Package.

```
1 #-*- coding: utf-8 -*-
2 """
3 Created on Mon Aug 20 14:29:52 2018
4
5 @author: cvuppapapati
6 """
7 # https://machinelearningmastery.com/clean-text-machine-learning-python/
8 #import nltk and download
9 import nltk
10 nltk.download()
11 # load text
12 filename = 'textProc.txt'
13 file = open(filename, 'rt')
14 text = file.read()
15 file.close()
16 # split into words by white
17 # split into sentences
18 from nltk import sent_tokenize
19 sentences = sent_tokenize(text)
20
21
22 print("\n sentences \n")
23 print(sentences)
24 # remove punctuation from each sentence
25
26 # split into words
27 from nltk.tokenize import word_tokenize
28 tokens = word_tokenize(text)
29 print("\n Words - tokens \n")
30 print(tokens)
31
32 # remove all tokens that are not alphabetic
33 words = [word for word in tokens if word.isalpha()]
34
35 print("\n remove all tokens that are not alphabetic \n")
36 print(words)
```

NLTK Tokenizer Package

Tokenizers divide strings into lists of substrings. For example, tokenizers can be used to find the words and punctuation in a string:



NLP Techniques

Background

- **Tokenizer.** Splitting the text into words or phrases.

```
Anaconda Prompt

(base) C:\Hanumayamma\MLPOC>python textProcwithNLTK.py

sentences

['THIS Kinged INVESTMENT AND MANAGEMENT AGREEMENT is made and entered into effective as of the 1st day of December, 2000 by and between RIGGS CAPITAL PARTNERS, LLC (the "Fund") a Delaware limited liability company and RCP INVESTMENTS II, LLC (the "Management Company"), a Delaware limited partnership.', 'Capitalized terms used herein and not otherwise defined herein shall have the defined meaning given to such terms in the Operating Agreement of the Fund dated as of October 1, 2000, as such Agreement may be amended from time to time (the "Operating Agreement").', 'King']

Words - tokens

['THIS', 'Kinged', 'INVESTMENT', 'AND', 'MANAGEMENT', 'AGREEMENT', 'is', 'made', 'and', 'entered', 'into', 'effective', 'as', 'of', 'the', '1st', 'day', 'of', 'December', '2000', 'by', 'and', 'between', 'RIGGS', 'CAPITAL', 'PARTNERS', 'LLC', 'the', 'Fund', 'a', 'Delaware', 'limited', 'liability', 'company', 'and', 'RCP', 'INVESTMENTS', 'II', 'LLC', 'the', 'Management', 'Company', 'Capitalized', 'terms', 'used', 'herein', 'and', 'not', 'otherwise', 'defined', 'herein', 'shall', 'have', 'the', 'defined', 'meaning', 'given', 'to', 'such', 'terms', 'in', 'the', 'Operating', 'Agreement', 'of', 'the', 'Fund', 'dated', 'as', 'of', 'October', '1', '2000', 'as', 'such', 'Agreement', 'may', 'be', 'amended', 'from', 'time', 'to', 'time', 'the', 'Operating', 'Agreement', 'King']

remove all tokens that are not alphabetic

['THIS', 'Kinged', 'INVESTMENT', 'AND', 'MANAGEMENT', 'AGREEMENT', 'is', 'made', 'and', 'entered', 'into', 'effective', 'as', 'of', 'the', 'day', 'of', 'December', 'by', 'and', 'between', 'RIGGS', 'CAPITAL', 'PARTNERS', 'LLC', 'the', 'Fund', 'a', 'Delaware', 'limited', 'liability', 'company', 'and', 'RCP', 'INVESTMENTS', 'II', 'LLC', 'the', 'Management', 'Company', 'a', 'Delaware', 'limited', 'partnership', 'Capitalized', 'terms', 'used', 'herein', 'and', 'not', 'otherwise', 'defined', 'herein', 'shall', 'have', 'the', 'defined', 'meaning', 'given', 'to', 'such', 'terms', 'in', 'the', 'Operating', 'Agreement', 'of', 'the', 'Fund', 'dated', 'as', 'of', 'October', 'as', 'such', 'Agreement', 'may', 'be', 'amended', 'from', 'time', 'to', 'time', 'the', 'Operating', 'Agreement', 'King']

stop_words

{'her', 'hasn', 'its', 'up', 'all', 'nor', 'a', 'our', 'y', 'having', 'an', 'or', 'such', 'hadn't', 'how', 'each', 'll', 'mightn', 'who', 'herself', 'is', 'me', 'am', 'when', 'wasn't', 'shouldn', 'was', 'ain', 'ma', 'and', 'then', 'own', 'now', 'aren't', 'wouldn't', 'your', 'you', 'ourselves', 'what', 'd', 'to', 'mustn', 'so', 'than', 'couldn', 't', 'weren't', 'his', 'against', 'haven', 'myself', 'further', 's', 'mightn't', 'which', 'did', 'will', 'their', 'it's', 'yourselves', 'some', 'no', 'don', 'had', 'should', 'at', 'isn', 'you've', 'theirs', 'him', 'before', 'be', 'won't', 'both', 'as', 'i', 'why', 'mustn't', 'you'd', 'himself', 'o', 'we', 'isn't', 'were', 'ours', 'aren', 'where', 'only', 'more', 'you're', 'above', 'by', 'just', 'between', 'through', 't', 'few', 'for', 'very', 'she's', 'they', 'any', 'my', 'needn', 'into', 'most', 'she', 'he', 'too', 'should've', 'been', 'hers', 'shan't', 'same', 'whom', 've', 'them', 'has', 'over', 'here', 'being', 'couldn', 'hadn', 'down', 'once', 'won', 'not', 'that', 'those', 'below', 'after', 'during', 'in', 'on', 'there', 'doesn', 'under', 'needn't', 'it', 'do', 'again', 'other', 'have', 'but', 'haven't', 'because', 'off', 'doing', 'weren', 'until', 'wouldn', 'from', 'didn', 'out', 'are', 'about', 'doesn't', 'itself', 'yourself', 'that'll', 'if', 'don't', 'shan', 'hasn't', 'you'll', 'themselves', 'these', 'didn', 't', 'yours', 'of', 'while', 'the', 'can', 'wasn', 'm', 'with', 'shouldn't', 'this', 're', 'does'}
```

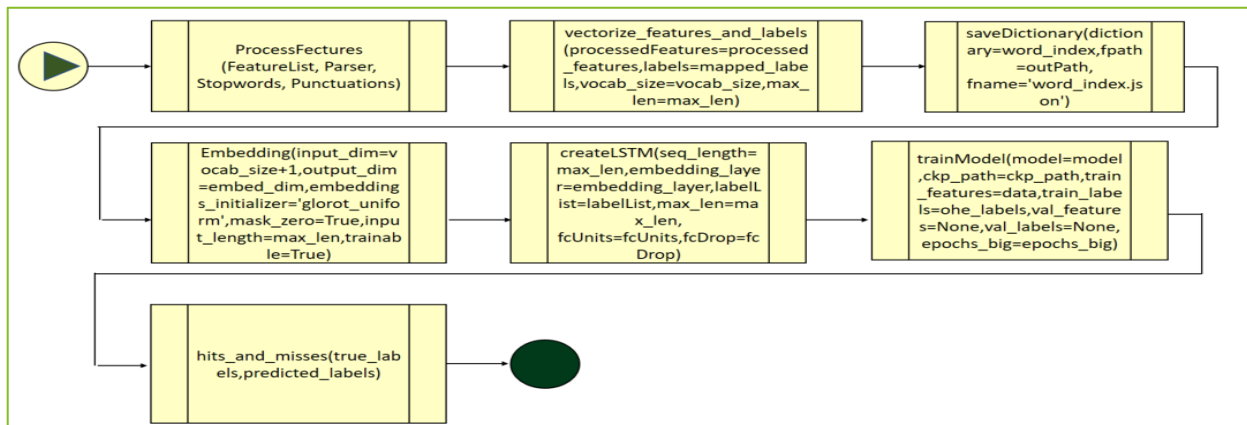


Stemming and lemmatization

NLP Techniques

Background

- **Tokenizer.** Splitting the text into words or phrases.
- **Stemming and lemmatization.** Normalizing words so that that different forms map to the canonical word with the same meaning. For example, "running" and "ran" map to "run."
- **Entity extraction.** Identifying subjects in the text.
- **Part of speech detection.** Identifying text as a verb, noun, participle, verb phrase, and so on.
- **Sentence boundary detection.** Detecting complete sentences within paragraphs of text.



Argus

Source: NLP <https://docs.microsoft.com/en-us/azure/architecture/data-guide/scenarios/natural-language-processing>



Porter Stemmer

The Porter Stemmer

Daniel Waegel
CISC889 / Fall 2011

<http://www.eecis.udel.edu/~trnka/CISC889-11S/lectures/dan-porters.pdf>



Porter Stemmer

Stemming approaches

- Many different approaches:
 - Brute force look up
 - Suffix, affix stripping
 - Part-of-speech recognition
 - Statistical algorithms (n-grams, HMM)
- Porter stemmer utilizes suffix stripping
- It does not address prefixes

<http://www.eecis.udel.edu/~trnka/CISC889-11S/lectures/dan-porters.pdf>



Porter Stemmer

Porter Stemmer Steps

- Step 1: Gets rid of plurals and -ed or -ing suffixes
- Step 2: Turns terminal y to i when there is another vowel in the stem
- Step 3: Maps double suffixes to single ones: -ization, -ational, etc.
- Step 4: Deals with suffixes, -full, -ness etc.
- Step 5: Takes off -ant, -ence, etc.
- Step 6: Removes a final -e

<http://www.eecis.udel.edu/~trnka/CISC889-11S/lectures/dan-porters.pdf>



Porter Stemmer

Step 1

Gets rid of plurals and -ed or -ing suffixes

```
if b[k] == 's'
    if ends("sses")
        k -= 2
    else if ends("ies")
        setto("i")
    else if b[k-1] != 's'
        k--
if ends("eed")
    if m() > 0
        k--
else if ends("ed") || ends("ing") &&
vowelinstem()
    k = j
    if ends("at")
        setto("ate")
    else if ends("bl")
        setto("ble")
    else if ends("iz")
        setto("ize")
    else if doublec(k)
        k--
        int ch = b[k]
        if ch=='l' || ch=='s' || ch=='z'
            k++
    else if (m() == 1 && cvc(k))
        setto("e")
```

- Possesses → possess
- Ponies → poni
- Operatives → operative
- **Markedly → markedly**
- Interesting → interest
- **Confess → confess**
- Consumables → consumable
- Realizes → realize
- Infuriating → Infuriate
- Fables → fable
- Fated → fate

<http://www.eecis.udel.edu/~trnka/CISC889-11S/lectures/dan-porters.pdf>



Porter Stemmer

Step 2

Turns terminal y to i when there is another vowel in the stem

```
if ends("y") && vowelinstem()  
    b[k] = 'i'
```

- Coolly → coolli
- Furry → furri
- Fry → fry
- Grey → grei
- **Interestingly →
Interestinglyli**

<http://www.eecis.udel.edu/~trnka/CISC889-11S/lectures/dan-porters.pdf>



Porter Stemmer

Step 3

Maps double suffixes to single ones

```
switch b[k-1]
case 'a':
    if ends("ational") -> r("ate")
    if ends("tional") -> r("tion")
case 'c':
    if ends("enci") -> r("ence")
    if ends("anci") -> r("ance")
case 'e':
    if ends("izer") -> r("ize")
case 'l':
    if ends("bli") -> r("ble")
    if ends("alli") -> r("al")
    if ends("entli") -> r("ent")
    if ends("eli") -> r("e")
    if ends("ousli") -> r("ous")
case 'o':
    if ends("ization") -> r("ize")
    if ends("ation") -> r("ate")
    if ends("ator") -> r("ate")
case 's':
    if ends("alism") -> r("al")
    if ends("iveness") -> r("ive")
    if ends("fulness") -> r("ful")
    if ends("ousness") -> r("ous")
case 't':
    if ends("aliti") -> r("al")
    if ends("iviti") -> r("ive")
    if ends("biliti") -> r("ble")
case 'g':
    if ends("logi") -> r("log")
```

- **Rational** → **rational**
- **Optional** → **option**
- **Operational** → **operate**
- **Possibly** → **possibli** → **possible**
- **Really** → **realli** → **realli**
- **Realization** → **realize**
- **Feudalism** → **feudal**
- **Playfulness** → **playful**
- **Liveness** → **liveness**

<http://www.cecis.udel.edu/~trnka/CISC889-11S/lectures/dan-porters.pdf>



Porter Stemmer

Step 4

Deals with suffixes, -full, -ness etc.

```
switch b[k]
case 'e':
    if ends("icate") -> r("ic")
    if ends("ative") -> r("")
    if ends("alize") -> r("al")
case 'i':
    if ends("iciti") -> r("ic")
case 'l':
    if ends("ical") -> r("ic")
    if ends("ful") -> r("")
case 's':
    if ends("ness") -> r("")
```

- Authenticate → authentic
- Predicate → predic
- **Realize** → **realize**
- Felicity → felicit → felic
- Practical → practic
- Playful → play
- **Gleeful** → **gleeful**
- **Large** → **large**

<http://www.eecis.udel.edu/~trnka/CISC889/11S/lectures/dan-porters.pdf>



Porter Stemmer

Step 5

Takes off -ant, -ence, etc.

```
switch b[k-1]
case 'a':
    if ends("al") -> break
case 'c':
    if ends("ance") -> break
    if ends("ence") -> break
case 'e':
    if ends("er") -> break
case 'i':
    if ends("ic") -> break
case 'l':
    if ends("able") -> break
    if ends("ible") -> break
... more rules here...
default: return
if m() > 1
    k = j;
```

- Precedent → preced
- Operational → operate
→ oper
-
- Interestingly
- Infuriating
- Fable → fable
- Parable → parable
- Controllable →

<http://www.eecis.udel.edu/~trnka/CSC889-11S/lectures/dan-porters.pdf>



Porter Stemmer

Step 6

Removes a final -e

```
if b[k] == 'e'
    int a = m()
    if a > 1 || a == 1 && !cvc(k-1)
        k--
if b[k]=='l' && doublec(k) && m() > 1
    k--
```

- Parable → parabl
- Fate → fate (cvc)
- Deflate → deflat
- Bee → bee
- Controllable →
control → control
- Petrol → petrol
- Stall → stall
- Resell → resel

<http://www.eecis.udel.edu/~trnka/CISC889-11S/lectures/dan-porters.pdf>



Porter Stemmer

NLP Techniques

Stemmer

python D_textProcwithStemmer.py

```
# Stemming of words
from nltk.stem.porter import PorterStemmer
porter = PorterStemmer()
stemmed = [porter.stem(word) for word in tokens]
print(stemmed[:100])
```

```
'THIS', 'Kinged', 'INVESTMENT', 'AND', 'MANAGEMENT', 'AGREEMENT', 'is', 'made', 'and', 'entered', 'into', 'effective', 'as', 'of', 'the', '1st', 'day', 'of', 'December', '1', '2000', 'by', 'and', 'between', 'RIGGS', 'CAPITAL', 'PARTNERS', 'LLC', '(', 'the', 'Fund', 'a', 'Delaware', 'limited', 'liability', 'company', 'and', 'RCP', 'INVESTMENTS', 'II', 'LLC', '(', 'the', 'Management', 'Company', 'a', 'Delaware', 'limited', 'partnership', 'Capitalized', 'terms', 'used', 'herein', 'and', 'not', 'otherwise', 'defined', 'herein', 'shall', 'have', 'the', 'defined', 'meaning', 'given', 'to', 'such', 'terms', 'in', 'the', 'Operating', 'agreement', 'of', 'the', 'Fund', 'dated', 'as', 'of', 'October', '1', '2000', 'as', 'such', 'Agreement', 'may', 'be', 'amended', 'from', 'time']
```

PORTER STEMMER

```
'thi', 'king', 'invest', 'and', 'manag', 'agreement', 'is', 'made', 'and', 'enter', 'into', 'effect', 'as', 'of', 'the', '1st', 'day', 'of', 'decemb', '1', '2000', 'by', 'nd', 'between', 'rigg', 'capit', 'partner', 'llc', '(', 'the', 'fund', 'a', 'delawar', 'limit', 'liabil', 'compani', 'and', 'rcp', 'invest', 'II', 'llc', '(', 'the', 'manag', 'compani', 'a', 'delawar', 'limit', 'partnership', 'capit', 'term', 'use', 'herein', 'and', 'not', 'otherwis', 'defi', 'herein', 'shall', 'have', 'the', 'defin', 'mean', 'given', 'to', 'such', 'term', 'in', 'the', 'oper', 'agreement', 'of', 'the', 'fund', 'date', 'as', 'of', 'octob', '1', '2000', 'as', 'such', 'agreement', 'may', 'be', 'amend', 'from', 'time']
```



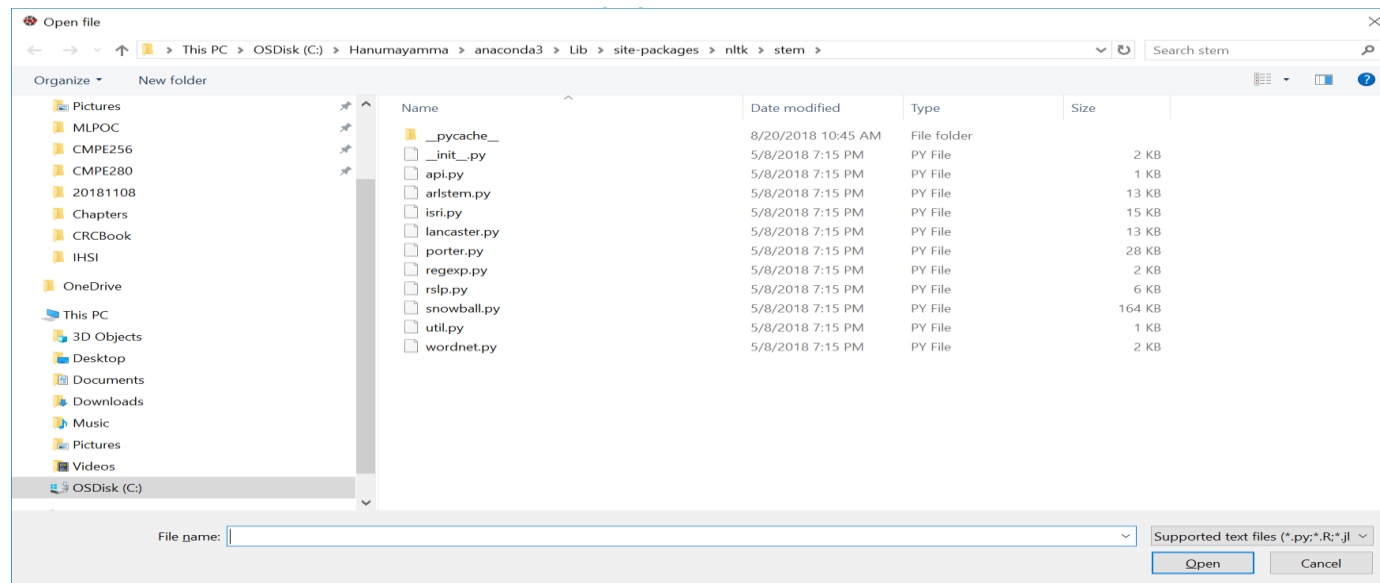
NLP Techniques

Stemmer

python D_textProcwithStemmer.py

```
# Stemming of words
from nltk.stem.porter import PorterStemmer
porter = PorterStemmer()
stemmed = [porter.stem(word) for word in tokens]
print(stemmed[:100])
```

<https://tartarus.org/martin/PorterStemmer/pytho>

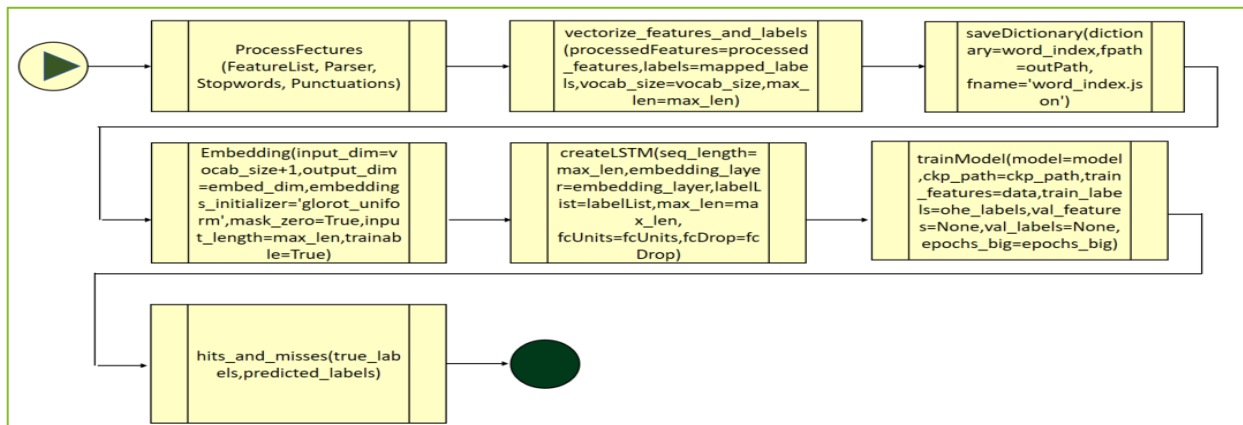


Entity Extraction

NLP Techniques

Background

- **Tokenizer.** Splitting the text into words or phrases.
- **Stemming and lemmatization.** Normalizing words so that that different forms map to the canonical word with the same meaning. For example, "running" and "ran" map to "run."
- **Entity extraction.** Identifying subjects in the text.
- **Part of speech detection.** Identifying text as a verb, noun, participle, verb phrase, and so on.
- **Sentence boundary detection.** Detecting complete sentences within paragraphs of text.



Argus

Source: NLP <https://docs.microsoft.com/en-us/azure/architecture/data-guide/scenarios/natural-language-processing>



NLP Techniques

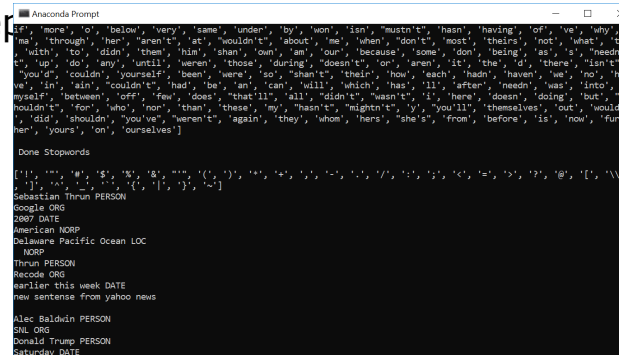
Entity Extraction

python D_spaCyExpSimilarity.py

```
# Process whole documents
```

```
text = (u"When Sebastian Thrun started working on self-driving cars at "  
        u"Google in 2007, few people outside of the company took him "  
        u"seriously. "I can tell you very senior CEOs of major American Delaware  
"  
        u"Pacific Ocean car companies would shake my hand and turn away  
because I wasn't "  
        u"worth talking to," said Thrun, now the co-founder and CEO of "  
        u"online higher education startup Udacity, in an interview with "  
        u"Recode earlier this week.")  
doc = nlp(text)
```

```
# Find named entities, phrases and concepts  
for entity in doc.ents:  
    print(entity.text, entity.label_)
```



```
Done Stopwords  
[('Sebastian Thrun', 'PERSON'), ('Google', 'ORG'), ('2007', 'DATE'), ('American', 'NORP'), ('Delaware Pacific Ocean', 'LOC'), ('Thrun', 'PERSON'), ('Recode', 'ORG'), ('earlier this week', 'DATE'), ('New sentence from yahoo news', 'TEXT'), ('Alec Baldwin', 'PERSON'), ('SNL', 'ORG'), ('Donald Trump', 'PERSON'), ('Saturday', 'DATE')]
```

**Entity
Extraction**



NLP Techniques

Entity Extraction

python D_spaCyExpSimilarity.py

```
Anaconda Prompt
if, 'more', 'o', 'below', 'very', 'same', 'under', 'by', 'won', 'isn', "mustn't", 'hasn', 'having', 'of', 've', 'why',
'ma', 'through', 'her', "aren't", 'at', "wouldn't", 'about', 'me', 'when', "don't", 'most', 'theirs', 'not', 'what', 't'
, 'with', 'to', 'didn', 'them', 'him', 'shan', 'own', 'am', 'our', 'because', 'some', 'don', 'being', 'as', 's', "needn'
t", 'up', 'do', 'any', 'until', 'weren', 'those', 'during', "doesn't", 'or', 'aren', 'it', 'the', 'd', 'there', "isn't",
'you'd', 'couldn', 'yourself', 'been', 'were', 'so', "shan't", 'their', 'how', 'each', 'hadn', 'haven', 'we', 'no', 'ha
ve', 'in', 'ain', "couldn't", 'had', 'be', 'an', 'can', 'will', 'which', 'has', 'll', 'after', 'needn', 'was', 'into', '
myself', 'between', 'off', 'few', 'does', "that'll", 'all', "didn't", "wasn't", 'i', 'here', 'doesn', 'doing', 'but', "s
houldn't", 'for', 'who', 'nor', 'than', 'these', 'my', 'hasn't', "mightn't", 'y', "you'll", 'themselves', 'out', 'wouldn
', 'did', 'shouldn', "you've", "weren't", 'again', 'they', 'whom', 'hers', "she's", 'from', 'before', 'is', 'now', 'furt
her', 'yours', 'on', 'ourselves']

Done Stopwords

['!', '"', '#', '$', '%', '&', "'", '(', ')', '*', '+', ',', '-', '.', '/', ':', ';', '<', '=', '>', '?', '@', '[', '\\',
']', '^', '_', '{', '|', '}', '~']
Sebastian Thrun PERSON
Google ORG
2007 DATE
American NORP
Delaware Pacific Ocean LOC
NORP
Thrun PERSON
Recode ORG
earlier this week DATE
new sentence from yahoo news

Alec Baldwin PERSON
SNL ORG
Donald Trump PERSON
Saturday DATE
```

**Entity
Extraction**

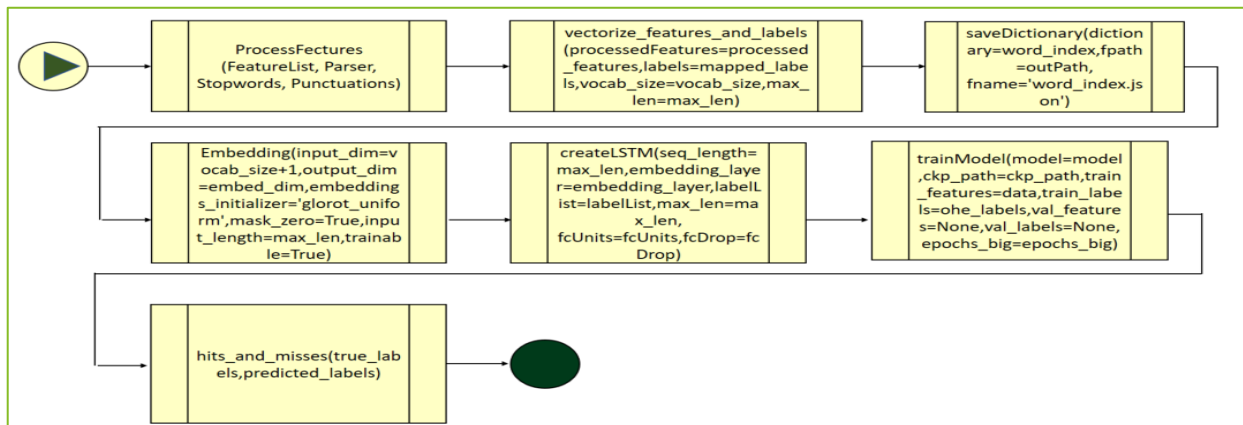


Part of Speech Detection

NLP Techniques

Background

- **Tokenizer.** Splitting the text into words or phrases.
- **Stemming and lemmatization.** Normalizing words so that that different forms map to the canonical word with the same meaning. For example, "running" and "ran" map to "run."
- **Entity extraction.** Identifying subjects in the text.
- **Part of speech detection.** Identifying text as a verb, noun, participle, verb phrase, and so on.
- **Sentence boundary detection.** Detecting complete sentences within paragraphs of text.



Argus

Source: NLP <https://docs.microsoft.com/en-us/azure/architecture/data-guide/scenarios/natural-language-processing>



NLP Techniques

Parts of Speech

python D_spaCyExpSimilarity.py

CC **coordinating conjunction**

CD cardinal digit

DT determiner

EX existential there (like: "there is" ... think of it like "there exists")

FW foreign word

IN **preposition/subordinating conjunction**

JJ adjective 'big'

JJR adjective, comparative 'bigger'

JJS adjective, superlative 'biggest'

LS list marker 1)

MD modal could, will

NN noun, singular 'desk'

NNS noun plural 'desks'

NNP proper noun, singular 'Harrison'

NNPS **proper noun, plural 'Americans'**

PDT predeterminer 'all the kids'

POS possessive ending parent's

PRP personal pronoun I, he, she

PRP\$ possessive pronoun my, his, hers

RB **adverb very, silently,**

RBR adverb, comparative better

RBS adverb, superlative best

RP particle give up

TO to go 'to' the store.

UH interjection errrrrrrm

VB verb, base form take

VBD **verb, past tense took**

VBG verb, gerund/present participle taking

VBN verb, past participle taken

VBP verb, sing. present, non-3d take

VBZ verb, 3rd person sing. present takes

WDT wh-determiner which

WP wh-pronoun who, what

WP\$ possessive wh-pronoun whose

WRB wh-abverb where, when

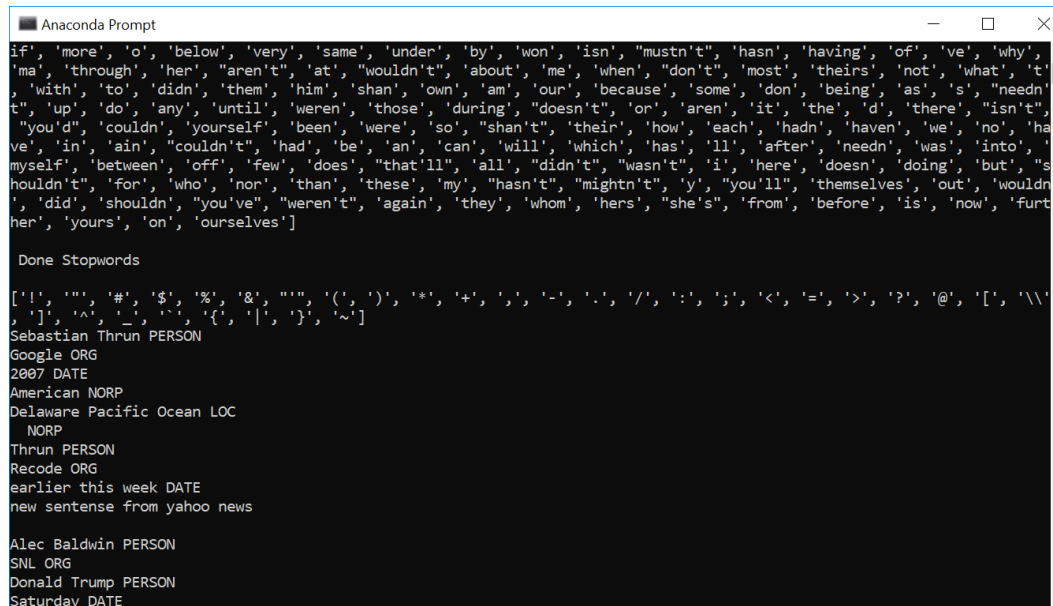


NLP Techniques

Part of Speech

python D_spaCyExpSimilarity.py

```
# print("\n Part of Speech \n")
# Using a Tagger. Which is part-of-speech
# tagger or POS-tagger.
tagged = nltk.pos_tag(words)
print(tagged)
```



```
if, 'more', 'o', 'below', 'very', 'same', 'under', 'by', 'won', 'isn', "mustn't", 'hasn', 'having', 'of', 've', 'why',
'ma', 'through', 'her', 'aren't', 'at', "wouldn't", 'about', 'me', 'when', "don't", 'most', 'theirs', 'not', 'what', 't'
, 'with', 'to', 'didn', 'them', 'him', 'shan', 'own', 'am', 'our', 'because', 'some', 'don', 'being', 'as', 's', "needn
t", 'up', 'do', 'any', 'until', 'weren', 'those', 'during', "doesn't", 'on', 'aren', 'it', 'the', 'd', 'there', "isn't",
"you'd", 'couldn', 'yourself', 'been', 'were', 'so', "shan't", 'their', 'how', 'each', 'hadn', 'haven', 'we', 'no', 'ha
ve', 'in', 'ain', "couldn't", 'had', 'be', 'an', 'can', 'will', 'which', 'has', 'll', 'after', 'needn', 'was', 'into', '
myself', 'between', 'off', 'few', 'does', "that'll", 'all', "didn't", "wasn't", 'i', 'here', 'doesn', 'doing', 'but', "s
houldn't", 'for', 'who', 'nor', 'than', 'these', 'my', "hasn't", "mightn't", 'y', "you'll", 'themselves', 'out', 'wouldn
', 'did', 'shouldn', "you've", "weren't", 'again', 'they', 'whom', 'hers', "she's", 'from', 'before', 'is', 'now', 'furt
her', 'yours', 'on', 'ourselves']

Done Stopwords

['!', '"', '#', '$', '%', '&', "'", '(', ')', '*', '+', ',', '-', '.', '/', ':', ';', '<', '=', '>', '?', '@', '[', '\\
', ']', '^', '_', '`', '{', '|', '}', '~']
Sebastian Thrun PERSON
Google ORG
2007 DATE
American NORP
Delaware Pacific Ocean LOC
NORP
Thrun PERSON
Recode ORG
earlier this week DATE
new sentence from yahoo news

Alec Baldwin PERSON
SNL ORG
Donald Trump PERSON
Saturday DATE
```

Part of Speech



NLP Techniques

Entity Extraction

python D_textProcwithStemmer.py

Part of Speech

```
[('THIS', 'NNP'), ('Kinged', 'NNP'), ('INVESTMENT', 'NNP'), ('AND', 'NNP'), ('MANAGEMENT', 'NNP'), ('AGREEMENT', 'NNP'), ('is', 'VBZ'), ('made', 'VBN'), ('and', 'CC'), ('entered', 'VBN'), ('into', 'IN'), ('effective', 'JJ'), ('as', 'IN'), ('of', 'IN'), ('the', 'DT'), ('day', 'NN'), ('of', 'IN'), ('December', 'NNP'), ('by', 'IN'), ('and', 'CC'), ('between', 'IN'), ('RIGGS', 'NNP'), ('CAPITAL', 'NNP'), ('PARTNERS', 'NNP'), ('LLC', 'NNP'), ('the', 'DT'), ('Fund', 'NNP'), ('a', 'DT'), ('Delaware', 'NNP'), ('limited', 'VBN'), ('liability', 'NN'), ('company', 'NN'), ('and', 'CC'), ('RCP', 'NNP'), ('INVESTMENTS', 'NNP'), ('II', 'NNP'), ('LLC', 'NNP'), ('the', 'DT'), ('Management', 'NNP'), ('Company', 'NNP'), ('a', 'DT'), ('Delaware', 'NNP'), ('limited', 'JJ'), ('partnership', 'NN'), ('Capitalized', 'NNP'), ('terms', 'NNS'), ('used', 'VBD'), ('herein', 'NN'), ('and', 'CC'), ('not', 'RB'), ('otherwise', 'RB'), ('defined', 'VBN'), ('herein', 'NN'), ('shall', 'MD'), ('have', 'VB'), ('the', 'DT'), ('defined', 'JJ'), ('meaning', 'NN'), ('given', 'VBN'), ('to', 'TO'), ('such', 'JJ'), ('terms', 'NNS'), ('in', 'IN'), ('the', 'DT'), ('Operating', 'NNP'), ('Agreement', 'NNP'), ('of', 'IN'), ('the', 'DT'), ('Fund', 'NNP'), ('dated', 'VBD'), ('as', 'IN'), ('of', 'IN'), ('October', 'NNP'), ('as', 'IN'), ('such', 'JJ'), ('Agreement', 'NNP'), ('may', 'MD'), ('be', 'VB'), ('amended', 'VBN'), ('from', 'IN'), ('time', 'NN'), ('to', 'TO'), ('time', 'NN'), ('the', 'DT'), ('Operating', 'NNP'), ('Agreement', 'NNP'), ('King', 'NNP')]
```



Text-Classification Demo

Python text-classification.py

Document Vector

[illegible]

Text to Sequence

[illegible]

Text Classification

Source: Preparing and Architecting for Machine Learning - https://www.gartner.com/binaries/content/assets/events/keywords/catalyst/catus8/preparing_and_architecting_for_machine_learning.pdf



Bag Of Words

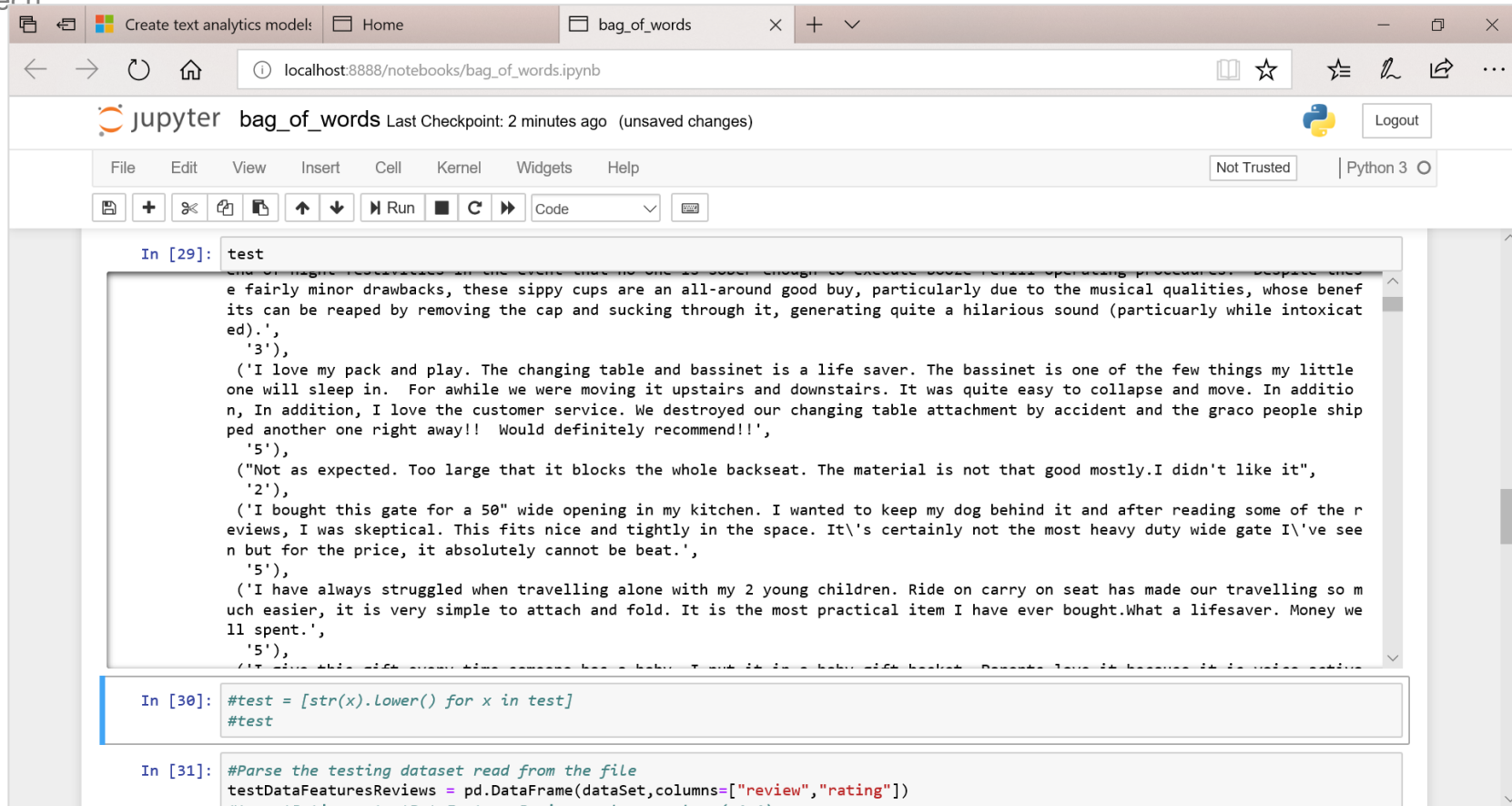
Bag Of Words

Amazon Reviews Data

amazon_test.csv - Excel		Vuppalapati, Chandrasekar (US - Hermitage)
File Home Insert Draw Page Layout Formulas Data Review View Add-ins Acrobat Power Pivot Team Tell me what you want to do		Share
A7	Summer Infant Snuzzler, Ivory	
	A	B
1	name	review
2	OXO Tot 4-Piece Feeding Set, Green	Meals can be quite stressful in our house but now they are a little less messy. My daughter lik
3	Jeep Cherokee Sport Stroller, Brick Red	When shopping for an umbrella stroller we wanted something compact and sturdy. We also d
4	Skip Hop Moby Bathtub Elbow Rest, Blue	Recently purchased this, so I've only used it a handful of times so far. Can't complain about m
5	Cloud b Mimicking Monkey	The "white noise" function on this thing is so quiet that we had to send it back. My k
6	Evenflo Single Breast Pump	I used this product because my son would not latch and I had to pump. This product works w
7	Summer Infant Snuzzler, Ivory	Mt granddaughter always cried when we put her in the carseat,until we put this in they crying
8	Born Free Twin Pack Wide Neck Bottles, 5 Ounce	This is the best bottles ever build. My baby, (3 month old) is happy besides is BPA free.. I recc
9	Boppy Pillow with Brocade Slipcover, Black and White	So far I have only used this for feeding but it helps to provide support to my arms when feedin
10	Chicco Smart Support Backpack, Red	No problems with it. Rugged construction. The windshield is really great. Very sturdy, and ea
11	BRICA UV Alert Window Shade, Gray	I was looking for shades for the back (side) windows in our Jeep and these are a great fit. My h
12	Philips AVENT BPA Free Bottle Brush, Blue	I don't know why would it matter that a brush is made with BPA plastic unless you are plannin
13	Thermos Foogo Leak-Proof SS 7 Ounce Sippy Cup with Handles, Pink, 6 Months	For parents who are concerned about chemicals in what has become traditional baby product
14	NTM-910YIC - Sony Baby Call Nursery Monitor	This monitor works beautifully. It is so clear that we can hear the baby breathing while he is sl
15	Tobbles	MY Grandson loves this toy when he is not playing with it i am playing with it, colors are vibr
16	'The Insulated Sippy' by Eco Vessel Insulated Stainless Steel Sippy Cup - 10 Oz - Hudson Blue - 10 oz - Contain	Pretty. 1 yr old has not gone for using this odd nipple as a sippy for longer than 20 seconds. I'
17	Luvable Friends Flannel Fitted Crib Sheet, Yellow Circle	I love these sheets for our cold Michigan winter nights. Great fit and soft too. They hold up w
18	Beaba 3-Stage Baby Bottle Training Sippy Cup - Pink	My daughter has loved this cup since we bought it over a year ago -- it's always been her favor
19	Hello Kitty Large Collage Pink Childs Backpack	This is perfect for any Hello Kitty fan. She loves it! It was delivered on time and in perfect con
20	JJ Cole Original Infant Bundleme, Snow	I was delighted to receive the bundle me for a baby gift but was quickly disappointed when I re
21	SOHO 4 in 1 Soft Kahki Diaper bag	I loved the design, and I still do, but the size is weird, very square, and seems to be cheaply ma
22	C.R. Gibson Loose-Leaf Memory Book, Baby Bots	This book is awesome! It has pages for everything u could possible want to record. I love the f
23	Delta 48 Piece Nursery Storage Set, Beige	Great starter set for a new baby's closet. Hangers are the perfect size for baby clothes and the
24	Munchkin Nursery Projector and Sound System, White	Fine when it works, which is for about 2 months. I have gone through three of them, too bad t
25	Regalo 192-Inch Super Wide Gate and Play Yard	I bought this pay yard for my four-month old puppy and I am very satisfied. It's sturdy and sa
26	Mary Meyer Wubbanub Plush Pacifier, Cutsie Caterpillar	Our 6 month old loves her ""Wubby""! I would highly recommend it for any youngling who p

Bag Of Words

Part of Speech



The screenshot shows a Jupyter Notebook titled "bag_of_words" running on a local server at localhost:8888. The notebook has a menu bar with File, Edit, View, Insert, Cell, Kernel, Widgets, and Help. Below the menu bar is a toolbar with icons for saving, running, and other actions. The notebook content is divided into three cells:

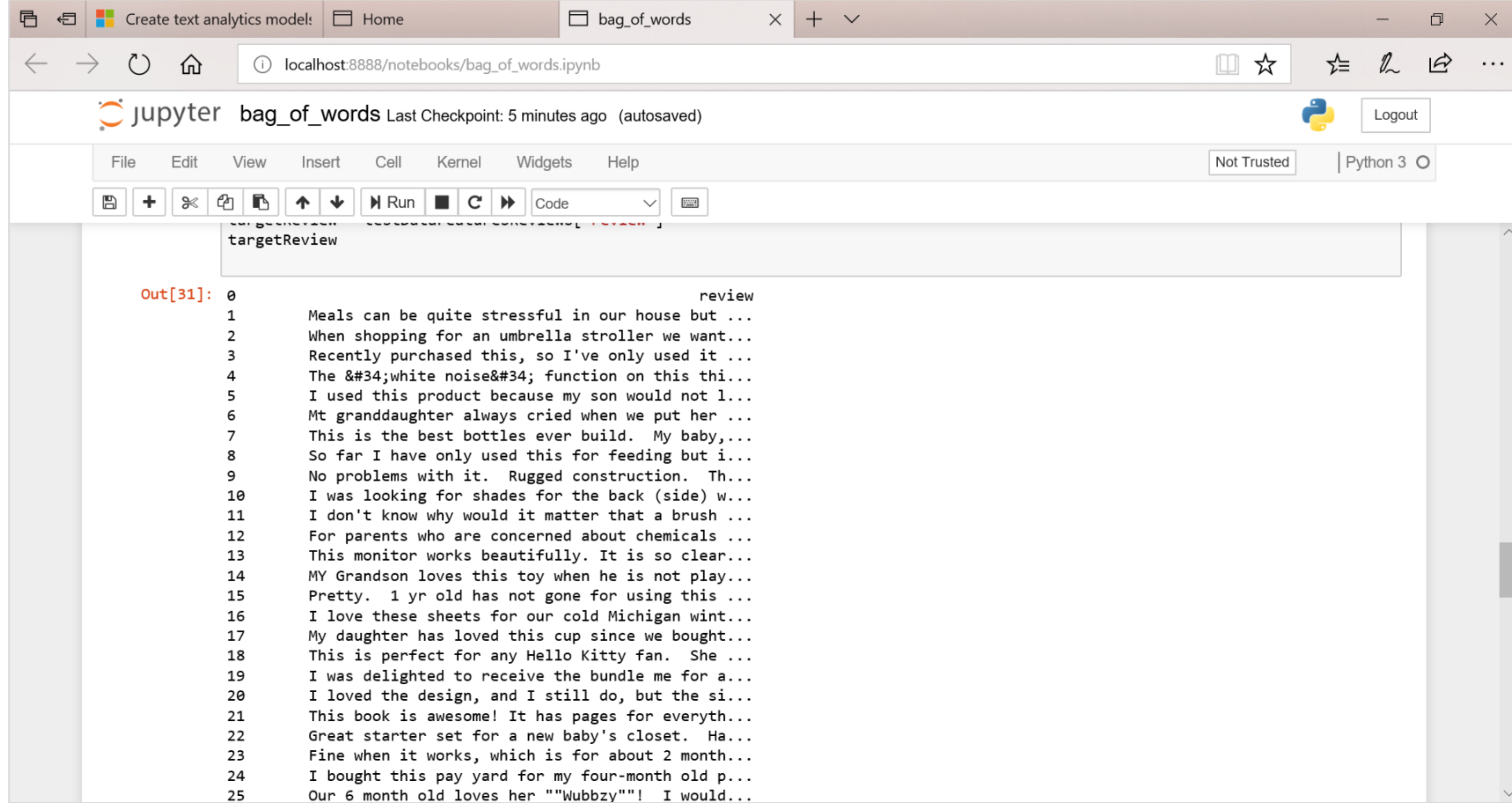
```
In [29]: test
end of night festivities in the event that no one is sober enough to execute some retail operating procedures. Despite these fairly minor drawbacks, these sippy cups are an all-around good buy, particularly due to the musical qualities, whose benefits can be reaped by removing the cap and sucking through it, generating quite a hilarious sound (particularly while intoxicated).',
'3'),
('I love my pack and play. The changing table and bassinet is a life saver. The bassinet is one of the few things my little one will sleep in. For awhile we were moving it upstairs and downstairs. It was quite easy to collapse and move. In addition, In addition, I love the customer service. We destroyed our changing table attachment by accident and the graco people shipped another one right away!! Would definitely recommend!!',
'5'),
("Not as expected. Too large that it blocks the whole backseat. The material is not that good mostly.I didn't like it",
'2'),
('I bought this gate for a 50" wide opening in my kitchen. I wanted to keep my dog behind it and after reading some of the reviews, I was skeptical. This fits nice and tightly in the space. It\'s certainly not the most heavy duty wide gate I\'ve seen but for the price, it absolutely cannot be beat.',
'5'),
('I have always struggled when travelling alone with my 2 young children. Ride on carry on seat has made our travelling so much easier, it is very simple to attach and fold. It is the most practical item I have ever bought.What a lifesaver. Money well spent.',
'5'),
('I give this gift every time someone has a baby. I want it to be a baby gift basket. Parents love it because it is versatile
```

```
In [30]: #test = [str(x).lower() for x in test]
#test
```

```
In [31]: #Parse the testing dataset read from the file
testDataFeaturesReviews = pd.DataFrame(dataSet,columns=["review","rating"])
#testDataFeaturesReviews.columns
```

Bag Of Words

Part of Speech



The screenshot shows a Jupyter Notebook window titled "bag_of_words" with a "Last Checkpoint: 5 minutes ago (autosaved)" status. The interface includes a top bar with navigation icons, a menu bar (File, Edit, View, Insert, Cell, Kernel, Widgets, Help), and a toolbar with icons for saving, running, and other actions. The main area displays a code cell with the following output:

```
Out[31]: 0          review
1      Meals can be quite stressful in our house but ...
2      When shopping for an umbrella stroller we want...
3      Recently purchased this, so I've only used it ...
4      The &#34;white noise&#34; function on this thi...
5      I used this product because my son would not l...
6      Mt granddaughter always cried when we put her ...
7      This is the best bottles ever build. My baby,...
8      So far I have only used this for feeding but i...
9      No problems with it. Rugged construction. Th...
10     I was looking for shades for the back (side) w...
11     I don't know why would it matter that a brush ...
12     For parents who are concerned about chemicals ...
13     This monitor works beautifully. It is so clear...
14     MY Grandson loves this toy when he is not play...
15     Pretty. 1 yr old has not gone for using this ...
16     I love these sheets for our cold Michigan wint...
17     My daughter has loved this cup since we bought...
18     This is perfect for any Hello Kitty fan. She ...
19     I was delighted to receive the bundle me for a...
20     I loved the design, and I still do, but the si...
21     This book is awesome! It has pages for everyth...
22     Great starter set for a new baby's closet. Ha...
23     Fine when it works, which is for about 2 month...
24     I bought this pay yard for my four-month old p...
25     Our 6 month old loves her ""Wubbzy""! I would...
```

Cosine Distance

The cosine distance makes sense in spaces that have dimensions, including Euclidean spaces and discrete versions of Euclidean spaces, such as spaces where points are vectors with integer components or boolean (0 or 1) components. In such a space, points may be thought of as directions. We do not distinguish between a vector and a multiple of that vector. Then the cosine distance between two points is the angle that the vectors to those points make. This angle will be in the range 0 to 180 degrees, regardless of how many dimensions the space has.

We can calculate the cosine distance by first computing the cosine of the angle, and then applying the arc-cosine function to translate to an angle in the 0-180 degree range. Given two vectors x and y , the cosine of the angle between them is the dot product $x \cdot y$ divided by the L2-norms of x and y (i.e., their Euclidean distances from the origin). Recall that the dot product of vectors

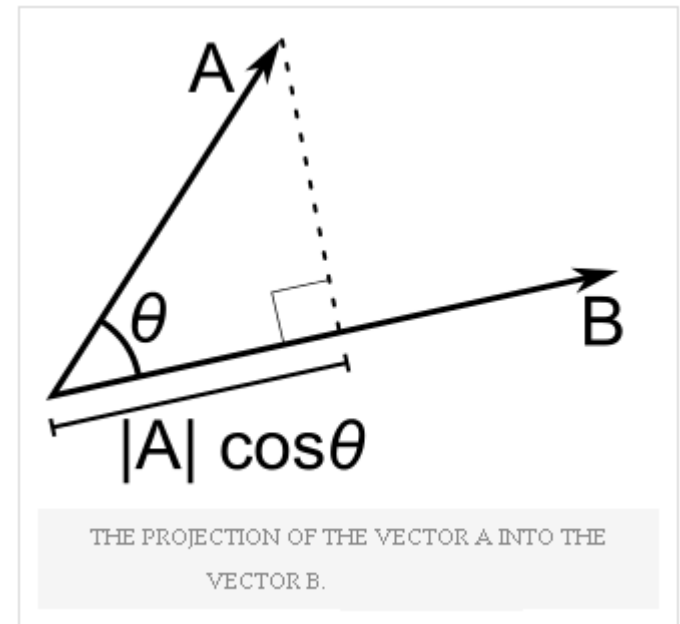
$$[x_1, x_2, \dots, x_n] \cdot [y_1, y_2, \dots, y_n] \text{ is } \sum_{i=1}^n x_i y_i.$$



Cosine Distance

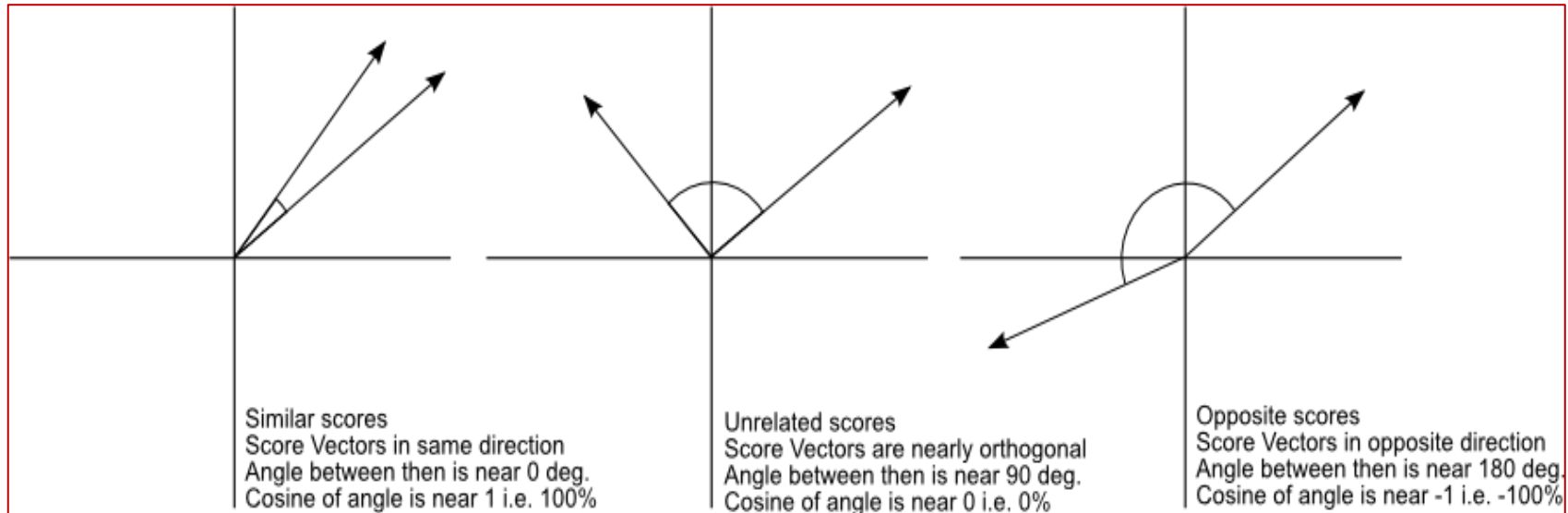
The cosine similarity between two vectors (or two documents on the Vector Space) is a measure that calculates the cosine of the angle between them. This metric is a measurement of orientation and not magnitude, it can be seen as a comparison between documents on a normalized space because we're not taking into the consideration only the magnitude of each word count (Text Feature Extraction (tf-idf)) of each document, but the angle between the documents. What we have to do to build the cosine similarity equation is to solve the equation of the dot product for the

$$\vec{a} \cdot \vec{b} = \|\vec{a}\| \|\vec{b}\| \cos \theta$$
$$\cos \theta = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|}$$



Cosine Distance

And that is it, this is the cosine similarity formula. Cosine Similarity will generate a metric that says how related are two documents by looking at the angle instead of magnitude, like in the examples below:



THE COSINE SIMILARITY VALUES FOR DIFFERENT DOCUMENTS, 1 (SAME DIRECTION), 0 (90 DEG.), -1 (OPPOSITE DIRECTIONS).

Note that even if we had a vector pointing to a point far from another vector, they still could have a small angle and that is the central point on the use of Cosine Similarity, the measurement tends to ignore the higher term count on documents. Suppose we have a document with the word “sky” appearing 200 times and another document with the word “sky” appearing 50, the Euclidean distance between them will be higher but the angle will still be small because they are pointing to the same direction, which is what matters when we are comparing documents.

Cosine Similarity

Suppose the only features of movies are the set of actors and the average rating. Consider two movies with five actors each. Two of the actors are in both movies. Also, one movie has an average rating of 3 and the other an average of 4. The vectors look something like

$$\begin{matrix} 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 3\alpha \\ 1 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 4\alpha \end{matrix}$$

Calculate Cosine Similarity?

$$\text{Dot Product} = (0 \times 1) + (1 \times 1) + (1 \times 0) + (0 \times 1) + (1 \times 0) + (1 \times 1) + (0 \times 1) + (1 \times 0) + (3\alpha \times 4\alpha)$$

$$\text{Dot Product} = 2 + 12\alpha^2$$

$$\text{Lengths of the Vectors} = \sqrt{5 + 9\alpha^2} \text{ and } \sqrt{5 + 16\alpha^2}.$$

$$\text{Cosine Similarity} = \frac{2 + 12\alpha^2}{\sqrt{25 + 125\alpha^2 + 144\alpha^4}}$$

If we choose $\alpha = 1$, that is, we take the average ratings as they are, then the value of the above expression is 0.816. If we use $\alpha = 2$, that is, we double the ratings, then the cosine is 0.940. That is, the vectors appear much closer in direction than if we use $\alpha = 1$. Likewise, if we use $\alpha = 1/2$, then the cosine is 0.619, making the vectors look quite different. We cannot tell which value of α is “right,” but we see that the choice of scaling factor for numerical features affects our decision about how similar items are.



Classification Algorithms

A completely different approach to a recommendation system using item profiles and utility matrices is to treat the problem as one of machine learning. Regard the given data as a training set, and for each user, build a classifier that predicts the rating of all items. There are a great number of different classifiers, and it is not our purpose to teach this subject here. However, you should be aware of the option of developing a classifier for recommendation, so we shall discuss one common classifier – decision trees – briefly.

A decision tree is a collection of nodes, arranged as a binary tree. The leaves render decisions; in our case, the decision would be “likes” or “doesn’t like.” Each interior node is a condition on the objects being classified; in our case the condition would be a predicate involving one or more features of an item.



Classification Algorithms

User Profiles - Who is similar?

	Harry Potter I	Harry Potter II	Harry Potter III	Twilight	Star Wars 1	Star Wars 2	Star Wars 3
Sriya	4			5	1		
Shruti	5	5	4				
John				2	4	5	
Tracy		3					3



Classification Algorithms

We can treat blanks as a 0 value. This choice is questionable, since it has the effect of treating the lack of a rating as more similar to disliking the movie than liking it.

	Harry Potter I	Harry Potter I	Harry Potter I	Twilight	Star Wars 1	Star Wars 2	Star Wars 3
Sriya	4			5	1		
Shruti	5	5	4				
John				2	4	5	
Tracy		3					3

The cosine of the angle between Sriya and Shruti is

$$\frac{4 \times 5}{\sqrt{4^2 + 5^2 + 1^2} \sqrt{5^2 + 5^2 + 4^2}} = 0.386$$

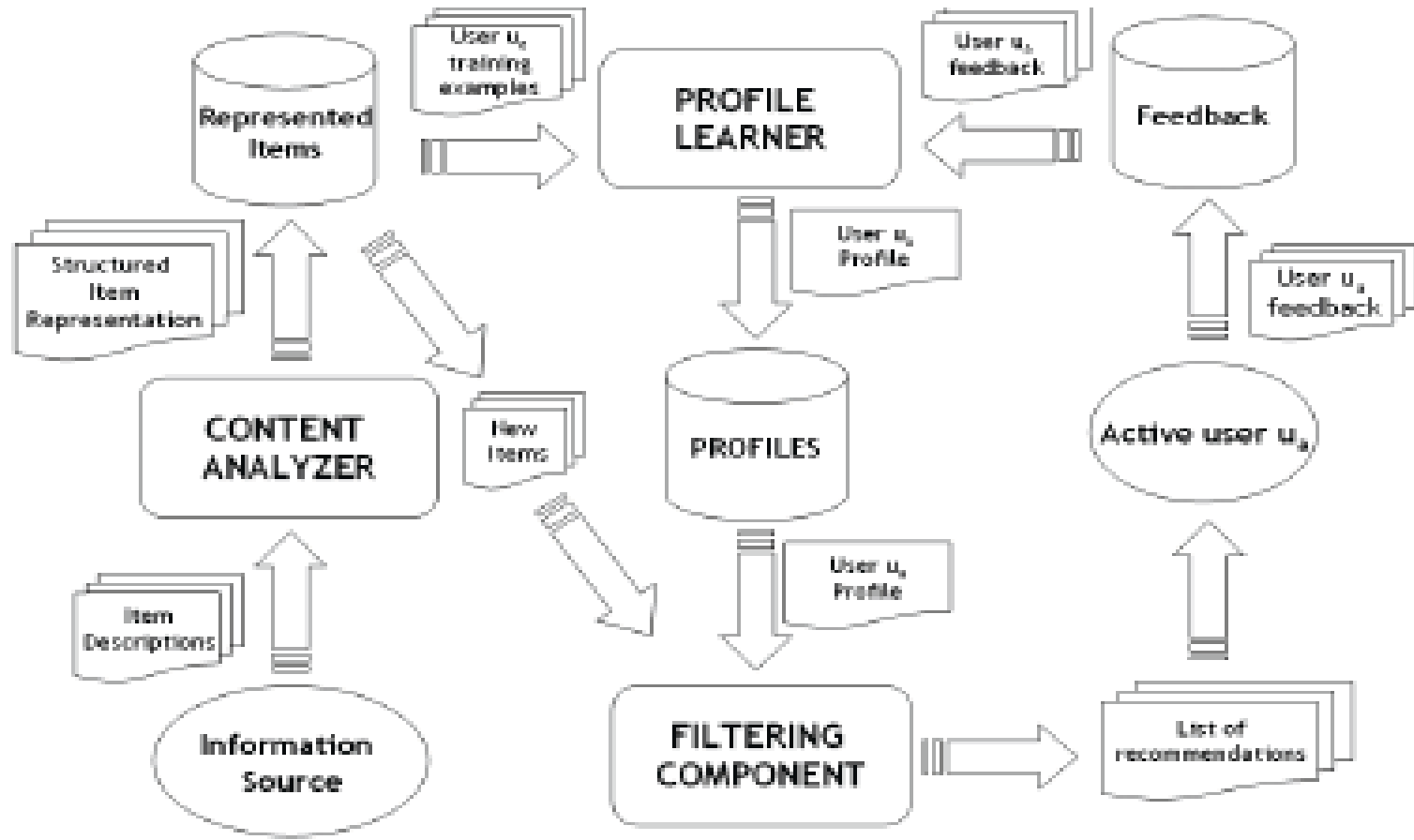
The cosine of the angle between Sriya and John is

$$\frac{5 \times 2 + 1 \times 4}{\sqrt{4^2 + 5^2 + 1^2} \sqrt{2^2 + 4^2 + 5^2}} = 0.322$$

Sriya & John are similar movie watching profiles.



High level architecture of a Content-based Recommender



Source:

<http://www.ics.uci.edu/~welling/teaching/CS77Bwinter12/ComputerEngineeringandJosef>

Chandrasekar Vuppapapati, Professor, MS Software Engineering,
Computer Engineering and Jose State University.

In-Class Exercise

Three computers, A , B , and C , have the numerical features listed below:

Feature	A	B	C
Processor Speed	3.06	2.68	2.92
Disk Size	500	320	640
Main-Memory Size	6	4	6

We may imagine these values as defining a vector for each computer; for instance, A 's vector is $[3.06, 500, 6]$. We can compute the cosine distance between any two of the vectors, but if we do not scale the components, then the disk size will dominate and make differences in the other components essentially invisible. Let us use 1 as the scale factor for processor speed, α for the disk size, and β for the main memory size.

- (a) In terms of α and β , compute the cosines of the angles between the vectors for each pair of the three computers.
- (b) What are the angles between the vectors if $\alpha = \beta = 1$?
- (c) What are the angles between the vectors if $\alpha = 0.01$ and $\beta = 0.5$?



Disney Reveals New 'Star Wars' Details, 'Incredibles 2'

[In Your Spring, Adventure Inspirations](#)[Lost Writers De Theories About Series 10 Years](#)

ARTICLE

COMMENTS (1)

[CARS](#) [DISNEY](#) [FROZEN](#) [PIRATES OF THE CARIBBEAN](#) [ROBERT IGER](#) [THE INCREDIBLES](#)

Recommend 154

[Email](#) [Print](#)[f](#) [t](#) [g+](#) [in](#)

A A

By BEN FRITZ [CONNECT](#)

Scene from "The Incredibles" (2004). —Walt Disney/courtesy Everett Collection

Disney has a third "Cars" and a second "The Incredibles" movie in the works, but won't be releasing a fifth "Pirates of the Caribbean" anytime soon.

At Walt Disney Co.'s annual shareholders meeting held in Portland, Ore. Tuesday, chief executive **Robert Iger** confirmed the two sequels from his company's **Pixar Animation Studios** are in the works and will be

released sometime after 2015.

Asked about another sequel to the hit "Pirates" franchise starring Johnny Depp, Mr. Iger said that another is "in development," but "not officially greenlit." "Pirates of the Caribbean 5" had been scheduled for release in the summer of 2015, but Disney

Pay less to power your electric car



Discovering Features of a Document

There are other classes of items where it is not immediately apparent what the values of features should be. We shall consider two of them: document collections and images. Documents present special problems, and we shall discuss the technology for extracting features from documents in this section.

There are many kinds of documents for which a recommendation system can be useful. For example, there are many news articles published each day, and we cannot read all of them. A recommendation system can suggest articles on topics a user is interested in, but how can we distinguish among topics? Web pages are also a collection of documents. Can we suggest pages a user might want to see? Likewise, blogs could be recommended to interested users, if we could classify blogs by topics.

Unfortunately, these classes of documents do not tend to have readily available information giving features. A substitute that has been useful in practice is the identification of words that characterize the topic of a document.



Vector Space Model

The basic idea of the vector space model is the following: We represent a document and a query both as vectors in a high-dimensional space corresponding to all the keywords and use an appropriate similarity measure to compute the similarity between the query vector and the document vector. The similarity values can then be used for ranking documents.

“How do we tokenize text?” The first step in most retrieval systems is to identify keywords for representing documents, a preprocessing step often called **tokenization**. To avoid indexing useless words, a text retrieval system often associates a *stop list* with a set of documents. A stop list is a set of words that are deemed “irrelevant.” For example, *a, the, of, for, with*, and so on are stop words, even though they may appear frequently. Stop lists may vary per document set. For example, *database systems* could be an important keyword in a newspaper. However, it may be considered as a stop word in a set of research papers presented in a database systems conference.

A group of different words may share the same word stem. A text retrieval system needs to identify groups of words where the words in a group are small syntactic variants of one another and collect only the common word stem per group. For example, the group of words *drug, drugged, and drugs*, share a common word stem, *drug*, and can be viewed as different occurrences of the same word.



Source:

Data Mining Concepts and Techniques, Jaiwei Han and Micheline Kamber
Chandrasekar Vuppapapati, Professor, MS Software Engineering,
Electrical Engineering, San Jose State University.

Vector Space Model

“How can we model a document to facilitate information retrieval?”

Starting with a set of d documents and a set of t terms, we can model each document as a vector v in the t dimensional space R^t , which is why this method is called the vector-space model.

Let the term frequency be the number of occurrences of term t in the document d , that is, $freq(d; t)$. The (weighted) **term-frequency matrix $TF(d; t)$ measures the association of a Term t with respect to the given document d** : it is generally defined as 0 if the document does not contain the term, and nonzero otherwise. There are many ways to define the term-weighting for the nonzero entries in such a vector.

For example, we can simply set $TF(d; t) = 1$ if the term t occurs in the document d , or use the term frequency $freq(d; t)$, or the relative term frequency, that is, the term frequency versus the total number of occurrences of all the terms in the document. There are also other ways to normalize the term frequency. For example, the Cornell SMART system uses the following formula to compute the (normalized) term frequency:

$$TF(d, t) = \begin{cases} 0 & \text{if } freq(d, t) = 0 \\ 1 + \log(1 + \log(freq(d, t))) & \text{otherwise.} \end{cases}$$



Source:

Data Mining Concepts and Techniques, Jaiwei Han and Micheline Kamber, 2011, Morgan Kaufmann Publishers, San Jose State University.

Chandrasekar Vuppapapati, Professor, MS Software Engineering,

San Jose State University.

Vector Space Model

Besides the term frequency measure, there is another important measure, called **inverse document frequency (IDF)**, that represents the scaling factor, or the importance, of a term t . **If a term t occurs in many documents, its importance will be scaled down due to its reduced discriminative power.** For example, the term *database systems* may likely be less important if it occurs in many research papers in a database system conference. According to the same Cornell SMART system, $IDF(t)$ is defined by the following formula:

$$IDF(t) = \log \frac{1 + |d|}{|d_t|},$$

where d is the document collection, and d_t is the set of documents containing term t . If $|d_t| \ll |d|$, the term t will have a large IDF scaling factor and vice versa.

In a complete vector-space model, TF and IDF are combined together, which forms the TF-IDF measure:

$$TF\text{-}IDF(d, t) = TF(d, t) \times IDF(t).$$



Source:

Data Mining Concepts and Techniques, Jaiwei Han and Micheline Kamber
Chandrasekar Vuppapapati, Professor, MS Software Engineering,
Electrical Engineering, San Jose State University.

TF-IDF Calculation

Calculate TF-IDF of a Document matrix:

A term frequency matrix showing the frequency of terms per document.

document/term	t_1	t_2	t_3	t_4	t_5	t_6	t_7
d_1	0	4	10	8	0	5	0
d_2	5	19	7	16	0	0	32
d_3	15	0	0	4	9	0	17
d_4	22	3	12	0	5	15	0
d_5	0	7	0	9	2	4	12



Source:

Data Mining Concepts and Techniques, Jaiwei Han and Micheline Kamber, 4th Edition, Morgan Kaufmann, 2011.
Chandrasekar Vuppapapati, Professor, MS Software Engineering,
Department of Computer Engineering, San Jose State University.

TF-IDF Calculation

shows a term frequency matrix where each row represents a document vector, each column represents a term, and each entry registers $freq(di; t_j)$, the number of occurrences of term t_j in document di . Based on this table we can calculate the TF-IDF value of a term in a document. For example, for t_6 in d_4 , we have

$$TF(d_4, t_6) = 1 + \log(1 + \log(15)) = 1.3377$$

$$IDF(t_6) = \log \frac{1 + 5}{3} = 0.301.$$

Therefore,

$$TF-IDF(d_4, t_6) = 1.3377 \times 0.301 = 0.403$$

“How can we determine if two documents are similar?”



Source:

Data Mining Concepts and Techniques, Jaiwei Han and Micheline Kamber
Chandrasekar Vuppalaapati, Professor, MS Software Engineering,
Electrical Engineering, San Jose State University.

TF-IDF Calculation – In class

Calculate TF-IDF for (d_1, t_3) and (d_6, t_7)

	t_1	t_2	t_3	t_4	t_5	t_6	t_7
d_1	0	40	5	9	1	0	0
d_2	0	0	10	11	12	0	0
d_3	1	2	3	29	0	0	1
d_4	20	12	11	10	9	5	1
d_5	12	10	22	43	9	12	10
d_6	10	10	100	100	10	10	1

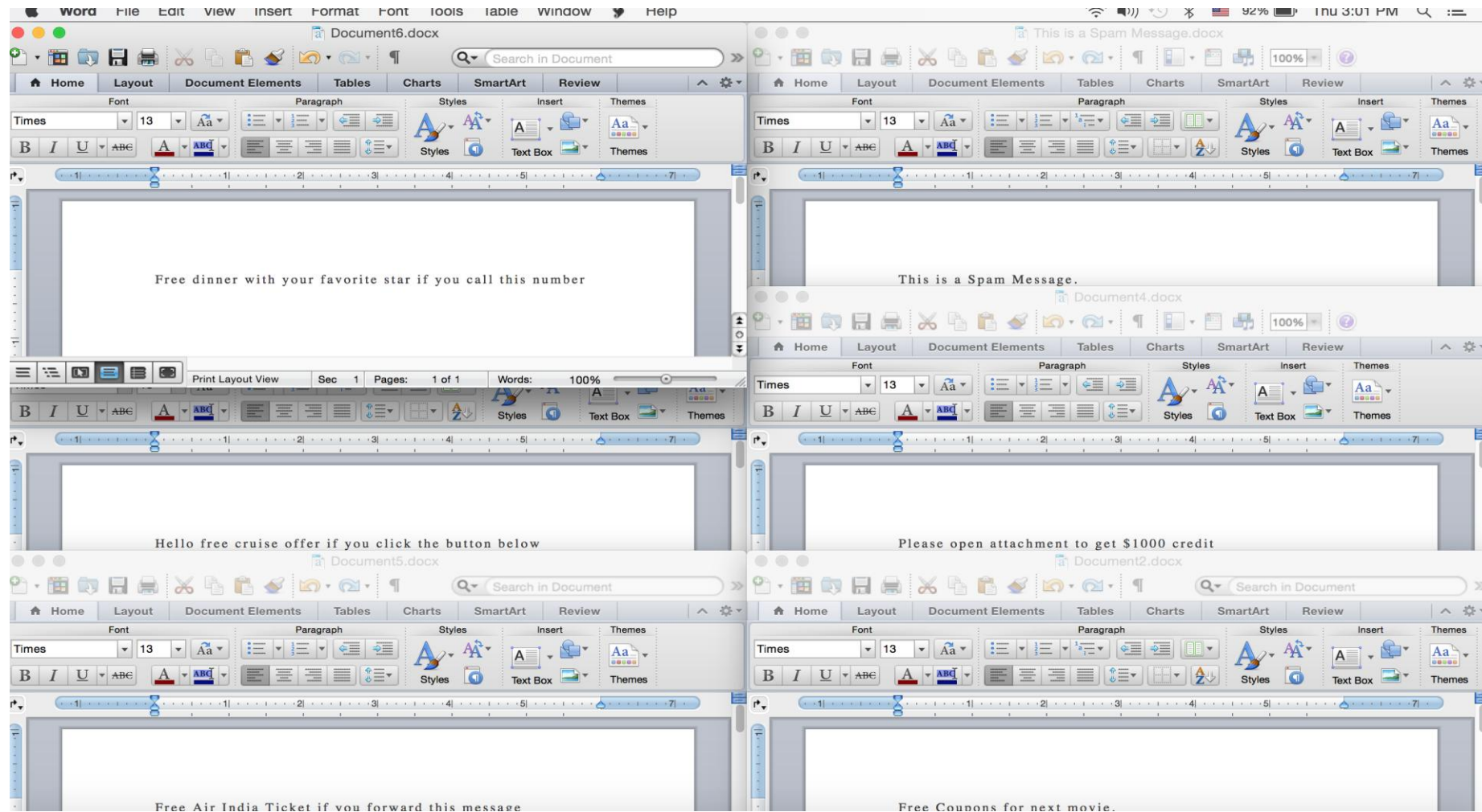


Source:

Data Mining Concepts and Techniques, Jaiwei Han and Micheline Kamber
Chandrasekar Vuppalapati, Professor, MS Software Engineering,
Electrical Engineering, San Jose State University.

Home Assignment – Spam Filter

Calculate TF-IDF of a Document matrix:



Vector Space Model

```
1 Train Document Set:
2
3 d1: The sky is blue.
4 d2: The sun is bright.
5
6 Test Document Set:
7
8 d3: The sun in the sky is bright.
9 d4: We can see the shining sun, the bright sun.
```

The first step in modeling the document into a vector space is to create a dictionary of terms present in documents. To do that, you can simply select all terms from the document and convert it to a dimension in the vector space, but we know that there are some kind of words (stop words) that are present in almost all documents, and what we're doing is extracting important features from documents, features do identify them among other similar documents, so using terms like "the, is, at, on", etc.. isn't going to help us, so in the information extraction, we'll just ignore them.



Vector Space Model

Now, what we have to do is to create a index vocabulary (dictionary) of the words of the train document set, using the documents and from the document set, using documents d1 and d2 from the document set, we'll have the following index vocabulary denoted as E where the t is the term:

$$E(t) = \begin{cases} 1, & \text{if } t \text{ is "blue"} \\ 2, & \text{if } t \text{ is "sun"} \\ 3, & \text{if } t \text{ is "bright"} \\ 4, & \text{if } t \text{ is "sky"} \end{cases}$$

Note that the terms like “is” and “the” were ignored as cited before. Now that we have an index vocabulary, we can convert the test document set into a vector space where each term of the vector is indexed as our index vocabulary, so the first term of the vector represents the “blue” term of our vocabulary, the second represents “sun” and so on. **Now, we're going to use the term-frequency to represent each term in our vector space; the term-frequency is nothing more than a measure of how many times the terms present in our vocabulary $E(t)$ are present in the documents d3 or d4**



Vector Space Model

we define the term-frequency as a counting function:

$$tf(t, d) = \sum_{x \in d} fr(x, t)$$

where the $fr(x, t)$ is a simple function defined as:

$$fr(x, t) = \begin{cases} 1, & \text{if } x = t \\ 0, & \text{otherwise} \end{cases}$$



Vector Space Model

So, what the $tf(t, d)$ returns is how many times is the term t is present in the document d . An example of this, could be $tf(\text{"sun"}, d_4) = 2$ since we have only two occurrences of the term "sun" in the document d_4 . Now you understood how the term-frequency works, we can go on into the creation of the document vector, which is represented by:

$$\vec{v}_{d_n} = (tf(t_1, d_n), tf(t_2, d_n), tf(t_3, d_n), \dots, tf(t_n, d_n))$$

Each dimension of the document vector is represented by the term of the vocabulary, for example, the $tf(t_1, d_2)$ represents the frequency-term of the term 1 or t_1 (which is our "blue" term of the vocabulary) in the document d_2 .

Let's now show a concrete example of how the documents d_3 and d_4 are represented as vectors:

$$\begin{aligned}\vec{v}_{d_3} &= (tf(t_1, d_3), tf(t_2, d_3), tf(t_3, d_3), \dots, tf(t_n, d_3)) \\ \vec{v}_{d_4} &= (tf(t_1, d_4), tf(t_2, d_4), tf(t_3, d_4), \dots, tf(t_n, d_4))\end{aligned}$$



Vector Space Model

Let's now show a concrete example of how the documents are represented as vectors:

```
1 Train Document Set:
2
3 d1: The sky is blue.
4 d2: The sun is bright.
5
6 Test Document Set:
7
8 d3: The sun in the sky is bright.
9 d4: We can see the shining sun, the bright sun.
```

Document

$$V_{d3} = (0,1,1,1)$$
$$V_{d4} = (0,2,1,0)$$

The resulting vector V_d shows that we have, in order, 0 occurrences of the term "blue", 1 occurrence of the term "sun", and so on. In the , we have 0 occurrences of the term "blue", 2 occurrences of the term "sun", etc.

index vocabulary
(dictionary)

$$E(t) = \begin{cases} 1, & \text{if } t \text{ is "blue"} \\ 2, & \text{if } t \text{ is "sun"} \\ 3, & \text{if } t \text{ is "bright"} \\ 4, & \text{if } t \text{ is "sky"} \end{cases}$$

Source: <http://pyevolve.sourceforge.net/wordpChandrasekhar>

Uppalapati, Professor, N
Computer Engineering, San Jose State University.



Vector Space Model

$$M_{|D| \times F} = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 0 & 2 & 1 & 0 \end{bmatrix}$$



In-Class Assignment

Develop Vector Space Model?

d1: User selected Wedding gown.
d2: User ordered on-line rose flowers.
d3: User searched diamond ring.
d4: User selected white wedding gown, online flowers, 3 carat diamond ring.



Patents – Item based Recommendation System & TFIDF

Patent	Description
<u>US 8266003 B1</u>	Data feed item classification
<u>US 8510307 B1</u>	Systems and methods for automatic item classification
<u>US 8489603 B1</u>	Automatic item categorizer
<u>US 8838618 B1</u>	System and method for identifying feature phrases in item description information



Patents – Item based Recommendation System & TFIDF

How we do the identification:

First, eliminate stop words – the several hundred most common words, which tend to say little about the topic of a document. For the remaining words, compute the TF.IDF score for each word in the document. The ones with the highest scores are the words that characterize the document.

We may then take as the features of a document the n words with the highest TF.IDF scores. It is possible to pick n to be the same for all documents, or to let n be a fixed percentage of the words in the document. We could also choose to make all words whose TF.IDF scores are above a given threshold to be a part of the feature set.

Now, documents are represented by sets of words. Intuitively, we expect these words to express the subjects or main ideas of the document. For example, in a news article, we would expect the words with the highest TF.IDF score to include the names of people discussed in the article, unusual properties of the event described, and the location of the event. To measure the similarity of two documents, there are several natural distance measures we can use:

1. We could use the Jaccard distance between the sets of words
2. We could use the cosine distance between the sets, treated as vectors.



Cosine Distance

The cosine distance makes sense in spaces that have dimensions, including Euclidean spaces and discrete versions of Euclidean spaces, such as spaces where points are vectors with integer components or boolean (0 or 1) components. In such a space, points may be thought of as directions. We do not distinguish between a vector and a multiple of that vector. Then the cosine distance between two points is the angle that the vectors to those points make. This angle will be in the range 0 to 180 degrees, regardless of how many dimensions the space has.

We can calculate the cosine distance by first computing the cosine of the angle, and then applying the arc-cosine function to translate to an angle in the 0-180 degree range. Given two vectors x and y , the cosine of the angle between them is the dot product $x \cdot y$ divided by the L2-norms of x and y (i.e., their Euclidean distances from the origin). Recall that the dot product of vectors

$$[x_1, x_2, \dots, x_n] \cdot [y_1, y_2, \dots, y_n] \text{ is } \sum_{i=1}^n x_i y_i.$$

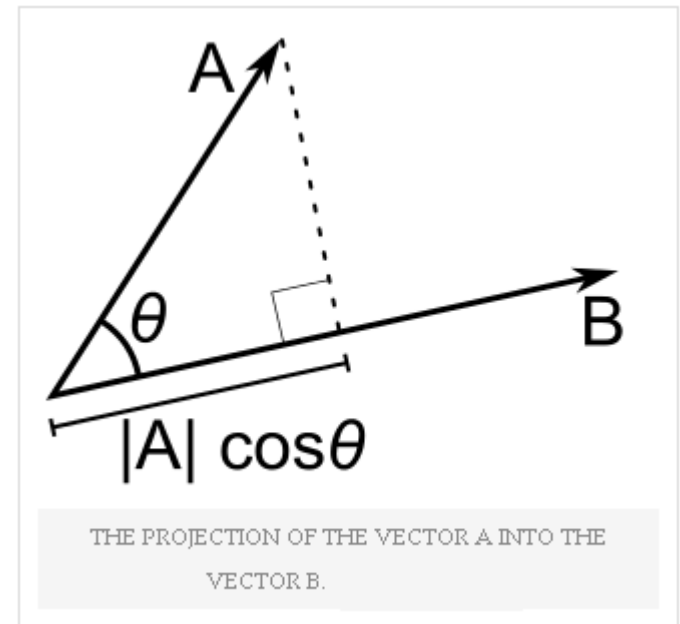


Cosine Distance

The cosine similarity between two vectors (or two documents on the Vector Space) is a measure that calculates the cosine of the angle between them. This metric is a measurement of orientation and not magnitude, it can be seen as a comparison between documents on a normalized space because we're not taking into the consideration only the magnitude of each word count (Text Feature Extraction (tf-idf)) of each document, but the angle between the documents. What we have to do to build the cosine similarity equation is to solve the equation of the dot product for the

$$\vec{a} \cdot \vec{b} = \|\vec{a}\| \|\vec{b}\| \cos \theta$$
$$\cos \theta = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|}$$

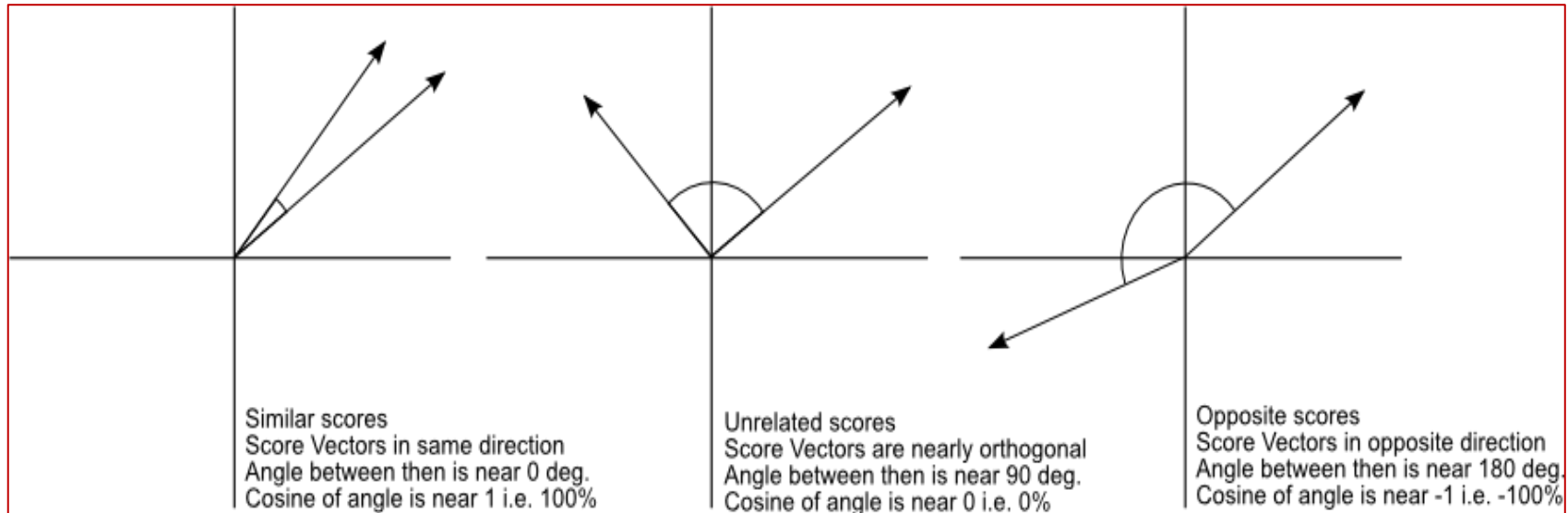
α	0°	30°	45°	60°	90°
$\sin \alpha$	0	$\frac{1}{2}$	$\frac{1}{2} \sqrt{2}$	$\frac{1}{2} \sqrt{3}$	1
$\cos \alpha$	1	$\frac{1}{2} \sqrt{3}$	$\frac{1}{2} \sqrt{2}$	$\frac{1}{2}$	0



Source: <http://pyevolve.sourceforge.net/wordpress/?p=2497>

Cosine Distance

And that is it, this is the cosine similarity formula. Cosine Similarity will generate a metric that says how related are two documents by looking at the angle instead of magnitude, like in the examples below:



THE COSINE SIMILARITY VALUES FOR DIFFERENT DOCUMENTS, 1 (SAME DIRECTION), 0 (90 DEG.), -1 (OPPOSITE DIRECTIONS).

Note that even if we had a vector pointing to a point far from another vector, they still could have a small angle and that is the central point on the use of Cosine Similarity, the measurement tends to ignore the higher term count on documents. Suppose we have a document with the word “sky” appearing 200 times and another document with the word “sky” appearing 50, the Euclidean distance between them will be higher but the angle will still be small because they are pointing to the same direction, which is what matters when we are comparing documents.

Cosine Similarity

Suppose the only features of movies are the set of actors and the average rating. Consider two movies with five actors each. Two of the actors are in both movies. Also, one movie has an average rating of 3 and the other an average of 4. The vectors look something like

$$\begin{array}{cccccccc} 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 3\alpha \\ 1 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 4\alpha \end{array}$$

Calculate Cosine Similarity?

$$\text{Dot Product} = (0 \times 1) + (1 \times 1) + (1 \times 0) + (0 \times 1) + (1 \times 0) + (1 \times 1) + (0 \times 1) + (1 \times 0) + (3\alpha \times 4\alpha)$$

$$\text{Dot Product} = 2 + 12\alpha^2$$

$$\text{Lengths of the Vectors} = \sqrt{5 + 9\alpha^2} \text{ and } \sqrt{5 + 16\alpha^2}.$$

$$\text{Cosine Similarity} = \frac{2 + 12\alpha^2}{\sqrt{25 + 125\alpha^2 + 144\alpha^4}}$$

If we choose $\alpha = 1$, that is, we take the average ratings as they are, then the value of the above expression is 0.816. If we use $\alpha = 2$, that is, we double the ratings, then the cosine is 0.940. That is, the vectors appear much closer in direction than if we use $\alpha = 1$. Likewise, if we use $\alpha = 1/2$, then the cosine is 0.619, making the vectors look quite different. We cannot tell which value of α is “right,” but we see that the choice of scaling factor for numerical features affects our decision about how similar items are.



Classification Algorithms

A completely different approach to a recommendation system using item profiles and utility matrices is to treat the problem as one of machine learning. Regard the given data as a training set, and for each user, build a classifier that predicts the rating of all items. There are a great number of different classifiers, and it is not our purpose to teach this subject here. However, you should be aware of the option of developing a classifier for recommendation, so we shall discuss one common classifier – decision trees – briefly.

A decision tree is a collection of nodes, arranged as a binary tree. The leaves render decisions; in our case, the decision would be “likes” or “doesn’t like.” Each interior node is a condition on the objects being classified; in our case the condition would be a predicate involving one or more features of an item.



Classification Algorithms

User Profiles - Who is similar?

	Harry Potter I	Harry Potter II	Harry Potter III	Twilight	Star Wars 1	Star Wars 2	Star Wars 3
Sriya	4			5	1		
Shruti	5	5	4				
John				2	4	5	
Tracy		3					3



Classification Algorithms

We can treat blanks as a 0 value. This choice is questionable, since it has the effect of treating the lack of a rating as more similar to disliking the movie than liking it.

	Harry Potter I	Harry Potter I	Harry Potter I	Twilight	Star Wars 1	Star Wars 2	Star Wars 3
Sriya	4			5	1		
Shruti	5	5	4				
John				2	4	5	
Tracy		3					3

The cosine of the angle between Sriya and Shruti is

$$\frac{4 \times 5}{\sqrt{4^2 + 5^2 + 1^2} \sqrt{5^2 + 5^2 + 4^2}} = 0.386$$

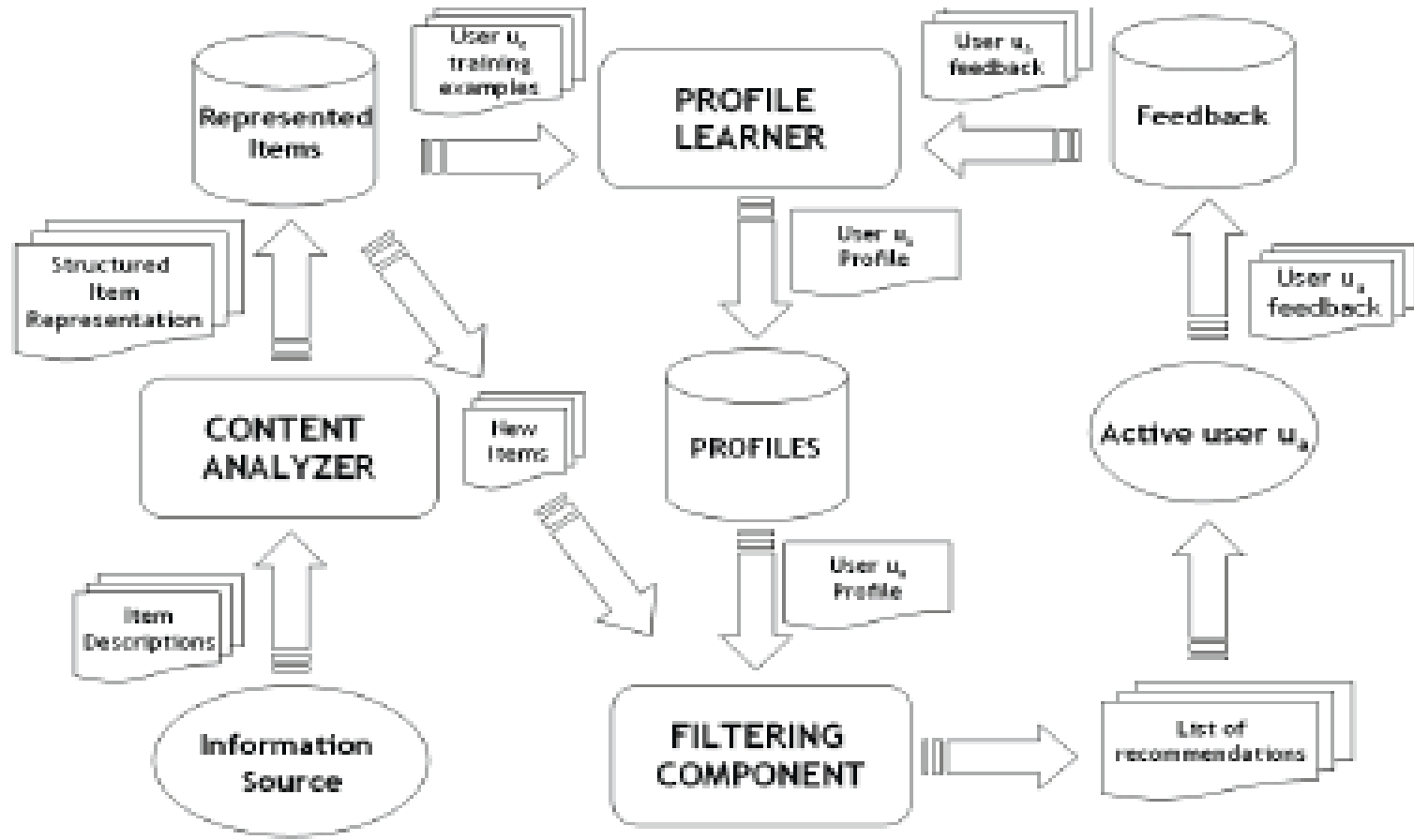
The cosine of the angle between Sriya and John is

$$\frac{5 \times 2 + 1 \times 4}{\sqrt{4^2 + 5^2 + 1^2} \sqrt{2^2 + 4^2 + 5^2}} = 0.322$$

Sriya & Shruti are similar movie watching profiles.



High level architecture of a Content-based Recommender



Source:

<http://www.ics.uci.edu/~welling/teaching/CS77Bwinter12/ComputerEngineeringandJosef>

Chandrasekar Vuppapapati, Professor, MS Software Engineering,
Computer Engineering and Jose State University.

In-Class Exercise

Three computers, A , B , and C , have the numerical features listed below:

Feature	A	B	C
Processor Speed	3.06	2.68	2.92
Disk Size	500	320	640
Main-Memory Size	6	4	6

We may imagine these values as defining a vector for each computer; for instance, A 's vector is $[3.06, 500, 6]$. We can compute the cosine distance between any two of the vectors, but if we do not scale the components, then the disk size will dominate and make differences in the other components essentially invisible. Let us use 1 as the scale factor for processor speed, α for the disk size, and β for the main memory size.

- (a) In terms of α and β , compute the cosines of the angles between the vectors for each pair of the three computers.
- (b) What are the angles between the vectors if $\alpha = \beta = 1$?
- (c) What are the angles between the vectors if $\alpha = 0.01$ and $\beta = 0.5$?



References

- Mining of Massive Data Sets, <http://infolab.stanford.edu/~ullman/mmds/book.pdf>
- <http://hbr.org/download/locating-your-next-strategic-opportunity/F1103Z-PDF-ENG/F1103Z-PDF-ENG.PDF>
- <http://infolab.stanford.edu/~ullman/mmds.html>
- http://home.deib.polimi.it/matteucc/Clustering/tutorial_html/hierarchical.html
- Content-based Recommender Systems: State of the Art and Trends, <http://www.ics.uci.edu/~welling/teaching/CS77Bwinter12/handbook/ContentBasedRS.pdf>
- Content-based Recommender Systems: State of the Art and Trends, <http://www.ics.uci.edu/~welling/teaching/CS77Bwinter12/handbook/ContentBasedRS.pdf>



An item categorization service is described that automatically categorizes items of interest to a user. The user may possess an item that they wish to offer for sale using a network-based service. The user may submit item information to the categorization service to categorize the item of interest. Upon receipt, the categorization service may assess the relevance of the item information to hierarchically organized categories maintained by the network-based service. Categories having the highest relevance may be identified as first category candidates. The deepest common ancestor of the first category candidates may be identified the first category. One or more categories related to the first category may also be identified and subjected to relevance assessment. Those related categories having the highest relevance may be identified as second category candidates. The deepest common ancestor of the second category candidates may be identified as a second category for the item of interest.



Recommendation System Patents For Reference

* Not all – Amazon Patents



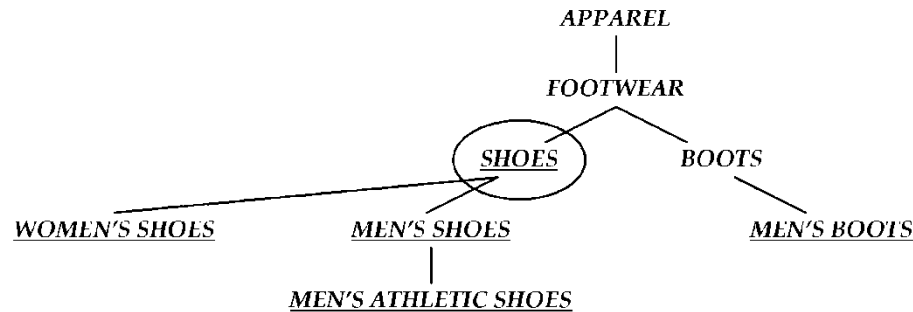


Fig.2A.

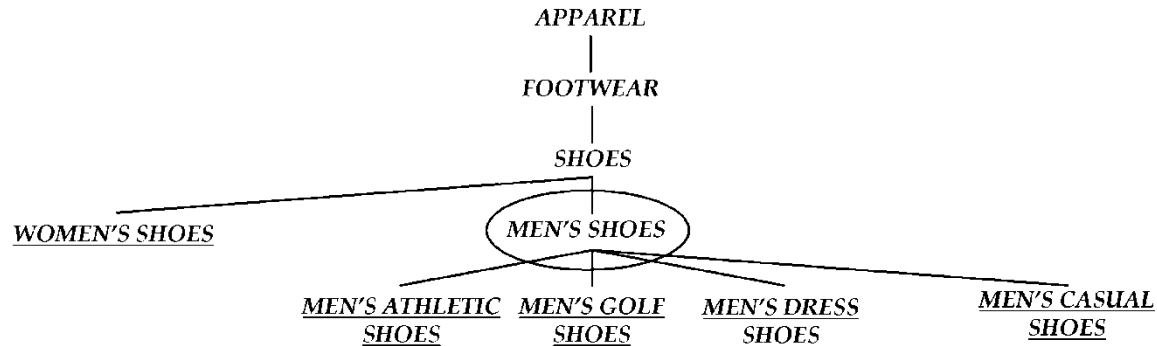


Fig.2B.



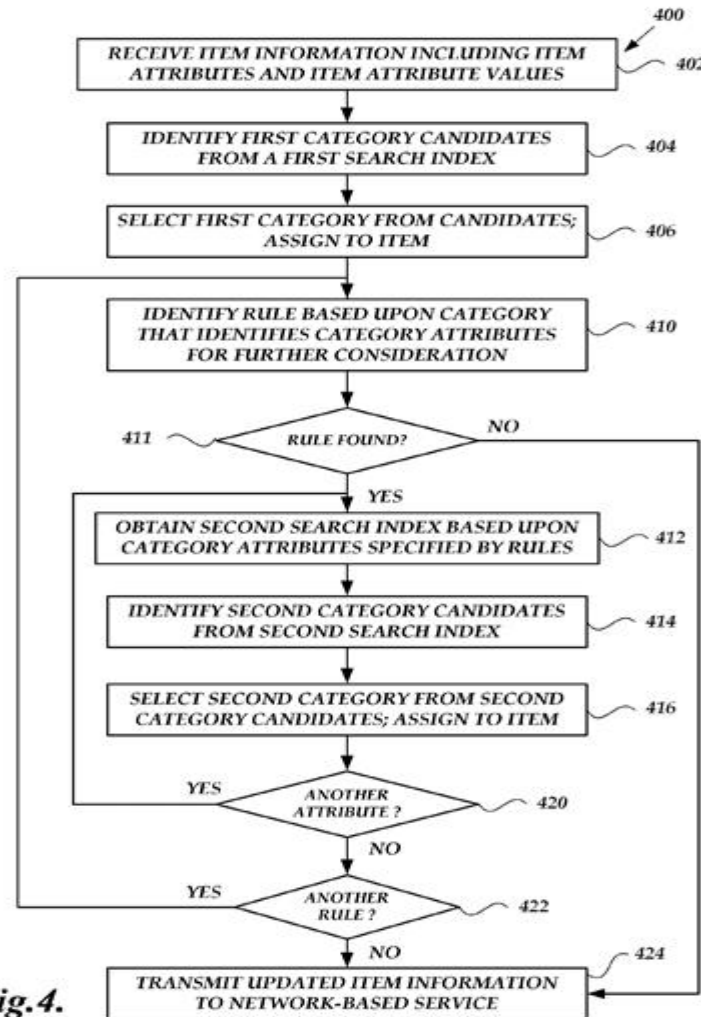


Fig.4.



Abstract:

A categorization service is described that automatically categorizes items of interest to a user. The user may possess an item that they wish to offer for sale using the network-based service. The user may prepare an electronic description of the item and submit this item information to a categorization service for categorization. Upon receipt, the categorization service may compare the item information to item category descriptions representing item categories of the network-based server in order to determine category recommendations for the item of interest. The recommended categories are returned to the user to enable the user to assign a category, selected from the category recommendations, to the item of interest.



Source:

[https://www.google.com/patents/US8489603?dq=inassignee+\"Chandrasekar Vuppapalapati, Professor, MS Software Engineering, Computer Engineering, San Jose State University\"+\"item+category+recommendation+%2B+TF-IDF&hl=en&sa=X&ei=WSU-vIXwMuK7igLM7YG4BA&ved=0CCYQ6AEwAQ](https://www.google.com/patents/US8489603?dq=inassignee+\)

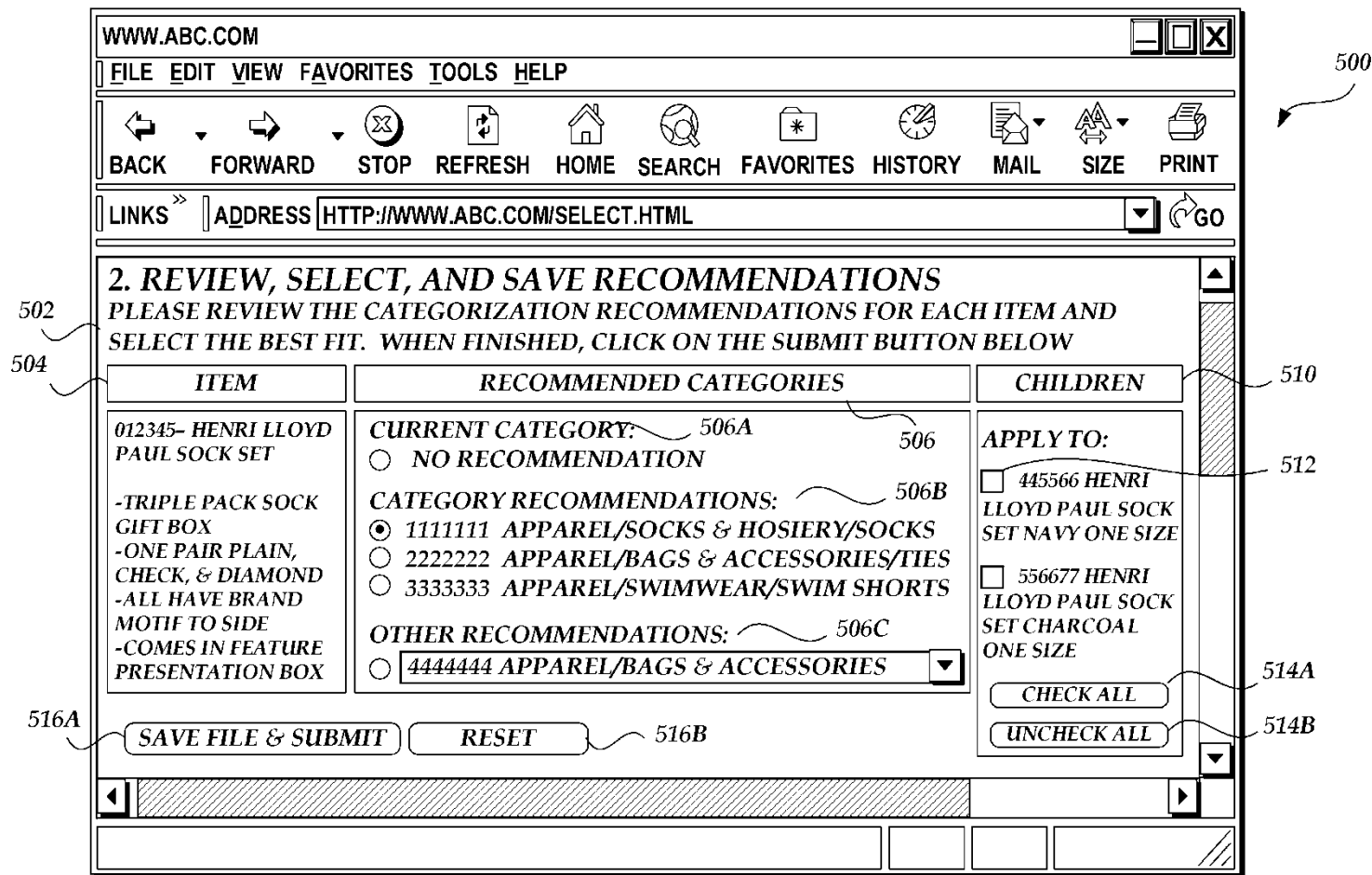


Fig.5



Background:

Generally described, computing devices and communication networks facilitate network-based commerce. For example, a user may employ his or her computing device to access a network-based retailer for the purchase of items and services (collectively and individually referred to as “items”) such as music, books, and electronics, just to name a few.

To take advantage of this growing marketplace, sellers frequently elect to sell items through network-based retailers. For example, a book-seller may elect to sell books through a network-based retailer which offers books for sale. Accordingly, a seller may record some information regarding items they wish to offer for sale in an electronic form, such as a written description. This recorded item information may be provided to the network-based retailer to enable the item to be offered for sale by the network-based retailer.



Source:

<https://www.google.com/patents/US8489603?dq=inassignee%20computer%20engineering%20san%20jose%20state%20university&hl=en&sa=X&ei=WSU-viXwWuK7igLM7YG4BA&ved=0CCYQ6AEwAQ>

Detailed Description:

Generally described, embodiments of the present disclosure relate to automatically categorizing items of interest to user (e.g., a seller of the item, such a manufacturer, wholesaler, or retailer) based upon associated item information. The items of interest may be categorized in accordance with item categories maintained by a network-based service (e.g., a web site).

In this regard, a categorization service is described that categorizes the items of interest on behalf of the user. For example, a user employing a user computing device may possess an item that they wish to offer for sale using the network-based service. The item may be anything capable of being offered for sale, for example, any good, service, or information. Accordingly, the network-based service may include network-based retailers such as those which sell items for purchase to the customer, network-based sources of entertainment and/or information (e.g., network-based encyclopedias, media sharing, etc), network-based social networking services which enable users to share content with one another, and the like.



Source:

<https://www.google.com/patents/US8489603?dq=inassigned+Computer+Engineering+San+Jose+State+University+TF-IDF&hl=en&sa=X&ei=WSU-viXwMuK7igLM7YG4BA&ved=0CCYQ6AEwAQ>

Detailed Description:

Upon receipt of the item information, the categorization service analyzes the item information and compares the item information to item categories descriptions representing item categories of a selected network-based server in order to determine category recommendations for the items of interest represented by the item information. The categorization service may receive the item information directly from the user computing device or via the network-based service. For example, the categorization service may generate a mathematical description of the item information and compare this to a mathematical description of the item category descriptions.

Based upon these mathematical descriptions, **the degree of similarity between the item information and item category descriptions may be established. Categories having item category descriptions that are determined to be sufficiently similar to the submitted item information may be determined to be category recommendations. Categories having item category descriptions that are determined to lack sufficient similarity with the item information may be ignored as category recommendations.**



Source:

<https://www.google.com/patents/US8489603?dq=inassignee+computer+engineering+san+jose+state+university+documentation+%2B+TF-IDF&hl=en&sa=X&ei=WSU-viXwWuK7igLM7YG4BA&ved=0CCYQ6AEwAQ>

Abstract

Embodiments may include, for each item in a subset of items from a larger group of items, evaluating item description information about that item to identify a respective set of candidate phrases to be evaluated. Embodiments may also include, for each phrase in the sets of candidate phrases, generating multiple component scores based on one or more of the frequency with which that phrase occurs in the item description information for the subset of items and/or the frequency with which that phrase occurs in a corpus of item description information for the overall group of items. Embodiments may also include, for each phrase in the sets of candidate phrases, generating a respective phrase score based on the component scores generated for that phrase. Embodiments may include, based on phrase scores, selecting a subset of phrases from the sets of candidate phrases as being feature phrases for the subset of items.



Background

Merchants that operate network-accessible marketplaces may maintain electronic catalogs that include thousands of items (or more) offered for sale. These electronic catalogs often include item detail pages accessible through one or more networks (e.g., web pages accessible over the Internet). These item detail pages often include descriptive information (e.g., text descriptions, pictures or video) in order to assist a buyer in determining whether an item is worth purchasing. In many cases, this descriptive information may be based on information from manufacturers or suppliers of the items offered for sale. In some cases, different manufacturers and suppliers may provide the descriptive information according to different structured formats. The merchant may synthesize this information into lists of features that are useful for categorizing and/or searching for items. As feature sets may vary widely among different product types, identifying these features is not an insignificant undertaking in some cases, particularly for large electronic catalogs with many different items.



ETAILED DESCRIPTION OF EMBODIMENTS Introduction

Various embodiments of a system and method for identifying feature phrases in item description information are described. Generally, item description information may include any information that is descriptive of an item, such as an item offered in commerce (e.g., for sale or trade). For example, a merchant that offers items for sale may receive item description information from suppliers or manufacturers. For clarity, portions of this description may refer to suppliers alone in some instances. It should be understood that these portions also apply to item manufacturers. In some cases, at least a portion of the item description information may adhere to a structured format, such as a format including structured fields for an item's name, brand, color, product type, weight or other common characteristics. For instance, these characteristics may include general characteristics that are largely applicable to any item offered for sale. For example, in almost all cases, any given item may have a name, brand and product type. Similarly, physical items (as opposed to electronic content, such as downloadable music) will in most cases have physical dimensions and weight. In some cases, a supplier may provide item description information to the merchant when the merchant orders supply of items from the supplier.

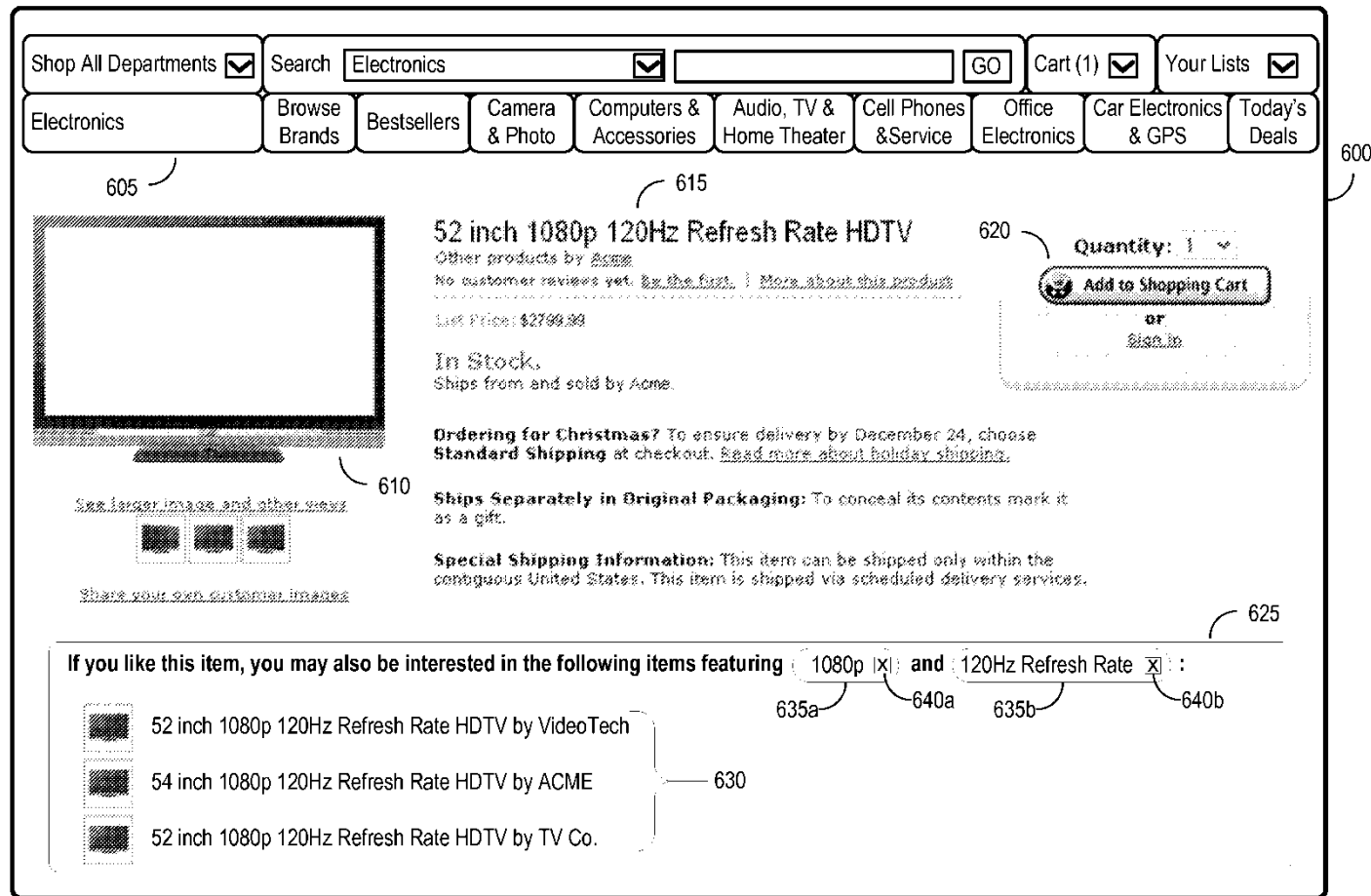


While some item description information may include item characteristics that are easily identifiable due to the use of the aforesaid structured fields, item description information may include useful item features that are not explicitly enumerated by the item description information. This information may be found both within and outside of structured fields. For instance, item description information may include a miscellaneous field that includes a textual description of a respective item. For instance, such information might include a few paragraphs of marketing literature that describes a product's features. In other cases, an existing field (e.g., a field that specifies a product name) may specify one or more features for which there is not a structured field. For instance, consider the non-limiting example product name “ACME 52 inch 1080P HDTV with 120 Hz Refresh Rate.” While this information may be enumerated as a product name, this information also includes phrases that convey features of the product, such as a specific display resolution (e.g., 1080P) and a specific refresh rate (e.g., 120 Hz). In various embodiments, the presence of these features may be applicable for some sub-portion of a larger group of items, such as a particular type of item in a product catalog that include many different types of items. For instance, while display resolutions and refresh rates may be pertinent to televisions and other electronic displays, they may not be relevant to apparel items, such as shirts or pants. In many cases, a merchant may have numerous different types of items within its catalog. As item description information may not explicitly enumerate these types of feature phrases, embodiments may include techniques to identify feature phrases pertinent to different product types. In various embodiments, a given feature phrase may include one or more words that specify a feature or characteristic of an item, such as an item offered in commerce. For example, in various embodiments, a feature phrase may include one or more words (e.g., words from an item's name or product description) that differentiate that item from other similar items.



Source:

<https://www.google.com/patents/US8838618?dq=inassignee%20computer%20engineering%20recommendation+2B+TF-IDF&hl=en&sa=X&ei=WSU-viXwMuK7igLM7YG4BA&ved=0CDQQ6AEwAw>



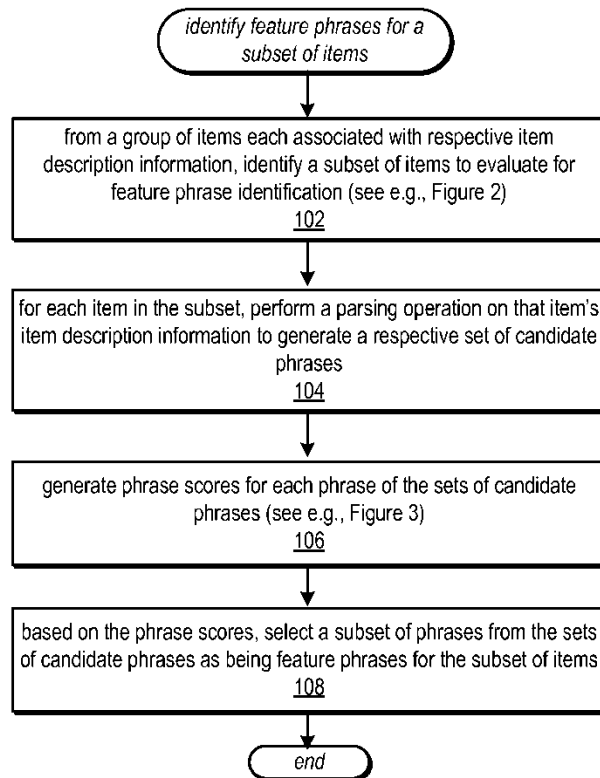


FIG. 1

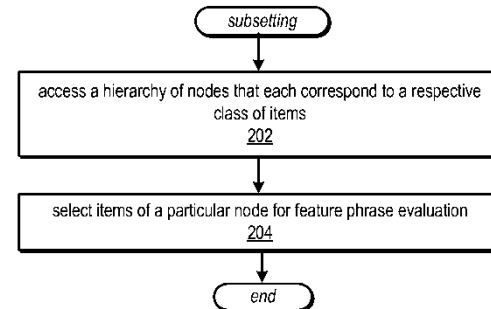


FIG. 2

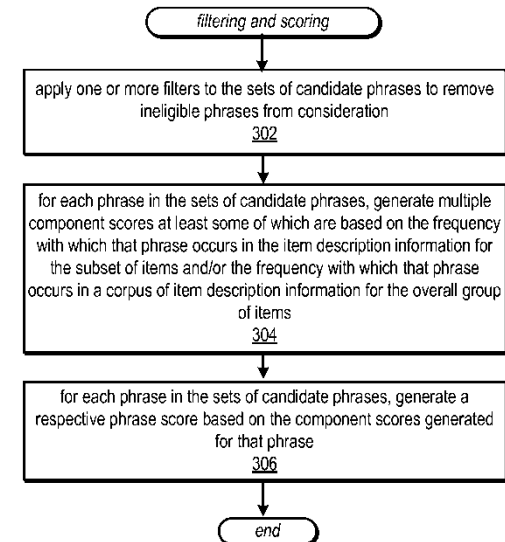


FIG. 3



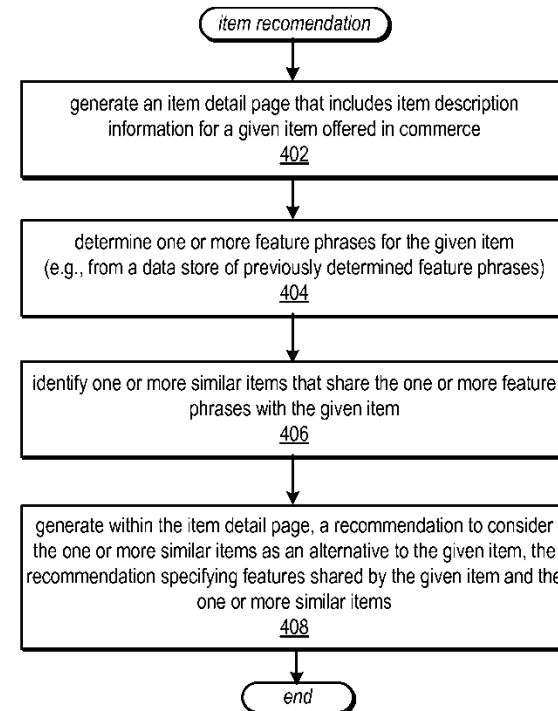


FIG. 4

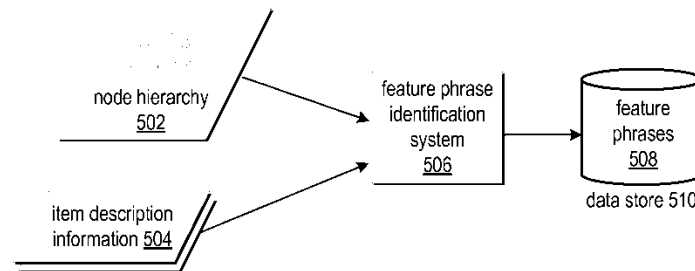


FIG. 5

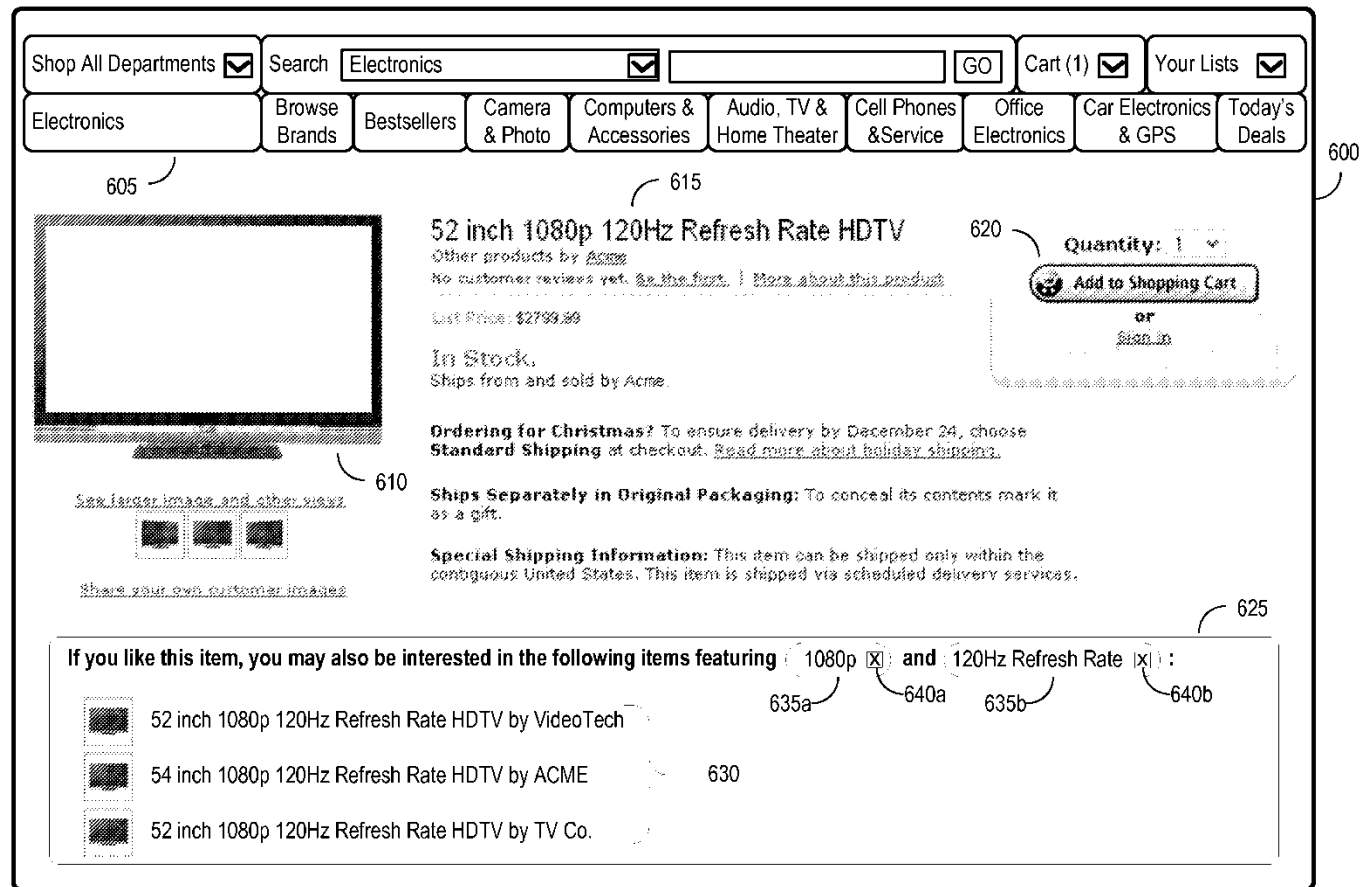


FIG. 6



References

- Mining of Massive Data Sets, <http://infolab.stanford.edu/~ullman/mmds/book.pdf>
- <http://hbr.org/download/locating-your-next-strategic-opportunity/F1103Z-PDF-ENG/F1103Z-PDF-ENG.PDF>
- <http://infolab.stanford.edu/~ullman/mmds.html>
- http://home.deib.polimi.it/matteucc/Clustering/tutorial_html/hierarchical.html
- Content-based Recommender Systems: State of the Art and Trends, <http://www.ics.uci.edu/~welling/teaching/CS77Bwinter12/handbook/ContentBasedRS.pdf>
- Content-based Recommender Systems: State of the Art and Trends, <http://www.ics.uci.edu/~welling/teaching/CS77Bwinter12/handbook/ContentBasedRS.pdf>

